

Università di Pisa

Dipartimento di Informatica

Master's Degree in Computer Science

Master's Thesis

Data Analysis and Data Mining on Decentralised Prediction Markets: the Price Process of Polymarket from One Minute to Twelve Hours

Supervisor

Prof. Davide Bacciu

Co-supervisor

Prof. Giulia Livieri
London School of Economics

Candidate

Vito Pampinella

Academic Year 2025/2026

Abstract

Polymarket is the largest decentralised prediction market: contracts on real-world events are traded on a central limit order book and settled on the Polygon blockchain, so that every order-book update and every trade is publicly observable. This thesis treats that venue as a data-analysis and data-mining object and asks whether, and how, the direction of its prices can be predicted over horizons from one minute to twelve hours.

The thesis first explains how a prediction market and Polymarket work, and introduces the notions of time series, order books, graphs, learners and evaluation that it uses. It then follows twenty-one days of order-book, on-chain and market data from collection, through preprocessing and audit, to a leakage-free three-class benchmark (down, unchanged, up), and explores them as a price process, a transaction graph and a feature space. The methodology sets the objective and the criteria of comparison and presents five families of approaches: causal rules on the price, stateless learners on hand-made features, a continuous-time stateful model of the order-book stream, graph models on the wallet–market transactions, and sources of information outside the book. The experiments compare these families on the same samples, with bootstrap intervals and a locked test period, family by family and then across horizons, kinds of market and seeds.

The main findings are that a market's own activity is its most predictable feature; that gradient-boosted trees on hand-made features are the strongest model from fifteen minutes to twelve hours; that the stateful model of the book stream adds a small but reliable gain at one minute, where the task is whether the price will move at all; and that the transaction graph and the sources outside the book add no reliable gain over what the book already shows. The conclusions state the limits of the study, what a deployment would need, and the future work.

Contents

Abstract **iii**

1 Introduction **1**
1.1 Context . . . 1
1.2 The problem and why it is difficult . . . 1
1.3 Research questions . . . 2
1.4 Approach . . . 2
1.5 Contributions . . . 2
1.6 Structure of the thesis . . . 3

2 Background **5**
2.1 Prediction markets . . . 5
2.2 Polymarket . . . 6
2.2.1 How Polymarket works . . . 6
2.2.2 Tokens, contracts and transactions: the blockchain layer . . . 6
2.2.3 The limit order book . . . 7
2.2.4 The life of a market . . . 7
2.2.5 What is known about Polymarket . . . 8
2.3 Order books, price series and the forecasting task . . . 8
2.4 Notation . . . 9
2.5 Learning a classifier . . . 11
2.5.1 The model as a conditional distribution . . . 11
2.5.2 Maximum likelihood and the cross-entropy loss . . . 12
2.5.3 Gradient descent and its stochastic version . . . 12
2.5.4 Backpropagation . . . 13
2.5.5 Class imbalance: weights, the focal loss and the decision rule . . . 13
2.5.6 Generalisation, overfitting and early stopping . . . 14
2.6 Neural networks for sequences . . . 15
2.6.1 The multilayer perceptron . . . 15
2.6.2 Recurrent networks and the gated recurrent unit . . . 15
2.6.3 Training through time . . . 16
2.6.4 Attention and the transformer encoder . . . 17
2.6.5 Continuous time: decay and multi-rate memories . . . 18
2.6.6 Time-series foundation models . . . 19
2.7 Graphs and graph neural networks . . . 19
2.7.1 Graphs, bipartite graphs, dynamic graphs . . . 19
2.7.2 Message passing . . . 20
2.7.3 Dynamic graphs: memories and messages in time . . . 21
2.8 Gradient-boosted trees and neural tabular learners . . . 21
2.8.1 Decision trees . . . 22

2.8.2 Gradient boosting . . . 22
2.8.3 Neural tabular learners . . . 23
2.9 Evaluation under imbalance and dependence . . . 24

3 The Dataset 25
3.1 The four tables . . . 25
3.2 Collection . . . 26
3.2.1 Market metadata: the Gamma API . . . 26
3.2.2 The order-book archive . . . 26
3.2.3 The on-chain transactions: a blockchain scraper . . . 26
3.3 Scale, coverage and storage . . . 30
3.4 Data quality . . . 30
3.5 Preprocessing . . . 30
3.5.1 Time axis: blocks and the synchronisation of the two streams . . . 31
3.5.2 Derived tensors . . . 31
3.6 The final dataset: labels, samples, features and periods . . . 31
3.6.1 Labels . . . 32
3.6.2 Feature engineering . . . 32
3.6.3 Training, validation and test periods . . . 33
3.7 Exploratory and statistical analysis . . . 34
3.7.1 The price process . . . 34
3.7.2 The transactions as a graph . . . 37
3.7.3 The structure of the feature space . . . 42
3.7.4 What the data say before a model is trained . . . 51

4 Methodology 55
4.1 Objective, criteria and rules of inclusion . . . 55
4.1.1 Objective . . . 55
4.1.2 Criteria of success . . . 55
4.1.3 Rules of inclusion and exclusion . . . 56
4.1.4 Principles . . . 56
4.2 Baselines: causal rules on the price . . . 57
4.3 Stateless learners on hand-made features . . . 57
4.4 Sequence models of the order-book stream . . . 59
4.4.1 The continuous-time stateful model . . . 59
4.4.2 The stateful model as the residual of the trees . . . 63
4.4.3 A pretrained forecaster used zero-shot . . . 63
4.5 Graph models . . . 64
4.5.1 Message passing on transactions . . . 65
4.5.2 The two graphs . . . 66
4.6 Information outside the book . . . 66
4.7 The families at a glance . . . 66

5 Experiments and Results 69
5.1 Experimental setup . . . 69
5.2 Rules and stateless learners . . . 70
5.2.1 Causal rules . . . 70
5.2.2 Trees on hand-made features: the no-state control . . . 70
5.2.3 Neural tabular learners: trees against networks . . . 73
5.2.4 What a stateless model cannot see . . . 73
5.3 Stateful models . . . 73

5.3.1 First models . . . 74
5.3.2 Loss and imbalance variants . . . 74
5.3.3 Memory, direct path and lookbacks: the chain of improvements . . . 74
5.3.4 Window encoders: space first, or time first . . . 76
5.3.5 The state given the control's inputs . . . 77
5.3.6 The cost of the online regime . . . 78
5.3.7 The state as the residual of the trees . . . 79
5.3.8 What the state is worth . . . 80
5.4 Graph models . . . 81
5.4.1 The wallet–market graph at every stage . . . 81
5.4.2 Smart money . . . 81
5.4.3 The market–market view: sibling contracts . . . 81
5.4.4 Why the graph does not help . . . 82
5.5 Information outside the book . . . 82
5.5.1 Time-series foundation models . . . 82
5.5.2 News volume . . . 84
5.5.3 The text of the market . . . 84
5.5.4 What is outside the book . . . 84
5.6 Across horizons, kinds of market and seeds . . . 84
5.6.1 Short horizons: one and fifteen minutes . . . 85
5.6.2 One hour . . . 86
5.6.3 Six and twelve hours . . . 86
5.6.4 Synthesis . . . 86
5.6.5 Where the models win, by kind of market . . . 87
5.6.6 Robustness . . . 88
5.7 Test set . . . 89
5.8 Discussion: the four questions . . . 94

6 Conclusions **95**
6.1 What the thesis found, question by question . . . 95
6.2 Limitations . . . 96
6.3 Towards deployment . . . 96
6.4 What the thesis would start from today . . . 97
6.5 Future work . . . 97

A The Sixty-One Hand-Made Features **99**

B Run Labels and Configurations **101**

C Reproduction **103**

D Supplementary Tables **105**

Glossary and Notation **107**

Bibliography **111**

List of Figures

1.1 Three weeks of Polymarket as a graph: 646 thousand wallets (blue) and 158 thousand markets (orange), joined by 32 million distinct trading relations built from 219 million on-chain fill legs, 6–26 March 2026. Markets are placed by the similarity of the wallets that trade them and by their category; wallets at the centre of the markets they trade. The labels name the largest islands. Section 3.7.2 explains the map. 4

2.1 The training loop of a neural classifier. A mini-batch of samples goes through the model (forward pass), the loss measures how far the predicted probabilities are from the labels, backpropagation computes the gradient of the loss with respect to every parameter (backward pass), and the optimiser moves the parameters a small step against the gradient; after every pass over the training data the model is scored on the validation day, and training stops when that score stops improving. 11

2.2 The gated recurrent unit. From the input $\mathbf{x}_t$ and the previous state $\mathbf{h}_{t-1}$ the two gates and the candidate are computed in parallel; the new state is an interpolation between the old state and the candidate, component by component, controlled by the update gate. 16

2.3 Truncated backpropagation through time. The state is carried forward through the whole stream, but the gradient of the summed losses of a window flows back only through the states of that window and stops at its boundary, where the incoming state is treated as a constant. 17

2.4 One transformer encoder layer. Every token produces a query, a key and a value; attention mixes the values of all tokens with weights given by the match between queries and keys; a feed-forward MLP is then applied to every token; residual connections and layer normalisation wrap both. 18

2.5 Message passing on a bipartite graph of one block. For a market, every incoming fill produces a message from the wallet's state, the market's state and the fill's attributes; the messages are aggregated in an order-independent way and written into the market's state by an update function; the same is then done for every wallet from the markets it traded. 21

2.6 One round of gradient boosting. The gradient and the curvature of the loss are computed at every training row for the current model; one small tree per class is fitted to them, with leaf values given by Eq. (2.18); the tree is added with a small step; the model is scored on the early-stopping day and training stops when the score has not improved for forty rounds. 23

3.1 Class shares of the validation labels by horizon. Below one hour the task is dominated by CONST; at twelve hours the three classes are nearly balanced. . 35

3.2 Lead–lag between sibling contracts, 6–13 March: for every minute change of the more active contract of a pair, the number of changes of the quieter one at each lag, divided by the number expected under independence. The peak is at lag zero and the two sides are symmetric: siblings co-move, nobody leads. . . 37

3.3 One frame of the animation of the transaction graph: Wednesday 18 March 2026, 14:10 UTC. Every fill of the last minutes lights the edge between a wallet and a market and fades with a half-life of twenty minutes; wallets are blue, markets are coloured by category (crypto yellow, sports orange, esports red, politics violet, weather pink, economy and tech brown, culture dark red); the largest islands of the map carry their label. The side panel gives the activity of the bin and the active sub-graph of the last hour. The ribbon at the top right is the crypto five- and fifteen-minute contracts; the bright blobs are the markets whose public is active in those minutes. 39

3.4 The transaction graph in six panels. (a) Complementary cumulative distributions of the wallet degree, the market degree and the edge multiplicity (log–log). (b) Fill legs per hour over the window. (c) Active wallets and active markets per hour. (d) Connected components of the hourly graph. (e) Share of the hour's active nodes in its giant component. (f) Cumulative number of wallets, markets and edges seen since the start of the window. 40

3.5 Two maps of the whole transaction graph (646 thousand wallets in blue, 158 thousand markets in orange, six million of the edges as faint lines). (a) The markets are laid out by UMAP on the trading-similarity space alone; the islands are groups of markets traded by the same wallets, and the crypto five- and fifteen-minute contracts form one island per few days, because the wallets that trade them on a given day are largely the wallets of that day. (b) The category of the market is added to the distance, and the map separates into one region per category, with the date as the internal order of the crypto ribbon. Labels are the largest islands found by HDBSCAN on each map; wallets are placed at the centroid of the markets they trade, so the dense blue regions are the publics of the corresponding markets and the blue rings are the exclusive publics of single markets. 41

3.6 Spearman correlation of the fifty-nine features on 200,000 validation samples at 15 min, ordered by average-linkage clustering of $1 - |\rho|$. Tick labels are coloured by group: blue G1 price, orange G2 book, aqua G3 flow and clock, yellow G4 trades, violet G5 static. The blocks are the returns with their signs, the event counts, the order-flow imbalances with their standardised copies, the on-chain trade quantities, and the sixty-minute activity triple. 44

3.7 UMAP of 50,000 validation samples at 15 min (fifty-nine rank-gauss features, thirty neighbours), the same points coloured four ways: (a) the label at 15 min, (b) the category of the market, (c) the distance of the mid from one half, (d) the quartile of the number of book rows in the last sixty minutes. The islands are coherent in activity and, largely, in category; the label is spread inside each island. 45

3.8 The two discriminant directions of LDA at 15 min on 20,000 validation samples. LD1 separates CONST (right) from the moving classes (left); the two directions overlap. 47

3.9 Clustering of the 11,294 markets with at least a hundred validation samples at 15 min, on thirteen standardised per-market aggregates (activity, spread, depth, realised volatility, distance of the mid from one half, share of moving samples, duration, time to the end, dollars traded, cancellation ratio, contract type). (a) Ward dendrogram cut at four clusters; (b) UMAP of the markets coloured by cluster. 48
3.10 Balanced accuracy of the balanced-vote kNN against k at every horizon, on 300,000 validation samples. The optimum moves from $k = 20$ at 1 min to $k = 1000$ at 6 and 12 h. 51

4.1 Mean Reversion-H as a block diagram. Two numbers enter, one class comes out. 57
4.2 The stateless learners as block diagrams. (a) LightGBM: the 61 features of the sample enter as they are, one tree per class is added at every round, and the three class scores pass through a softmax and the two calibration offsets. (b) MLP and TabM: the same features are first expanded into 674 numbers (at 1 h; 706 at 1 min), then pass through three layers of 512 units; TabM runs sixteen members through the same body and averages their outputs. 58
4.3 One block step of the stateful model, with the sizes of the two-pass model on the hand features (Table 4.1 gives the sizes of the other configurations). Dashed boxes are inputs of the step, blue boxes are states kept between steps, orange boxes are trained modules, the green box is the output. The edges and the wallet states exist only in the graph variant; the hand features and the tree log-probabilities only in the runs that use them. 60
4.4 The pretrained forecaster as a block diagram. (a) Zero-shot: 512 mid prices enter, nine quantiles of the mid at $t + H$ come out, and a rule turns them into a class. (b) The same quantiles, computed on a five-minute grid for every active market, become eleven inputs of the stateful model. 64
4.5 Message passing on the transactions of one block. Each fill is an edge between a wallet and a market with seven numbers; the receiving state is updated by a GRU cell, first on the market side, then on the wallet side. 65

5.1 The stateless control by feature group at every horizon, against Mean Reversion (all samples). Price history alone (G1) is below the rule at one minute and above the stateful model at six and twelve hours; the activity clock (G3) carries most of the gain at the short horizons. 71
5.2 The chain of improvements of the LOB-only stateful model at 1 h and 12 h (seed 0, all samples; error bars are the 95% hour-block bootstrap intervals of each step's gain). The direct path is the large step at 1 h, the hierarchical memory at 12 h; at 12 h the model stays below Mean Reversion on the full stream. 76
5.3 The gain of the stateful family over the stateless control at each horizon (all samples; calibrated against calibrated at 1 min, 15 min and 1 h with 95% hour-block bootstrap intervals; raw argmax at 6 h and 12 h). The sign reverses with the horizon. 87
5.4 Balanced accuracy at 1 h by price level: the lookback stateful model against Mean Reversion on the same samples. Both are best just inside the boundaries and the model collapses above 0.95. 89

List of Tables

3.1 Data sources. Row counts of the DuckDB database. 26

3.2 The *markets* table: its columns, their meaning and three real rows (a one-hour Bitcoin market of 7 March 2026, a tennis match of 9 March and a long-dated question on the Fed), one row per column. Identifiers are cut short; times are UTC. 27

3.3 The *order book* table: columns, meaning and three consecutive rows of one market (a tennis match, 10 March 2026, the first milliseconds after midnight UTC), one row per column. Every row is one change of one price level: the first trims the size resting at the best ask, the second empties the level just behind it, the third posts a new ask inside the spread, and the best ask and the mid move with it. 28

3.4 The *transactions* table: columns, meaning and the three legs of one fill (an esports market, 10 March 2026, 14:17:59 UTC, block 84,016,123), one leg per column. A taker buys 1,790.02 tokens of 3DMAX at 0.30. Only 9 of them come from a maker selling that token (order 1); the other 1,781.02 are minted: a maker buying the opposite outcome at 0.70 (order 0) and the taker together pay one dollar per token pair, and the pair is split between them (Section 2.2.1). 29

3.5 Share of the three classes among the labelled validation samples of 13 March, by horizon (samples of the stateful models: every market of every block at the block time; 1 h, 6 h and 12 h from the causal-baseline evaluation, 1 and 15 min from the short-horizon runs). 35

3.6 The wallet–market transaction graph of 6–26 March 2026. 38

3.7 Neighbourhood purity in the maps and in the original space at 15 min: share of the fifteen nearest neighbours with the same label, the share expected by chance ($\sum_c p_c^2$) and their ratio. UMAP on 50,000 samples, t-SNE on 20,000. . 46

3.8 Separability of the three classes at 15 min on the validation day, balanced accuracy on all and on deduplicated samples. Linear models and trees are fitted on one million training rows; kNN on the same rows and evaluated on 300,000 validation samples, with the vote of the neighbours weighted by the inverse class frequency. LightGBM is the G5 control of Section 5.2, uncalibrated. 46

3.9 The four Ward clusters of the markets at 15 min: profile (means of the raw aggregates; the logged quantities are back-transformed), composition by category, and balanced accuracy of Mean Reversion and of the control inside the cluster. 49

3.10 Neighbourhood purity lift (same label among the fifteen nearest neighbours over chance) in the original space / in the UMAP, by horizon. Only the prediction label depends on the horizon. 50

3.11 Separability by horizon: balanced accuracy on all and deduplicated samples of the same learners as Table 3.8. kNN with the best k on 300,000 samples; Δ is the difference from the control on the same samples, hour-block bootstrap 95% interval. 50
3.12 Mutual information between a feature and the label, in nats, by horizon, for the fifteen features that are among the six most informative at some horizon (bold where they are), and the sum over each group. 300,000 validation samples, nearest-neighbour estimator. Label entropy 0.42, 0.83, 1.00, 1.10, 1.09 nats. . 52

4.1 The configurations of the stateful model that appear in the result tables, with the name used for them. All share the state of 128 numbers per market, the three-rate memory, the global state, the five lookbacks and the direct path; "columns" is the width of one row of the order-book window (55 book channels plus the constant columns), and the sizes are those of the direct path and of the predictor. 63
4.2 The five families of approaches compared in the thesis. 67
4.3 What enters and what leaves each model, with sizes. A sample is a market at a block time; the output of every learned model is three class scores, read through the same calibration (two logit offsets fitted on the validation day) and the largest score. 67

5.1 Causal rules on the validation day, balanced accuracy, all samples and deduplicated samples (one per market per ten minutes). Momentum-H and Mean Reversion-H use the change over the past H; Momentum last-tick uses the sign of the last mid change. At 1 min and 15 min only Mean Reversion was evaluated, on the samples of the short-horizon runs. 70
5.2 Gradient boosting without state against the deep models, 1h, on the same validation samples (balanced accuracy; last column: difference from the time-first model in points with 95% hour-block bootstrap intervals). 71
5.3 The no-state control at the other horizons, all samples and deduplicated samples (balanced accuracy; Δ in points with 95% hour-block bootstrap intervals). At 1 minute the samples are one tensor row in twenty of the validation day and the only comparator is Mean Reversion. 72
5.4 Models before the direct path (seed 0; the lookback rows are the mean of three seeds). 74
5.5 Loss and imbalance variants, LOB-only model with lookback 1/6/12h, seed 0. 74
5.6 Chain of improvements, LOB-only, seed 0, all samples. Each Δ is the gain over the previous row with its 95% hour-block bootstrap interval. 75
5.7 Best configuration at 6h (seed 0). 75
5.8 Three window encoders in the best 1h configuration, seed 0 (balanced accuracy; differences in points with 95% hour-block bootstrap intervals). 77
5.9 Time-first encoder against the reference GRU: seed 1 at 1h and seed 0 at 6h (balanced accuracy; differences in points with 95% hour-block bootstrap intervals). 77
5.10 The best 1h model with and without the control's features, seed 0 (balanced accuracy; differences in points with 95% hour-block bootstrap intervals). . . . 78
5.11 The control's table (61 features, training rows of 7–11 March, validation samples of 13 March) under different learners, and the combination of LightGBM with other models (geometric mean of the probabilities, weight cross-fitted on the validation hours). Balanced accuracy; differences from LightGBM in points with 95% hour-block bootstrap intervals. 79

5.12 The deep model as the residual of the trees (1 h, validation samples of the control). Balanced accuracy; differences from LightGBM in points with 95% hour-block bootstrap intervals; "offsets" = two logit offsets (DOWN, UP) fitted on the even validation hours for the odd ones and vice versa. 80

5.13 Graph contribution at each stage (LOB-only / graph, all samples, seed 0). . . 81

5.14 Zero-shot Chronos-2 on 60,000 validation samples per horizon. 83

5.15 Best 1h model with and without the Chronos-2 forecast features, two seeds (balanced accuracy; differences in points with 95% hour-block bootstrap intervals; the last column is the difference of the seed-1 pair). 83

5.16 Short horizons against one hour: the no-state control, the tabular network on the control's table and the stateful model, every one with its own two cross-fitted logit offsets (DOWN, UP), on the runs' validation samples of 13 March. Balanced accuracy after calibration; Δ in points against the calibrated LightGBM with 95% hour-block bootstrap intervals. Labels DOWN / CONST / UP: 5.7/89.1/5.2% at 1 min, 16.4/69.0/14.6% at 15 min. 85

5.17 The best model of each family at each horizon, all samples and deduplicated samples (balanced accuracy). At 1 min, 15 min and 1 h every model carries its own two cross-fitted logit offsets; at 6 h and 12 h the raw argmax (the stateless control was not calibrated there and the stateful models' probabilities were not saved). "Stateless" is LightGBM G5; "stateful" is the best run without trees; "residual" is the two-pass stateful model on the cross-fitted trees. Δ in points against the stateless control. 87

5.18 Balanced accuracy at 1 h by kind of market: the lookback stateful model (graph variant, seed 0) against Mean Reversion on the same validation samples, with the class mix of each group. Δ in points. 88

5.19 Test days 24–26 March: the best model of each family at each horizon, all samples and deduplicated samples (balanced accuracy), every model with its two logit offsets chosen once on the validation day and applied unchanged. "Stateless" is LightGBM G5 refitted as in validation and evaluated on the test samples of the stateful run; "stateful" is the best validated run without trees (two passes with the hand features at 1 min, 15 min and 1 h, the time-first encoder at 6 h, the reference model, seed 0, at 12 h); "residual" is the two-pass stateful model on the cross-fitted trees. Δ in points against the stateless control, 95% hour-block bootstrap intervals over the test hours. TabM, not shown, is +0.27 [+0.21, +0.34] at 1 min, +0.15 [+0.02, +0.27] at 15 min and −0.08 [−0.25, +0.09] at 1 h; a second seed of the one-minute stateful model is +0.39 [+0.24, +0.57] on all samples and +1.52 [+1.22, +1.83] on the deduplicated ones, and a second seed of the one-minute residual model +0.96 [+0.85, +1.08] and +1.82 [+1.61, +2.02]. 91

5.20 Every one-hour model of the study on the test days (balanced accuracy, all samples and deduplicated samples, each model with its validation-day offsets; the validation column is the validation day with the same offsets). Δ in points against the calibrated control, 95% hour-block bootstrap intervals. 92

5.21 The main models by test day (balanced accuracy on all samples, calibrated as in Table 5.19). 93

5.22 Zero-shot Chronos-2 on 60,000 random test samples per horizon, as in Table 5.14 (raw decisions, no offsets): balanced accuracy of Mean Reversion-H, of the sign of the forecast median and of the median with a half-tick dead zone, the difference of the latter from the rule with its 95% hour-block interval, the share of the rule's directional calls on which the forecaster agrees with it, and the share of the disagreements, on samples whose price then moved, in which the forecaster was right. 94

B.1 Components of the run labels. 101

D.1 The stateless control by market category at 15 minutes, on the validation samples of the 15-minute run (balanced accuracy inside the category; p_{move} is the share of samples whose mid moved within the horizon, p_{up} the share of UP among the moves). The stateful model is the two-pass run without trees. . . 105

D.2 Univariate diagnostics of the strongest features at 1 h, on the control's validation samples: directional hit rate on the samples that moved (best of the two orientations), and area under the ROC curve of the feature's magnitude for separating CONST from a move. 105

Chapter 1

Introduction

1.1 Context

A prediction market trades contracts whose payoff depends on the outcome of a future event, and the price of a contract that pays one dollar if the event occurs can be read as the market's probability for that event. Prediction markets have been studied for decades as aggregators of dispersed information [1, 2], almost always through their prices at a daily or coarser resolution. A decentralised prediction market is a different kind of object for a data scientist. On Polymarket, the venue studied in this thesis, tens of thousands of markets on politics, sports, cryptocurrency prices, weather and culture are open at the same time; each is traded through a limit order book whose every update is published, and every trade is settled on a public blockchain under a persistent wallet address. What the data contain is therefore a complete market-microstructure record, the book at every price level and every fill, *and* the identity of every counterparty, at a resolution of about two seconds, across contracts that range from five-minute bets on the next move of Bitcoin to months-long political questions. Figure 1.1 shows what three weeks of that object look like when every trade is drawn as an edge between the wallet that made it and the market it was made on.

This thesis treats that object as a data-analysis and data-mining problem. The question is not whether one particular model can be made to work but what the price process of a prediction market looks like, how its trading is organised, which of the publicly observable information predicts the direction of a price over the next minute, hour or half day, and which families of models are able to read that information. The answers are empirical and they are given with their intervals.

1.2 The problem and why it is difficult

The prediction task of the thesis is the direction of the mid price of a market, down, unchanged or up, over a horizon from one minute to twelve hours, at every moment at which the market's book changes.

Four properties of the object make the task hard, and each of them is measured in the thesis before any model meets it. First, a market price is designed to *consolidate* information: whatever is publicly known about the event is already in the price, and in its weak form the efficient-market hypothesis says that the past of the price carries nothing about its future (Section 2.3); what remains at short horizons is microstructure, the mechanics of a thin book, not knowledge of the event. Second, the price is *bounded and discrete*: it lives in $[0, 1]$ with a tick of a thousandth, four samples in ten (a *sample* is one market at one moment at which a prediction is asked, Section 2.4) sit within five cents of a boundary, and most of its changes are one-tick oscillations that revert (Section 3.7.1). Third, the price is *mostly still*: over one

hour the mid of more than half of the samples does not move at all, over one minute nine in ten, so the task is as much "will it move" as "which way" (Section 3.7.1), and a classifier is judged as much by where it puts its thresholds as by what it knows. Fourth, much of what moves a price is *outside the data*: the spot price of Bitcoin for the crypto contracts, the score of the game for in-play sports, the headline for politics; the order book shows their effect, not their cause. Against these, the data offer two things that most financial datasets do not: the complete book rather than its top, and the identity of every counterparty. Whether either is worth anything for prediction is the empirical question.

1.3 Research questions

The thesis asks four questions and answers each of them with a measurement.

Q1 What regularities does the price process of a prediction market have, and how is its trading organised? Which share of the price changes is information and which is noise, what the price level and the kind of contract do to the dynamics, whether related contracts lead one another, how the wallets and the markets are connected, and what structure the observable features have (Section 3.7).

Q2 Which publicly observable information predicts the direction of the mid price? The market's own price history, the shape of its book, its activity, the on-chain trades, the wallets behind them, the contracts of the same event, the news flow, the text of the question, and the forecast of a pretrained time-series model (Sections 5.2, 5.4 and 5.5).

Q3 Which families of models can read that information, and what is a sequential state or a neighbourhood worth over a stateless learner on the same inputs? Causal rules, gradient-boosted trees and tabular networks, a continuous-time stateful model of the order-book stream, and message passing on the wallet–market graph (Sections 5.2 to 5.4).

Q4 How do the answers change with the horizon, from one minute to twelve hours, and with the kind of market (Section 5.6)?

What counts as an answer, the same samples, the same post-hoc calibration, an interval that excludes zero, a locked test period, is fixed in Section 4.1.2.

1.4 Approach

The study proceeds in four steps, which are also the chapters of the thesis. Twenty-one days of Polymarket were collected from three public sources, audited and prepared into a leakage-free dataset with a three-class label, and read with the tools of exploratory analysis and classical data mining before any model was trained (Chapter 3). Five families of approaches were built under explicit rules of inclusion and comparison, from causal rules on the price to a continuous-time model with message passing on the wallet–market graph (Chapter 4). They were compared on the same samples at five horizons, with hour-block bootstrap intervals and the same calibration for every model, against the best rule and the best stateless learner on the data (Chapter 5). The conclusions say where short-horizon predictability lives in a prediction market and what a model of it should be made of (Chapter 6).

1.5 Contributions

- A description of Polymarket as a data source, from the mechanics of the market (outcome tokens, the hybrid order book, settlement, resolution) to the collection and audit of

a three-week dataset of order books, on-chain fills and market metadata (Section 2.2 and Chapter 3).
- An exploratory characterisation of the price process: the event clock, the weight of the price level, the activity clock as the main regularity, the role of the contract type, the co-movement of sibling contracts without lead or lag, and the persistence of wallet skill (Section 3.7.1).
- A characterisation of the wallet–market transaction graph as an object: its size, density, degree distributions, single connected component and dense core, its growth, a large-graph map of it organised by category, contract type and date, and the coordinated publics of single markets (Section 3.7.2).
- A data-mining study of the feature space: its dimensionality, its organisation by market and activity rather than by label, the migration of the information from the activity clock to the shape of the book as the horizon grows, and families of markets that cut across the categories (Section 3.7.3).
- A leakage-free preparation of the data into a benchmark with causal baselines, hour-block bootstrap intervals, a locked test period and a calibration rule applied to every model (Section 3.5), and the comparison on it of five model families at five horizons from one minute to twelve hours (Sections 5.2 to 5.6).
- A continuous-time stateful model with a three-rate hierarchical memory, a time-first order-book encoder and a direct path that contains the mean-reversion rule as a special case, and the measurement of what its state is worth over the best stateless learner on the same inputs: nothing at one hour, a small, significant gain at one minute (Sections 5.3 and 5.6).
- A per-family and per-horizon account of where short-horizon predictability lives in a prediction market and of why each family can or cannot reach it (Section 5.6 and Chapter 6).

1.6 Structure of the thesis

Chapter 2 gives the background: how a prediction market and Polymarket work, what is known about Polymarket, and the notions of time series, graphs, learners and evaluation that the thesis uses. Chapter 3 follows the data from the three sources through the preprocessing to the final dataset and closes with its exploratory and statistical analysis, as a price process, a transaction graph and a feature space. Chapter 4 is the methodology: the objective, the criteria and the rules that decided which approaches were included, and the five families of approaches with their advantages and limits. Chapter 5 gives the experimental setup, the experiments and the results, family by family and then across horizons, kinds of market and seeds, and discusses them against the four questions. Chapter 6 draws the conclusions, states the limits of the study and what a deployment would need, and lists the future work. The appendices list the features, the run configurations and the steps that reproduce the numbers; a glossary closes the thesis.

Code and data. Everything in the thesis is produced by one repository: a Python package (`polymarket_gnn`) with the data pipeline in DuckDB and NumPy and the models in PyTorch, the collection and preprocessing scripts, the audit, graph and comparison scripts, a test suite for the label construction and the feature indices, and the reports from which every table is taken. Appendices B and C give the run labels, the configurations and the steps that reproduce each number; the raw data are never modified and every derived dataset is rebuilt from them.

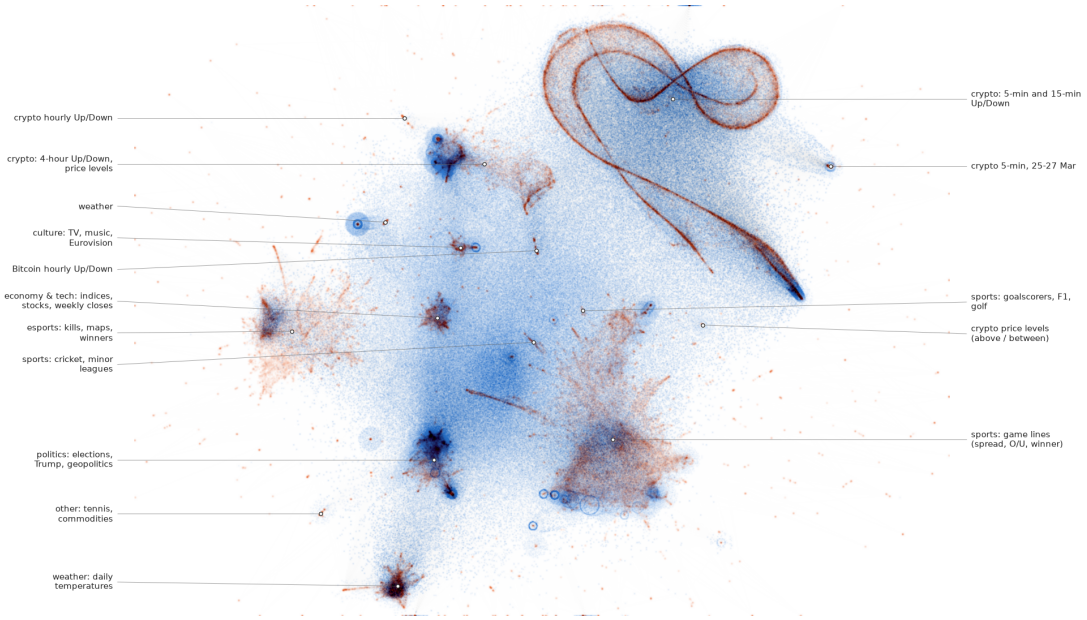


Figure 1.1: Three weeks of Polymarket as a graph: 646 thousand wallets (blue) and 158 thousand markets (orange), joined by 32 million distinct trading relations built from 219 million on-chain fill legs, 6–26 March 2026. Markets are placed by the similarity of the wallets that trade them and by their category; wallets at the centre of the markets they trade. The labels name the largest islands. Section 3.7.2 explains the map.

Chapter 2

Background

This chapter gives the background that the rest of the thesis assumes: what a prediction market is and how Polymarket, the venue studied here, works (Sections 2.1 and 2.2), what is known about it (Section 2.2.5), what an order book is and what is known about the predictability of its price (Section 2.3), the notation used from here on (Section 2.4), and then the models themselves: how a classifier is trained, from the likelihood to the loss, the gradient and the optimiser (Section 2.5); the neural networks for sequences, from the multilayer perceptron to recurrent units, attention and continuous-time states (Section 2.6); graph neural networks and dynamic graphs (Section 2.7); gradient-boosted trees and neural tabular learners (Section 2.8); and the evaluation of a classifier under class imbalance and dependent samples (Section 2.9).

2.1 Prediction markets

A *prediction market* is a market whose contracts pay according to the outcome of a future event, built so that the price of a contract can be read as a probability. The simplest contract is the *binary* one: it pays one unit if the event happens and nothing otherwise, so a trader who believes that the event has probability q is willing to buy below q and to sell above it, and the price at which buyers and sellers meet is a probability on which the market agrees. This is what prediction markets were created for: not to let people bet, but to *aggregate* what many people know or believe about an uncertain event into one number, the price, with money at stake as the incentive to be right. The argument behind them is that of market efficiency: anyone who knows better than the price can profit by trading against it, and in doing so moves the price towards what they know, so that the price ends up incorporating the information of everyone who trades [1, 2].

The idea has a history. The Iowa Electronic Markets have traded contracts on United States elections since 1988, and their prices the night before an election have been closer to the result than the final polls in most of the elections they covered [3]. Under the idealised assumptions of risk-neutral traders without budget constraints, the price of a binary contract equals the market's aggregate probability of the event; under weaker assumptions it is still close to the mean belief of the traders [1, 4]. Surveys of the empirical literature find that market prices are well calibrated and competitive with polls and expert forecasts, and that they react to news within minutes [1, 5]. Two systematic deviations are documented: a favourite–longshot bias, where unlikely events are over-priced and likely events under-priced, which grows with the time to expiration [6]; and, more recently on Polymarket itself, the same bias [7] together with a concentration of trading profits in a very small set of accounts [8].

2.2 Polymarket

Polymarket is the venue whose data the thesis studies. This section describes its mechanics in the detail that the data require, from outcome tokens and the hybrid order book to settlement and resolution, and closes with what the literature has found about it.

2.2.1 How Polymarket works

Polymarket is a decentralised prediction market built on the Polygon blockchain, with the United States dollar stablecoin USDC as collateral. The unit of trading is the *market*, a question with exactly two outcomes. For a binary question the outcomes are "Yes" and "No"; for the others they are the two sides of the question, "Up" and "Down" for a price interval, the two teams of a game, "Over" and "Under" for a total. An *event* can have more than two possible outcomes, the candidates of an election, the strikes of a price ladder, the teams of a tournament; on Polymarket such an event is not one market with many outcomes but a group of two-outcome markets, one per candidate, whose prices are linked because exactly one of them resolves to "Yes" (the "negative risk" structure described below). Each outcome of a market is a token, a conditional token in the sense of the Conditional Token Framework that Polymarket inherits from Gnosis: one dollar of collateral can at any time be *split* into one Yes token and one No token, a pair can be *merged* back into one dollar, and after resolution the winning token is *redeemed* for one dollar and the losing one for nothing. The two prices therefore sum to about one dollar, and a trader who wants to sell Yes at price p is indistinguishable, for the matching engine, from one who wants to buy No at $1 - p$.

Trading takes place on a central limit order book (CLOB) that is *hybrid*. Orders are signed messages that the trader submits to Polymarket's off-chain operator, which keeps the book, publishes its state and every change to it, and matches crossing orders; a match is then settled on-chain by the exchange contract, which transfers the tokens and the collateral and emits a fill event with the maker, the taker, the token, the side, the quantity, the price and the fee. The book is thus observable as a stream of price-level updates at the resolution of the operator's clock, and the trades are observable, with a delay of one block, as on-chain records attributed to wallet addresses. The two streams are not the same thing: most book events are orders placed or cancelled away from the best quotes and never trade; a fill is a book event that found a counterparty. In a multi-outcome event, the candidates of an election or the strikes of a "Bitcoin above x" ladder, the two-outcome markets of the candidates are linked by the payoff structure (exactly one candidate wins), with a "negative risk" conversion that lets a trader hold the No side of every candidate as one position; the prices of such siblings must respect bounds that the market does not always enforce [9].

2.2.2 Tokens, contracts and transactions: the blockchain layer

Polygon is a proof-of-stake chain that produces a block roughly every two seconds; every fill is a transaction in one of its blocks and carries the block number, the transaction hash and the wallet addresses of the maker and the taker. Because settlement is on-chain, every trade is attributed to a wallet that persists across markets and across time. This makes it possible to build a bipartite graph between wallets and markets, and to ask whether the history of a wallet (its activity, its past accuracy, the other markets it trades) is informative about the price of the markets it trades now. Works on Polymarket have used wallet-level data to study who wins and who loses [8], insider trading and informed flow [10], and arbitrage between related markets [11]. None of them, to my knowledge, had used the wallet–market graph as the input of a forecasting model; the graph models of Section 5.4 do. The block is also the

natural time step of the on-chain stream and the time step of every stateful model in the thesis (Section 3.5.1); the graph itself is described in Section 3.7.2.

2.2.3 The limit order book

A limit order book [12] is, for one token, two sorted lists of resting limit orders: bids, the prices at which traders are willing to buy, and asks, the prices at which they are willing to sell, each with the total size resting at that price. The highest bid and the lowest ask are the best quotes, their average is the *mid price* and their difference the *spread*; the size resting at the first few levels on each side is the *depth*. Prices on Polymarket live in $[0, 1]$ with a tick of 0.001 on most markets and 0.01 on some, so a contract that is almost certain trades at 0.99 or above and its price can only move by a few ticks. Because the Yes and No books mirror each other (a bid for Yes at p is an ask for No at $1 - p$), the book of one side describes the market, and this thesis reads the Yes side. An *order-book event* is a change of the size resting at one price on one side, caused by an order arriving, being cancelled or being filled; the operator publishes it with the new best quotes, the new mid and the new spread. Most events do not touch the best quotes: on a typical day about four events in a hundred change the mid (Section 3.7.1). A *snapshot* is the full state of the book at one moment.

Two consequences of this structure recur in every result of the thesis. A mid price is a bounded, discrete process observed through a thin book, so most of its changes are one-tick oscillations produced by an order arriving at or leaving the best quote, which are not information and which revert; and the dynamics depend on the price level, since a market at 0.98 has two ticks of room on one side and a hundred on the other. Dubach [13] measures this microstructure on a 52-day sample of Polymarket books and reports three findings that matter here. The depth of a book depends on how long the market lives, on its price level and on its volume. The resting orders are not concentrated near the best quotes but spread over hundreds of price levels, so the first levels of the book are thin even in a liquid market. And the direction of a trade, whether the taker bought or sold, guessed from the public book agrees with the on-chain record only about 60% of the time. For this reason the thesis reads the direction of every trade from the on-chain fill, where it is recorded exactly, and not from the book.

2.2.4 The life of a market

A market is created with its question, its resolution rules, its two outcomes and an end date; its contract is deployed on-chain and it starts accepting orders at known times, both recorded in the market's metadata and both known in advance. It then trades until the event is decided. Some markets are born and die within minutes: the crypto interval markets ask whether the price of Bitcoin, Ethereum or Solana will be above its starting value at the end of a five- or fifteen-minute interval, and a new one opens as the previous one closes; their prices follow an external reference, the spot price on a centralised exchange, with a lag of a few hundred milliseconds [14]. Sports markets trade before and during the game, with a recorded start time; political, economic and cultural questions trade for weeks or months; daily temperature and weather questions resolve every day. During the three weeks studied here about 175,000 markets existed and 15,000–21,000 were active on any given day (Chapter 3).

Resolution is not decided by Polymarket. When the event is over, a user proposes the outcome to UMA's optimistic oracle and posts a bond in collateral; the proposal stands if nobody disputes it within a two-hour challenge window, a wrong or premature proposal forfeits the bond, and a disputed proposal goes through the oracle's dispute-resolution process, a vote of the holders of its token [15]. The market's metadata records the resolution source named in the question and the end date of the oracle's process, and the winning token

becomes redeemable once the oracle has ruled. Resolution is therefore an event that happens after the end date, sometimes well after it; the thesis never uses the outcome or the resolution time as an input, and it predicts the price while the market is still open.

2.2.5 What is known about Polymarket

Machine-learning work on Polymarket has so far concentrated on *outcome* forecasting with language models and news [16, 17], on dataset construction [14, 18] and on the economics of the platform [19, 20]. Young [14] is the closest in spirit: a logistic model on 43 order-flow features does not beat the Polymarket mid price itself as a predictor of the outcome of the 15-minute Bitcoin markets. The present thesis predicts the *price*, not the outcome, over horizons at which the market is still open; the natural references are therefore the limit-order-book forecasting literature (Section 2.3) and the literature on dynamic graphs (Section 2.7.3).

2.3 Order books, price series and the forecasting task

A *time series* is a sequence of observations indexed by time. The series of this thesis are not sampled on a regular grid: the order book of a market changes whenever an order arrives, is cancelled or is filled, and a fill happens whenever two orders cross, so the two raw streams are sequences of timed *events*, hundreds of thousands per market on a busy day and a handful on a quiet one. Two ways of reading such a stream recur in what follows. The first summarises the past at a chosen time by a fixed set of quantities, the last price, its change over a few fixed lookbacks, the number of events in the last minutes, the shape of the book now, and hands the summary to a learner that has no memory of its own: the *stateless* reading, in which time enters only through the features. The second keeps a *state* per market, a vector updated at every event or every block from the new observations, and asks the state for the prediction: the *stateful* reading, that of recurrent networks. The two differ in what they can see (a state can in principle keep the order in which the events happened, a summary cannot), in how they are trained (a table of summaries can be shuffled and revisited; a state must be trained in time order through the stream) and in how they handle irregular time (a summary can include the time since the last event; a state has to be told how much time has elapsed, through a decay or an explicit input). The forecasting task itself takes two forms, the regression of the value at a future horizon H or the classification of the *direction* of its change; the thesis uses the second, with three classes (DOWN, CONST, UP) and horizons from one minute to twelve hours, and the null model against which every result is read is the random walk, under which the past of the price says nothing about its future.

The quantities of a book. With $p_1^b > p_2^b > \cdots$ the bid prices, $p_1^a < p_2^a < \cdots$ the ask prices and s_k^b, s_k^a the sizes resting at level k, the mid, the spread and the depth imbalance over the first n levels are

$$\text{mid} = \frac{p_1^b + p_1^a}{2}, \qquad \text{spread} = p_1^a - p_1^b, \qquad \text{imb}_n = \frac{\sum_{k\le n} s_k^b - \sum_{k\le n} s_k^a}{\sum_{k\le n} s_k^b + \sum_{k\le n} s_k^a} \in [-1, 1], \tag{2.1}$$

and the *microprice*, $(p_1^b s_1^a + p_1^a s_1^b)/(s_1^b + s_1^a)$, a weighted average of the two best quotes in which each price is weighted by the size resting on the *opposite* side: when little is resting at the best bid, the bid is the quote more likely to be consumed next, the weight of the bid price grows and the microprice sits closer to the bid. It is the best one-number guess of where the next mid will be. The *order-flow imbalance* of Cont, Kukanov, and Stoikov [21] accumulates, over a span of events, the signed changes of the size at the best quotes: an order that adds

size to the best bid or removes it from the best ask pushes the mid up, and the sum of such pushes over the recent past is strongly related to the next move of the mid. These quantities are the columns of the book vector $\mathbf{b}_i$ of Section 3.5.2 and of the hand-made features of Section 3.6.2.

Price predictability and limit order books. The efficient-market hypothesis in its weak form states that past prices carry no information about future prices [22]. At short horizons this is only approximately true: Lo and MacKinlay [23] reject the random-walk model on weekly stock returns, and the empirical "stylised facts" of high-frequency returns [24] include a negative autocorrelation at the shortest lags produced by the bid–ask bounce and by the discreteness of prices. Deep learning on limit order books started with DeepLOB [25], a convolutional network followed by a recurrent layer that reads a window of the last hundred book states and predicts the next move of the mid, on the FI-2010 benchmark of Finnish stocks [26]. Two recent studies are directly relevant. Briola, Bartolucci, and Aste [27] show that predictability depends on the microstructural properties of each asset (tick size, spread, depth) more than on the architecture, and Berti and Kasneci [28] show that a well-tuned MLP matches transformer models on the same benchmarks and that the label should be defined with a smoothing window that is chosen independently of the horizon. Both findings recur in this thesis: the price level (a proxy for the effective tick size) is the main driver of the results, and the simplest architectures are hard to beat. Three further results from the equity literature frame the choices of Chapter 4. Sirignano and Cont [29] train one recurrent model on the books of a thousand stocks and find that it beats models trained on each stock alone: the predictive relation between book and next move is to a large extent *universal*, which motivates one model pooled over all Polymarket markets rather than one per market. Kolm, Turiel, and Westray [30] compare raw book states with order-flow representations as inputs of the same networks and find that flows win, and that the useful horizon is of the order of a few price changes; Lucchese, Pakkanen, and Veraart [31] confirm on a larger panel that short-horizon predictability is present almost everywhere and that *what* the model is shown matters more than *which* model is used. Two lessons of the general time-series forecasting literature are also used. The first is a caution [32]: on the standard forecasting benchmarks a one-layer linear model on the raw window matches or beats several large transformer forecasters, so a complex encoder has to be checked against a trivial one on the same inputs. The second is a design pattern called *channel independence* [33]: when a series has many variables, reading each variable as its own sequence with shared weights, and mixing the variables only afterwards, is more accurate and more robust than mixing them at every step. The "time first" encoder of Section 4.4.1 applies this pattern to the order-book window.

2.4 Notation

The same symbols are used from here to the end of the thesis, and the glossary repeats them.

- **Markets, wallets, time.** $\mathcal{M}$ is the set of markets and $\mathcal{W}$ the set of wallets; $m \in \mathcal{M}$ and $w \in \mathcal{W}$ denote one of each. Time t is measured in seconds; a block of the Polygon chain, about two seconds, is the time step of the on-chain stream and of the stateful models (Section 3.5.1).
- **The book stream.** The order book of market m is observed as a sequence of timed events $(t_1, \mathbf{b}_1), (t_2, \mathbf{b}_2), \dots$ with $t_1 < t_2 < \cdots$, where $\mathbf{b}_i \in \mathbb{R}^{52}$ describes the state of the book just after the i-th event: best quotes, prices and sizes at ten levels, spread, mid price and order-flow statistics (Section 3.5.2). $\mathrm{mid}_m(t)$ is the mid price of the last event of m strictly before t, a step function of t.

- **Horizon and label.** H is the prediction horizon, one of 60 s, 900 s, 1 h, 6 h and 12 h. The label of market m at time t is

$$y_{m,t} = \begin{cases} \text{UP} & \text{if } \operatorname{mid}_m(t+H) > \operatorname{mid}_m(t), \\ \text{CONST} & \text{if } \operatorname{mid}_m(t+H) = \operatorname{mid}_m(t), \\ \text{DOWN} & \text{if } \operatorname{mid}_m(t+H) < \operatorname{mid}_m(t), \end{cases} \tag{2.2}$$

  where CONST means exactly equal: the mid moves in ticks of 0.001, and a move of one tick or more in either direction is UP or DOWN (Section 3.6.1). The classes are indexed $c \in \{1, 2, 3\}$ for DOWN, CONST, UP, and $\mathbf{e}_y \in \{0, 1\}^3$ is the one-hot vector of the label.
- **Samples.** A *sample* is a pair (m, t) at which a prediction is asked: a block t in which market m received at least one book event. Predictions are tied to the book events because an event is the moment at which something new about the market is observed; a market whose book did not change in a block has nothing new to read, its state is the one of its last event, and a prediction for it would repeat the previous one. Nothing is required of the market at $t + H$: the label reads the last mid before $t + H$, whatever happened in between (Section 3.6.1). A day has about ten million samples. When the market and the time are not needed, a sample is written $(\mathbf{x}_i, y_i)$, $i = 1, \ldots, N$.
- **Lookbacks.** For a span L, $\Delta_L^-(m, t) = \operatorname{mid}_m(t) - \operatorname{mid}_m(t - L)$ is the change of the mid over the past L; the rules of Section 4.2 read nothing else.
- **Models and parameters.** A learned model is a function $f(\cdot; \boldsymbol{\theta})$ of its inputs with parameters $\boldsymbol{\theta}$ (all the weights and biases, collected in one vector). A stateless model reads a vector of hand-made features $\mathbf{x}_{m,t} \in \mathbb{R}^{61}$, each a function of the data strictly before t (Section 3.6.2), and returns three class probabilities $\hat{\mathbf{p}}_{m,t} = f(\mathbf{x}_{m,t}; \boldsymbol{\theta}) \in \Delta^3$, where Δ^3 is the set of probability vectors over the three classes.
- **Stateful models.** A stateful model keeps a vector $\mathbf{h}_m(t) \in \mathbb{R}^d$ per market, with $d = 128$. Between two events of m the state decays, $\mathbf{h}_m(t) = e^{-r(t-t')}\,\mathbf{h}_m(t')$ for $t' < t$, with a learned rate r; at a block in which m is active the state is updated from the window $\mathbf{W}_{m,t} \in \mathbb{R}^{K \times 52}$ of its last K book states, $\mathbf{h}_m(t) \leftarrow U(\mathbf{h}_m(t^-), \phi(\mathbf{W}_{m,t}))$, where ϕ is the window encoder and U a gated recurrent update; the prediction is $\hat{\mathbf{p}}_{m,t} = g(\mathbf{h}_m(t), \mathbf{x}_{m,t})$ (Section 4.4.1).
- **Graphs.** A graph $G = (V, E)$ is a set of nodes V and a set of edges $E \subseteq V \times V$. The transaction graph is bipartite, $V = \mathcal{W} \cup \mathcal{M}$ with every edge joining a wallet to a market, and dynamic: an edge is an event $e = (w, m, t, \mathbf{a}_e)$, a fill of wallet w on market m at time t with attributes $\mathbf{a}_e \in \mathbb{R}^7$ (side, role, size, price, fee, token orientation; Section 4.5.1). E_t is the set of edges of block t and $N_t(v)$ the edges of E_t incident to node v. Wallets have states $\mathbf{h}_w(t) \in \mathbb{R}^d$ like markets.
- **Decision and score.** Every model outputs three logits $\mathbf{z}_{m,t} \in \mathbb{R}^3$; the probabilities are $\hat{\mathbf{p}} = \operatorname{softmax}(\mathbf{z} + \boldsymbol{\delta})$ with an offset $\boldsymbol{\delta} = (\delta_{\text{down}}, 0, \delta_{\text{up}})$, and the predicted class is $\hat{y} = \arg\max \hat{\mathbf{p}}$. The offset is the *calibration* of the decision rule. The training data are dominated by CONST and the models are trained with class weights or undersampling, so their raw probabilities are tilted and the most probable class is not the one that maximises balanced accuracy; two constants added to the DOWN and UP logits, chosen once on the validation day to maximise balanced accuracy and then frozen, move the decision boundary to where the score wants it (Sections 2.5.5 and 5.1). Every model receives its own offset, so that models are compared calibrated against calibrated. The score is balanced accuracy, $\text{BA} = \frac{1}{3}\sum_c \text{recall}_c$, where recall_c is the share of the samples of class c that are predicted as c (Section 2.9).

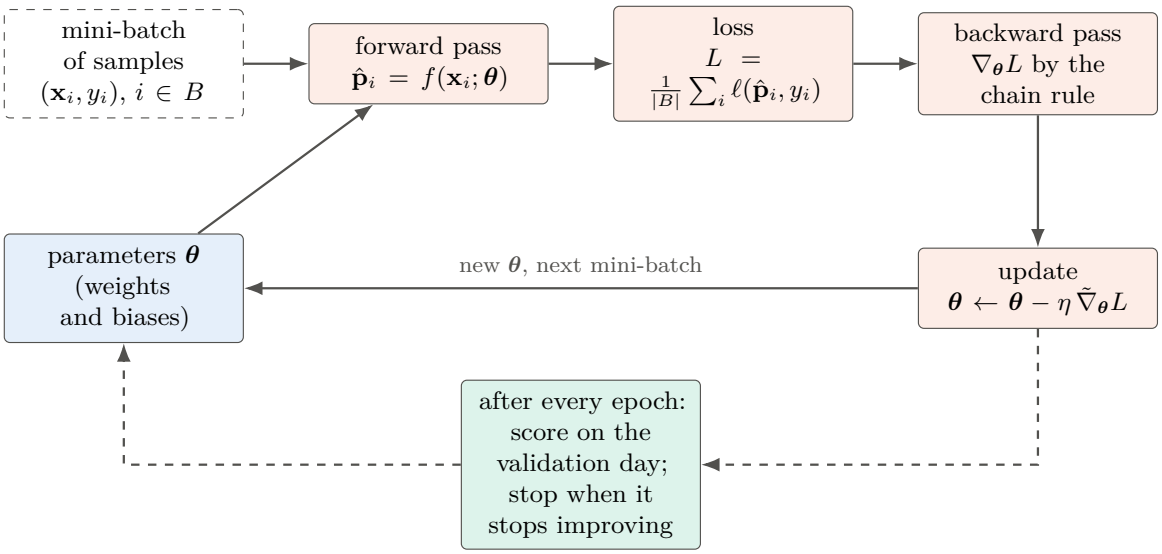


Figure 2.1: The training loop of a neural classifier. A mini-batch of samples goes through the model (forward pass), the loss measures how far the predicted probabilities are from the labels, backpropagation computes the gradient of the loss with respect to every parameter (backward pass), and the optimiser moves the parameters a small step against the gradient; after every pass over the training data the model is scored on the validation day, and training stops when that score stops improving.

2.5 Learning a classifier: likelihood, loss and training

Every learned model of the thesis, trees and networks alike, is trained in the same way: it is a function with parameters that turns the description of a sample into three probabilities, and the parameters are chosen to make the probabilities of the observed labels as large as possible on the training samples. This section spells that out, in the order in which it happens: the model as a conditional distribution, the likelihood and the loss that comes from it, the gradient and the optimiser that follows it, the corrections for class imbalance, and the guards against overfitting. Figure 2.1 is the loop that the rest of the section describes.

2.5.1 The model as a conditional distribution

A classifier with three classes is a function $f(\mathbf{x}; \boldsymbol{\theta})$ that maps the description $\mathbf{x}$ of a sample to a vector of three probabilities $\hat{\mathbf{p}} = (\hat{p}_1, \hat{p}_2, \hat{p}_3)$, one per class, that sum to one. Inside, the function first computes three unconstrained numbers, the *logits* $\mathbf{z} = (z_1, z_2, z_3)$, and then turns them into probabilities with the *softmax* function,

$$\hat{p}_c = \mathrm{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{c'} e^{z_{c'}}}, \qquad c = 1, 2, 3, \tag{2.3}$$

which makes every $\hat{p}_c$ positive and the three sum to one, and which preserves the order of the logits: the largest logit is the most probable class. Adding the same constant to every logit changes nothing; adding a constant to one logit raises that class's probability at the expense of the others, which is what the calibration offsets $\boldsymbol{\delta}$ of Section 2.4 do. The model is therefore a *conditional probability distribution* of the label given the inputs, $p(y = c \mid \mathbf{x}; \boldsymbol{\theta}) = \hat{p}_c$, and training is the choice of $\boldsymbol{\theta}$.

2.5.2 Maximum likelihood and the cross-entropy loss

Given N training samples $(\mathbf{x}_i, y_i)$ that are treated as independent, the probability that the model assigns to the observed labels, the *likelihood*, is the product of the probabilities it gives to each true class,

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}) = \prod_{i=1}^{N} \hat{p}_{i,y_i}, \tag{2.4}$$

where $\hat{p}_{i,y_i}$ is the probability that the model gives, for sample i, to the class y_i that actually occurred. The *maximum-likelihood* principle chooses the parameters that make this product largest [34]. A product of millions of numbers below one is unusable as it stands, so one takes its logarithm, which turns the product into a sum and does not change where the maximum is, and one changes its sign, so that the quantity is to be minimised and is a *loss*:

$$L(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{i,y_i} = \frac{1}{N}\sum_{i=1}^{N} \ell(\hat{\mathbf{p}}_i, y_i), \qquad \ell(\hat{\mathbf{p}}, y) = -\log \hat{p}_y = -\sum_{c} [y = c] \log \hat{p}_c. \tag{2.5}$$

This is the *cross-entropy* loss: it is zero when the model gives probability one to the true class and grows without bound as that probability goes to zero, so a confident mistake is punished far more than a hesitant one. The $1/N$ makes the loss an average per sample. Minimising the cross-entropy is the same as maximising the likelihood, and it has a second reading: it is the average number of nats of surprise that the labels cause the model, which is why a model with a lower cross-entropy is said to "fit" the data better. The cross-entropy is also what makes the outputs of the model *probabilities* in the useful sense: among all functions, the one that minimises it on infinitely many samples is the true conditional distribution $p(y \mid \mathbf{x})$, a property called *properness* that the squared error, say, does not have for classification.

With the softmax of Eq. (2.3) the derivative of this loss with respect to the logits is remarkably simple,

$$\frac{\partial \ell}{\partial z_c} = \hat{p}_c - [y = c], \qquad \text{or in vector form} \qquad \frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{p}} - \mathbf{e}_y, \tag{2.6}$$

the predicted probabilities minus the one-hot vector of the truth: the gradient pushes the logit of the true class up by the amount the model under-estimated it and the logits of the other classes down by the amount it over-estimated them. Every gradient computed in the thesis starts from this vector.

2.5.3 Gradient descent and its stochastic version

The loss is a function of millions of parameters and cannot be minimised in closed form. *Gradient descent* starts from random parameters and repeats a small step in the direction in which the loss decreases fastest, the negative of its gradient:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}), \tag{2.7}$$

where $\nabla_{\boldsymbol{\theta}} L$ is the vector of the partial derivatives of the loss with respect to every parameter and η, the *learning rate*, is the size of the step. Computing the gradient of the full loss Eq. (2.5) would require a pass over all N samples for every step, which is wasteful: the average over a random subset, a *mini-batch* B of a few thousand samples, is an unbiased estimate of the gradient and costs a thousandth of the time. Replacing the full gradient with the mini-batch gradient gives *stochastic gradient descent*, and one pass of mini-batches over the whole training set is an *epoch*. The noise of the estimate is not only harmless but useful: it lets the parameters escape the flat regions and poor minima in which the exact gradient would get stuck.

Plain stochastic gradient descent uses the same learning rate for every parameter, a poor choice when the parameters live on very different scales, as they do in a deep network. *Adam* [35], in its AdamW variant [36], is the optimiser of every network in the thesis: it keeps for each parameter a running average of the gradient, which smooths the noise of the mini-batches, and a running average of its square, by whose root the step is divided, so that every parameter moves by about η whatever the scale of its gradient; *weight decay* shrinks every parameter towards zero by a small fraction at every step and is the simplest guard against overfitting (Section 2.5.6). The learning rate is the one quantity set by hand: the stateful model uses $\eta = 10^{-4}$ throughout, the tabular networks a *one-cycle* schedule that raises η from a small value to 10^{-3} and lowers it again over the epochs, so that the early steps are large and the last ones fine.

2.5.4 Backpropagation

The gradient $\nabla_{\boldsymbol{\theta}} L$ has one component per parameter, millions of them, and *backpropagation* [37] computes all of them at the cost of about two forward passes. A network is a chain of simple operations, each with a known derivative. The forward pass computes every intermediate quantity from the input to the loss and keeps it; the backward pass applies the chain rule in reverse order, starting from the derivative of the loss with respect to the logits, Eq. (2.6), and multiplying at each step back by the local derivative of the operation. Each step reuses the result of the step before, and the derivative with respect to a weight matrix is the outer product of the gradient arriving at its output and the input it multiplied, which is why the forward quantities are kept. A network with branches, such as the stateful model of Chapter 4, in which the same state feeds several modules, sums the gradients that arrive along the different branches; a recurrent network applies the same rule through time (Section 2.6.3). In PyTorch [38] every operation records its inputs as it runs and the backward pass is generated automatically for any composition of operations (*automatic differentiation*), so no derivative is written by hand. The computation of each model is given with the model, in Sections 2.6 to 2.8 and in Chapter 4.

Activation functions and normalisation. The activation σ of a layer (Section 2.6.1) is, in the networks of the thesis, the SiLU, $\mathrm{SiLU}(a) = a/(1+e^{-a})$ [39, 40], a smooth version of the rectifier $\max(0, a)$ that passes positive inputs almost unchanged and suppresses negative ones; the logistic sigmoid $\sigma(a) = 1/(1+e^{-a})$, which maps any number into $(0, 1)$, and the hyperbolic tangent, which maps it into $(-1, 1)$, are the activations of the gates of recurrent units (Section 2.6.2). *Layer normalisation* [41] rescales a vector to zero mean and unit variance across its components and then applies a learned scale and shift, $\mathrm{LN}(\mathbf{h}) = \boldsymbol{\gamma} \odot (\mathbf{h} - \mu)/\sqrt{s^2 + \epsilon} + \boldsymbol{\beta}$ with μ and s^2 the mean and variance of the components of $\mathbf{h}$; it keeps the quantities that flow through a deep or recurrent network on a fixed scale, which keeps the gradients from vanishing or exploding as they are multiplied back through many layers.

2.5.5 Class imbalance: weights, the focal loss and the decision rule

The cross-entropy treats every sample alike, so on data in which one class is far more frequent than the others (89% CONST at one minute) the model learns first of all to predict that class, because that is where most of the loss is. Three corrections are used in the thesis. *Class weights* multiply the loss of each sample by a weight α_y that depends on its class, $\ell = -\alpha_y \log \hat{p}_y$, larger for the rare classes (2 for DOWN and UP, 1 for CONST in the stateful model; the inverse of the class frequency in the trees and the tabular networks), so that the

rare classes count as much as the frequent one in the gradient. The *focal loss* [42],

$$\ell_{\text{focal}}(\hat{\mathbf{p}}, y) = -\alpha_y (1 - \hat{p}_y)^{\gamma} \log \hat{p}_y, \qquad \gamma = 2, \tag{2.8}$$

multiplies the cross-entropy by $(1 - \hat{p}_y)^{\gamma}$, a factor that is close to one when the model is wrong and close to zero when it is already right: a sample that the model classifies with probability 0.9 contributes one hundredth of what it would under the cross-entropy, so training concentrates on the samples it gets wrong, which under imbalance are mostly those of the rare classes. Its gradient with respect to the logits is the cross-entropy gradient Eq. (2.6) scaled by the same factor and corrected by a term that vanishes as $\hat{p}_y \to 1$, so backpropagation proceeds unchanged.

The third correction acts on the *decision*, not on the training. A classifier is scored by which class it predicts, $\hat{y} = \arg\max_c \hat{p}_c$, and the class that is most probable is not the one that maximises balanced accuracy. Balanced accuracy weighs every class equally, which is the same as scoring the classifier on a population in which the three classes are equally frequent; on such a population the posterior probability of class c is, by Bayes' rule, proportional to $p(y = c \mid \mathbf{x})/\pi_c$, where π_c is the frequency of the class in the data the model learned from. The decision that maximises balanced accuracy is therefore

$$\hat{y} = \arg\max_c \frac{\hat{p}_c}{\pi_c} = \arg\max_c (z_c - \log \pi_c), \tag{2.9}$$

the most probable class *after* a constant has been subtracted from the logit of each class, larger for the frequent classes [43]. The constants are exact when the probabilities $\hat{\mathbf{p}}$ are correct; when they are not, and the focal loss and the class weights already distort them, the constants are better fitted than computed. The calibration rule of Section 5.1 is exactly this: two constants added to the DOWN and UP logits of every model, $\boldsymbol{\delta}$ in Section 2.4, chosen on the validation day to maximise balanced accuracy and applied unchanged afterwards, so that every model is compared with its decision rule placed the same way.

2.5.6 Generalisation, overfitting and early stopping

A model with millions of parameters can reach a very small loss on the training samples by memorising them, and such a model is useless on new data; what matters is the loss, or the balanced accuracy, on samples the model has not seen, its *generalisation*. The thesis guards against *overfitting* in the standard ways [44]. The data are split by date into training, validation and test periods (Section 3.6.3): the parameters are fitted on the training days, every choice about the model is made on the validation day, and the test days are read once at the end. *Early stopping* scores the model on a held-out day after every epoch (12 March for the trees and the tabular networks) and keeps the parameters of the best epoch, stopping when the score has not improved for a fixed number of epochs; it is the main regulariser of the trees and of the tabular networks. *Weight decay*, the shrinkage term of AdamW (Section 2.5.3), keeps the weights small. *Dropout* [45] sets a random fraction of the hidden units to zero at every training step, so that no unit can rely on a specific other unit being present, and uses all of them, scaled, at prediction time; the tabular networks use a rate of 0.15, the predictor of the stateful model 0.1. And the *deduplicated* evaluation of Section 3.6.1, which keeps one sample per market per fifteen minutes, guards against a subtler form of the same problem: ten million samples that are near-copies of one another make a model look far better on the busy markets than it is on independent ones.

2.6 Neural networks for sequences

The stateful models of the thesis read a stream of book events and keep a state per market. This section builds them up from their parts: the multilayer perceptron, the recurrent network and the gated recurrent unit that updates a state, the way such a network is trained through time, the attention mechanism and the transformer encoder that read a window of book states, the continuous-time treatment of a state between irregular events, and the pretrained time-series models that are used without training.

2.6.1 The multilayer perceptron

The multilayer perceptron (MLP) is the building block of every network here: a sequence of *layers*, each a linear map followed by an activation function σ applied element by element, with the last layer linear. With one hidden layer,

$$\mathbf{a} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{h} = \sigma(\mathbf{a}), \qquad \mathbf{z} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \qquad \hat{\mathbf{p}} = \operatorname{softmax}(\mathbf{z}), \tag{2.10}$$

where $\mathbf{W}_1 \in \mathbb{R}^{d\times n}$ and $\mathbf{W}_2 \in \mathbb{R}^{3\times d}$ are matrices of weights and $\mathbf{b}_1$, $\mathbf{b}_2$ vectors of biases; deeper networks stack more layers, $\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$ with $\mathbf{h}^{(0)} = \mathbf{x}$, and without the non-linear σ the layers would collapse into one linear map. The gradient of the loss with respect to the parameters of Eq. (2.10) is the backward pass of Section 2.5.4 written out: $\partial\ell/\partial\mathbf{z} = \hat{\mathbf{p}} - \mathbf{e}_y$, then $\partial\ell/\partial\mathbf{W}_2 = (\partial\ell/\partial\mathbf{z})\,\mathbf{h}^\top$, $\partial\ell/\partial\mathbf{h} = \mathbf{W}_2^\top\,\partial\ell/\partial\mathbf{z}$, $\partial\ell/\partial\mathbf{a} = \partial\ell/\partial\mathbf{h} \odot \sigma'(\mathbf{a})$ and $\partial\ell/\partial\mathbf{W}_1 = (\partial\ell/\partial\mathbf{a})\,\mathbf{x}^\top$, each line reusing the one before. A layer with d units maps its input to d numbers; the *width* d and the number of layers, the *depth*, are the two sizes of an MLP, and its parameters are the entries of the matrices and biases, $\sum_l d_l(d_{l-1}+1)$ in all. With one hidden layer of enough units an MLP can approximate any continuous function on a bounded domain, but the functions it learns easily are the smooth ones: an MLP is a poor learner of sharp thresholds on single inputs, which is where decision trees are strong (Section 2.8), and a good learner of smooth interactions between many inputs. The predictor of the stateful model is an MLP of one hidden layer of 64 units on the concatenation of its states; the tabular networks of Section 2.8 are MLPs of three layers of 512 units.

2.6.2 Recurrent networks and the gated recurrent unit

An MLP reads one vector and forgets it. A *recurrent neural network* reads a sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots$ one element at a time and keeps a *state* $\mathbf{h}_t \in \mathbb{R}^d$ that summarises what it has read so far, updated at every step from the previous state and the new input by the same function with the same parameters,

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}), \tag{2.11}$$

and the prediction at step t is read from $\mathbf{h}_t$. The state is the memory of the network: whatever the prediction at step t needs from the past has to be in $\mathbf{h}_t$, written there by the updates of the previous steps. The simple update of Eq. (2.11) has a well-known flaw. The gradient of a loss at step t with respect to the state at step $t-k$ is a product of k Jacobians $\partial\mathbf{h}_s/\partial\mathbf{h}_{s-1}$, each of which contains $\mathbf{U}$ and the derivative of the tanh; a product of k matrices shrinks or grows geometrically with k, so the gradient either *vanishes*, and the network cannot learn to use anything more than a few steps back, or *explodes* [46, 47]. Exploding gradients are tamed by *clipping*, rescaling the gradient vector whenever its norm exceeds a threshold (1.0 in the stateful model); vanishing ones are the reason for gated units.

The *gated recurrent unit* (GRU) [48], a lighter cousin of the LSTM [49], replaces Eq. (2.11) with an update in which the network decides, at every step and for every component of the

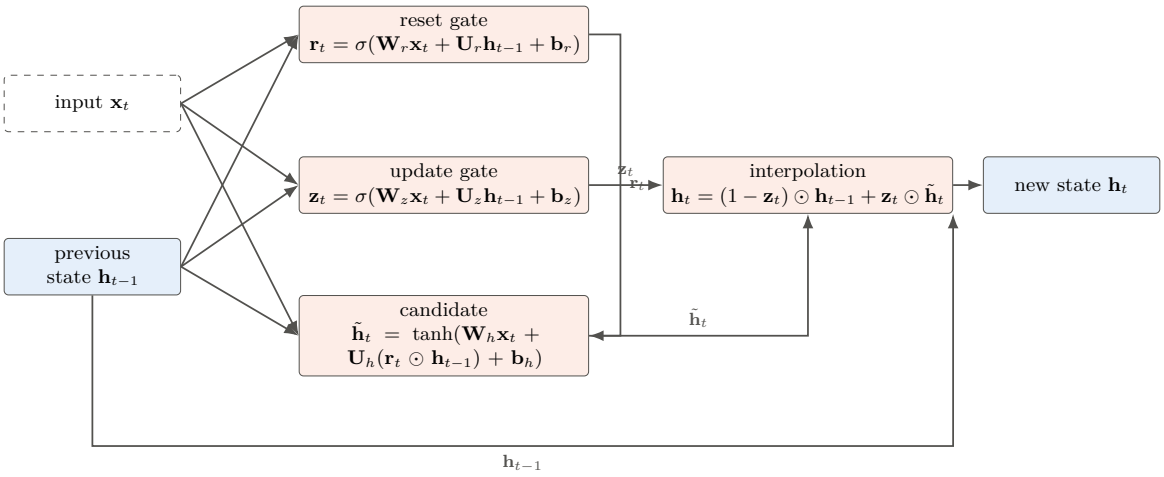


Figure 2.2: The gated recurrent unit. From the input $\mathbf{x}_t$ and the previous state $\mathbf{h}_{t-1}$ the two gates and the candidate are computed in parallel; the new state is an interpolation between the old state and the candidate, component by component, controlled by the update gate.

state, how much of the old state to keep and how much of a new candidate to write (Fig. 2.2):

$$\begin{aligned}
\mathbf{r}_t &= \sigma(\mathbf{W}_r\mathbf{x}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r) && \text{(reset gate)},\\
\mathbf{z}_t &= \sigma(\mathbf{W}_z\mathbf{x}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z) && \text{(update gate)},\\
\tilde{\mathbf{h}}_t &= \tanh\big(\mathbf{W}_h\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h\big) && \text{(candidate)},\\
\mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t && \text{(interpolation)}, \qquad (2.12)
\end{aligned}$$

where σ is the logistic sigmoid, so that the gates are vectors of numbers in $(0, 1)$. A component of the state whose update gate is near zero is copied unchanged from one step to the next, and the gradient flows back through that copy without shrinking, which is what lets a GRU keep information over many steps; the reset gate lets the candidate ignore the old state where that is useful. A GRU has three times the parameters of Eq. (2.11), $3d(n + d + 1)$ for an input of size n. In the thesis a GRU appears in two roles: as an *encoder* that reads the K book states of a window one after the other and returns its last state (Section 4.4.1), and as a *cell* that performs one update of a market's state from one input vector (Section 4.4.1); in the second role it is written $\text{GRUCell}(\mathbf{x}, \mathbf{h})$ and Eq. (2.12) is applied once.

2.6.3 Training through time

A recurrent network is trained like any other: the loss is summed over the steps and backpropagation computes its gradient. Because the state at step t depends on the parameters through every previous step, the computation is first *unrolled*, that is, written as one long chain of copies of the same update, one per step, and the backward pass runs through the whole chain, summing at each copy the gradient with respect to the shared parameters (*backpropagation through time*). On a stream that is days long, millions of steps, the chain cannot be kept in memory, and the gradient would in any case vanish beyond a few hundred steps. *Truncated* backpropagation through time [50] cuts the chain into windows of a fixed number of steps (Fig. 2.3): the forward pass runs through the whole stream and carries the state across windows, but the backward pass runs only inside the current window, and the state at the start of the window is treated as a constant (*detached*), so that no gradient flows into the previous windows. The stateful model of the thesis uses windows of five blocks and accumulates the gradients of twenty windows before an optimiser step; what the model can

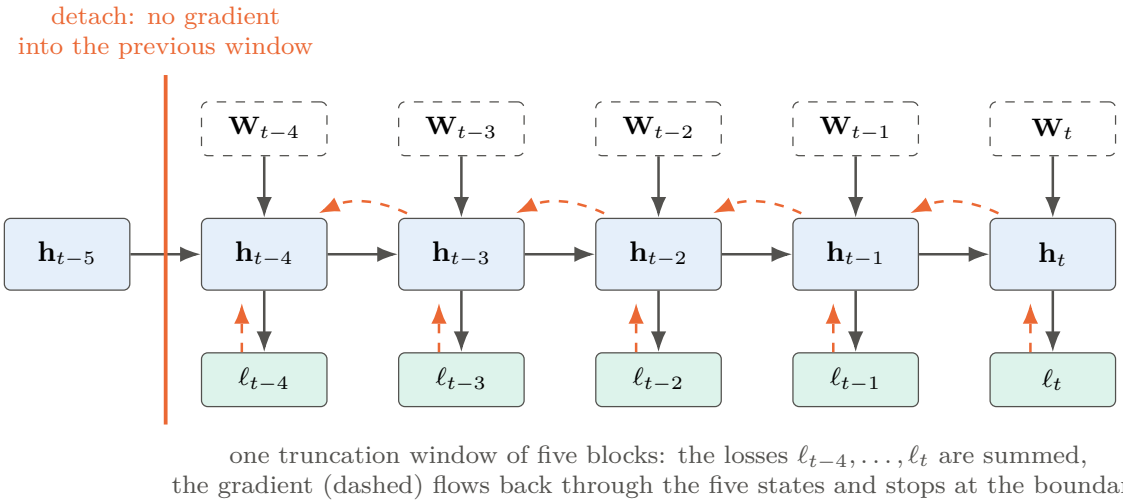


Figure 2.3: Truncated backpropagation through time. The state is carried forward through the whole stream, but the gradient of the summed losses of a window flows back only through the states of that window and stops at its boundary, where the incoming state is treated as a constant.

learn about dependencies longer than five blocks it learns through the state itself, which is why the state has to be a good memory on its own, and why the thesis adds to it the slow memories of Section 2.6.5.

This training regime differs from that of a tabular model in a way that matters for the results. A tabular model sees its training rows in random order, several times, in mini-batches that mix markets and hours; a stateful model sees the stream once, in time order, and the mini-batch of a step is whatever markets happen to be active in that block. The gradient estimates of the second regime are far more correlated from one step to the next, so the optimiser takes a noisier path, and the model has seen every row once when training ends. Section 5.3.6 measures what this costs on the same table.

2.6.4 Attention and the transformer encoder

A recurrent network reads a window of K book states in order and compresses them step by step; *attention* reads all of them at once and lets every element look at every other. Given n vectors, the *tokens* $\mathbf{X} \in \mathbb{R}^{n \times d}$ (the rows), three learned linear maps produce for every token a query $\mathbf{q}$, a key $\mathbf{k}$ and a value $\mathbf{v}$, and the output for token i is a weighted average of the values of all tokens, the weights being a softmax of how well its query matches their keys:

$$\mathrm{Attn}(\mathbf{X}) = \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\Big)\mathbf{V}, \qquad \mathbf{Q} = \mathbf{X}\mathbf{W}_Q,\ \mathbf{K} = \mathbf{X}\mathbf{W}_K,\ \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{2.13}$$

where the softmax is taken row by row and $\sqrt{d_k}$ keeps the dot products on a scale at which the softmax is not saturated [51]. Row i of the result is a mixture of the values of the tokens that are relevant to token i, with weights that the network learns; the operation does not depend on the order of the tokens, and when the order matters it is added as a *positional encoding* to the inputs. Several attentions with different weight matrices, the *heads*, run in parallel and their outputs are concatenated, so that different heads can attend to different things. A *transformer encoder layer* (Fig. 2.4) is a multi-head attention followed by a small MLP applied to each token, each of the two wrapped in a *residual connection* (the input is added to the output, so that the layer learns a correction rather than a replacement, and

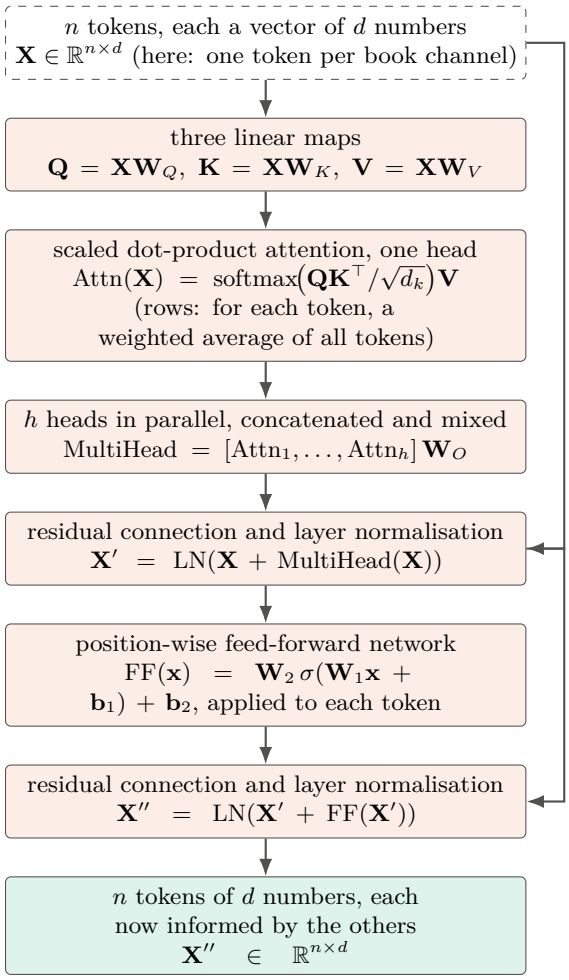


Figure 2.4: One transformer encoder layer. Every token produces a query, a key and a value; attention mixes the values of all tokens with weights given by the match between queries and keys; a feed-forward MLP is then applied to every token; residual connections and layer normalisation wrap both.

the gradient has a direct path back) and a layer normalisation. Stacking such layers gives the transformer, the architecture of large language models and of the time-series foundation models of Section 2.6.6.

In the thesis the tokens are not the time steps of a series but its *channels*: the time-first encoder of Section 4.4.1 maps each of the 55 book quantities, read as a series of K values, to one token with a linear map shared by all channels, adds a learned embedding that says which channel the token is, and lets one attention layer mix the 55 tokens. This is the channel-independence pattern of Section 2.3: time is read first, within each channel, by a map that is the same for all of them, and the channels are mixed afterwards. With 55 tokens of 32 numbers, four heads and one layer the encoder is small (about 40 thousand parameters); its cost is in the attention matrix, 55×55 per market per block.

2.6.5 Continuous time: decay and multi-rate memories

The updates above assume a sequence of steps with no notion of how much time passes between them, whereas a market's book events arrive at irregular times and a block may contain an update for a market whose previous event was a second or a day ago. Two devices

give the state a sense of time. The first is a *decay*: between two events the state is multiplied by $e^{-r\Delta t}$, where Δt is the elapsed time and $r > 0$ a rate, so that an old state fades towards zero and the network reads a faded state as "little is known". This is the solution of the differential equation $\mathrm{d}\mathbf{h}/\mathrm{d}t = -r\mathbf{h}$, the simplest continuous-time dynamics, and it has the useful property that decaying over a and then over b seconds is the same as decaying once over $a+b$, so the decay can be applied lazily, at a node's next event, without visiting the node at every block. GRU-D [52] introduced a learned decay of this kind for irregularly sampled medical series; in the stateful model the rate r is a single learned scalar (Section 4.4.1), and it converges to a half-life of seventeen seconds. The second device gives the model several memories at several speeds. A single state updated every two seconds and decaying in seconds cannot remember the morning; the *clockwork* RNN [53] partitions the state into modules that update at different rates, so that a slow module keeps a summary of the distant past while a fast one tracks the present. The hierarchical memory of Section 4.4.1 is a continuous-time version of this idea: besides the per-block state, each market has a minute state and an hour state, each a GRU cell that reads the state one level below once per period, and the predictor reads the three side by side.

2.6.6 Time-series foundation models

A *foundation model* for time series is a transformer pretrained once on a very large and varied corpus of series and then used on new series *zero-shot*, that is, without any training on them: given a context window of past values it outputs a forecast of the next values. Chronos [54] quantised the values of a series into tokens and trained a language model on them; Chronos-2 [55], the model used here, replaces the tokens with *patches*, short segments of the series mapped to vectors, and is trained with a *quantile loss*: for each quantile level τ among nine (0.1 to 0.9) and each future step, the output $\hat{q}$ is trained to minimise $\max\big(\tau(y-\hat{q}),\,(\tau-1)(y-\hat{q})\big)$, the pinball loss, whose minimiser is the τ-quantile of the distribution of y; the forecast is therefore not one number but a band of nine quantiles, from which a direction, a "no change" zone and a width can be read (Section 4.4.3). Every context window is standardised before it enters the model, which is what makes the model indifferent to the scale of a series and, on a one-tick process, what makes it read noise as signal (Section 5.5.1). The evidence of such models in finance is sobering: zero-shot forecasts of returns are close to a random walk, with directional accuracy near 50%, and the value found so far comes from a learned correction on top of a frozen model [56, 57]. This is the setting tested in Section 5.5.1.

2.7 Graphs and graph neural networks

The transactions of the market, read as a whole, form a graph between wallets and markets, and one of the families of the thesis reads it with message passing. This section fixes the vocabulary of graphs, gives the general form of a graph neural network and of the two operators used here, attention over the neighbours and a gated update, and describes how the same machinery is applied to a graph whose edges arrive one at a time.

2.7.1 Graphs, bipartite graphs, dynamic graphs

A *graph* $G = (V, E)$ is a set of nodes V and a set of edges $E \subseteq V \times V$; its *adjacency matrix* $\mathbf{A} \in \{0,1\}^{|V|\times|V|}$ has $A_{uv} = 1$ when $(u, v) \in E$, and the *neighbourhood* $N(v) = \{u : (u, v) \in E\}$ of a node is the set of nodes joined to it. The *degree* of a node is the size of its neighbourhood; a *connected component* is a maximal set of nodes between which a path exists; the *k-core* is the largest sub-graph in which every node has at least k neighbours inside it, which isolates the dense centre of a graph with a long tail of sparse nodes. The transactions of a prediction

market define a *bipartite* graph: $V = \mathcal{W} \cup \mathcal{M}$, wallets on one side and markets on the other, with an edge between a wallet and a market for every trade of the first on the second and no edge joining two wallets or two markets; its adjacency is a rectangular *biadjacency* matrix $\mathbf{B} \in \mathbb{R}^{|\mathcal{W}| \times |\mathcal{M}|}$, whose entries can carry the number of fills between the pair. A graph whose edges arrive one at a time, each with a timestamp, is a *dynamic* graph [58, 59]; it can be read as a sequence of *snapshots*, the graph of each hour, say, or as a *stream of events*, and the second reading is the one that matches the block-by-block arrival of the fills.

2.7.2 Message passing

A *graph neural network* computes a vector representation $\mathbf{h}_v$ of every node from its own features and from the representations of its neighbours [60–63]. The general form, the *message-passing neural network* [64], is one round of three operations (Fig. 2.5):

$$\mathbf{msg}_{u \to v} = M(\mathbf{h}_v, \mathbf{h}_u, \mathbf{a}_{uv}), \qquad \mathbf{s}_v = \mathrm{AGG}_{u \in N(v)}\, \mathbf{msg}_{u \to v}, \qquad \mathbf{h}'_v = U(\mathbf{h}_v, \mathbf{s}_v), \tag{2.14}$$

a *message* computed for every edge from the two ends and the edge's attributes $\mathbf{a}_{uv}$ by a learned function M (an MLP), an *aggregation* of the messages a node receives by a function that does not depend on their order or their number, a sum, a mean or a maximum, and an *update* of the node's representation from its old value and the aggregate by a learned function U. The aggregation is what makes the network a function of the graph rather than of a particular listing of it: the nodes of a neighbourhood have no natural order, and a sum or a mean gives the same result in any order (*permutation invariance*). Several rounds let information travel across the graph, one hop per round; the parameters of M and U are shared by all nodes and all rounds in the simplest designs, so that the network can be applied to a graph of any size and to nodes it has never seen (*inductive* learning), which is necessary here, where thirteen hundred new wallets appear every hour. Training is backpropagation through the three operations: the gradient of the loss at a node flows back through its update, through the aggregation to every message, and through every message to the representations of its neighbours and to the edge attributes, so that a node's parameters are shaped by the predictions made at its neighbours as well as at itself.

Attention over the neighbours. A plain sum treats every neighbour alike. The *graph attention network* [65] weighs the messages by a learned relevance: for every edge a score is computed from the two ends (and, here, the edge), the scores of the edges incident to a node are normalised with a softmax, and the aggregation is the weighted sum,

$$\alpha_{uv} = \frac{\exp\big(\mathrm{LeakyReLU}(\mathbf{q}^\top [\mathbf{h}_u, \mathbf{h}_v, \mathbf{e}_{uv}])\big)}{\sum_{u' \in N(v)} \exp\big(\mathrm{LeakyReLU}(\mathbf{q}^\top [\mathbf{h}_{u'}, \mathbf{h}_v, \mathbf{e}_{u'v}])\big)}, \qquad \mathbf{s}_v = \sum_{u \in N(v)} \alpha_{uv}\, \mathbf{msg}_{u \to v}, \tag{2.15}$$

where $[\cdot]$ is concatenation, $\mathbf{q}$ a learned vector and $\mathbf{e}_{uv}$ an encoding of the edge attributes. This is the attention of Eq. (2.13) restricted to the neighbours of a node, with one head. *Gated graph networks* [66] first used a GRU as the update function U, so that a node's representation is a state that each round of message passing updates by Eq. (2.12), with the aggregate as input; this is the update used in the stateful model, where a market's state is updated by the book (Section 4.4.1) and, in the graph variant, by the fills of the block (Section 4.5.1). The depth of message passing raises stability questions that Gravina, Bacciu, and Gallicchio [67] and Ceni, Gravina, Gallicchio, et al. [68] address with anti-symmetric and dissipative updates; one of these, the continuous-time anti-symmetric update, is tested in Section 5.3.2.

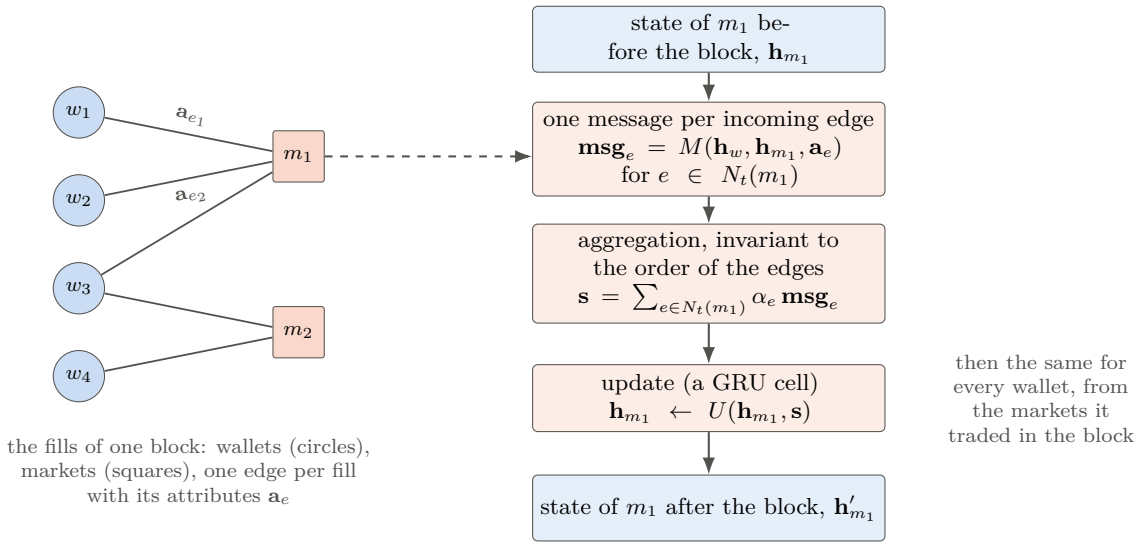


Figure 2.5: Message passing on a bipartite graph of one block. For a market, every incoming fill produces a message from the wallet's state, the market's state and the fill's attributes; the messages are aggregated in an order-independent way and written into the market's state by an update function; the same is then done for every wallet from the markets it traded.

2.7.3 Dynamic graphs: memories and messages in time

A transaction stream is a *continuous-time dynamic graph*: a sequence of timed events, each adding an edge between a wallet and a market. The event-based models of this family keep a *memory* vector per node and update it at every event. Temporal Graph Networks [69] give the general recipe, of which the earlier event-based models [70–72] are special cases: when an edge $(u, v, t, \mathbf{a})$ arrives, a message is computed for each end from the two memories, the elapsed time since the node's last event and the attributes, $\mathbf{msg}_v = M(\mathbf{s}_v, \mathbf{s}_u, \Delta t_v, \mathbf{a})$; the messages a node receives in a batch of events are aggregated; the memory is updated by a recurrent cell, $\mathbf{s}_v \leftarrow \mathrm{GRUCell}(\bar{\mathbf{msg}}_v, \mathbf{s}_v)$; and the prediction for a node is read from its memory, possibly after one more round of message passing over the recent neighbours. Training follows the stream in time order with truncated backpropagation, exactly as in Section 2.6.3, the batch being the events of a block. The model of Chapter 4 is in this family: market and wallet states are memories, the order-book window is a message a market sends to itself, and the transactions are messages between wallets and markets.

Three results shaped the design. Gravina et al. [73] (CTAN) propose a non-dissipative update for continuous-time dynamic graphs that propagates information over long ranges. Feldman et al. [74] revisit node-affinity prediction on temporal graphs and show that a strong heuristic, the recent interaction history, beats most learned models, and that a model should contain the heuristic as a special case; this is the origin of the direct path of Section 4.4.1. And benchmarks of spatiotemporal forecasting raise the same warning: per-node baselines are competitive and must be reported [75], and when the graph is not given it can be learned sparsely from the series [76].

2.8 Gradient-boosted trees and neural tabular learners

When the inputs are a table of hand-made features, one row per sample, the reference learner is not a neural network but an ensemble of decision trees built by gradient boosting. This section explains the decision tree, the boosting procedure that adds trees one at a time along

the gradient of the loss, the LightGBM implementation used here, and the two neural learners trained on the same table to tell the learner apart from the training regime.

2.8.1 Decision trees

A *decision tree* [77] partitions the input space with a sequence of binary questions on single features, "is the time since the last move below 17 minutes?", each question splitting the samples that reach it into two groups, until the groups, the *leaves*, are small or pure; every leaf then carries one output, the class frequencies of the training samples in it or, in the trees of a boosted ensemble, a single number. A tree is grown greedily: at every node the feature and the threshold are chosen that most reduce an *impurity* of the two children, the entropy or the Gini index of their class mix for a classification tree, or the squared error for a regression tree. A tree is invariant to any monotone rescaling of the features, needs no normalisation and no imputation beyond a rule for missing values, and represents sharp thresholds and interactions between a few features exactly, which an MLP can only approximate smoothly; a single deep tree, on the other hand, overfits, and a shallow one underfits. The depth-three tree of Section 3.7.3 is a tree of this kind read in full.

2.8.2 Gradient boosting

Boosting builds a strong model as a sum of many weak ones, each fitted to what the previous ones got wrong. *Gradient boosting* [78] makes this precise as gradient descent in the space of functions. The model after k rounds is an additive sum of trees,

$$F_k(\mathbf{x}) = F_{k-1}(\mathbf{x}) + \eta\, h_k(\mathbf{x}), \qquad F_0(\mathbf{x}) = \text{a constant (the log-frequencies of the classes)}, \tag{2.16}$$

where h_k is a small tree and η a *learning rate* (0.05 here) that shrinks every tree's contribution so that many trees share the work. For a classifier $F(\mathbf{x})$ is a vector of three scores, the logits of Eq. (2.3), and one tree per class is added at every round. Each round asks which tree would most reduce the loss $\sum_i \ell(F_{k-1}(\mathbf{x}_i) + h(\mathbf{x}_i), y_i)$, and answers it with a second-order Taylor expansion of the loss around the current model [79]: with

$$g_i = \frac{\partial \ell}{\partial F}\Big|_{F_{k-1}(\mathbf{x}_i)}, \qquad h_i = \frac{\partial^2 \ell}{\partial F^2}\Big|_{F_{k-1}(\mathbf{x}_i)}, \tag{2.17}$$

the gradient and the curvature of the loss at every training row (for the cross-entropy and class c, $g_i = \hat{p}_{i,c} - [y_i = c]$ and $h_i = \hat{p}_{i,c}(1 - \hat{p}_{i,c})$, the same quantities as in Eq. (2.6)), the loss change produced by a tree that assigns the value w_j to the rows of its leaf j is approximately $\sum_j (G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2)$ with $G_j = \sum_{i \in j} g_i$ and $H_j = \sum_{i \in j} h_i$, which is minimised by

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \text{gain of a split} = \frac{1}{2}\Big(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\Big), \tag{2.18}$$

where λ is an L2 penalty on the leaf values (1 here). The tree is grown by choosing, at every node, the feature and threshold whose split has the largest gain, and its leaves take the values w_j^*: a leaf's value is a Newton step of the loss, averaged over the rows in the leaf, so each round moves the scores of every row towards the direction that reduces its loss, by an amount that depends on how many rows share the leaf and how sure the model already is. Figure 2.6 draws the loop. Class weights enter by multiplying g_i and h_i of each row by α_{y_i}, and the offsets of Eq. (2.9) are added to the final scores exactly as for a network.

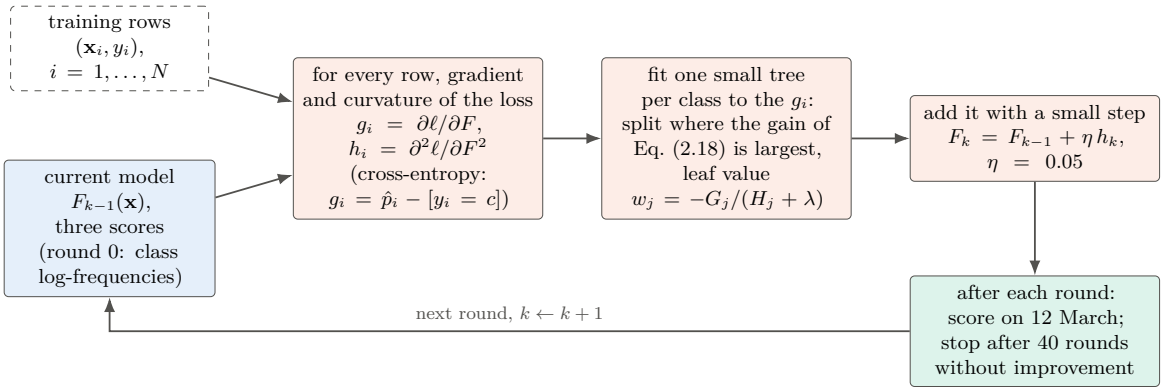


Figure 2.6: One round of gradient boosting. The gradient and the curvature of the loss are computed at every training row for the current model; one small tree per class is fitted to them, with leaf values given by Eq. (2.18); the tree is added with a small step; the model is scored on the early-stopping day and training stops when the score has not improved for forty rounds.

LightGBM. LightGBM [80] is the implementation used. Three of its choices matter here. It bins every feature into at most 255 *histogram* bins before training, so that the gain of every candidate split is computed from sums over bins rather than over rows, which makes a round on seven million rows take seconds. It grows trees *leaf-wise*, splitting at every step the leaf with the largest gain anywhere in the tree rather than completing one level at a time, up to a maximum number of leaves (63 here) and with a minimum number of rows per leaf (500), which is what limits how fine a threshold the trees can fit. And it subsamples the rows and the features of every tree (80% of each), so that consecutive trees see slightly different data and the ensemble does not fit the noise of any one row. The number of rounds is chosen by *early stopping* on a day the trees do not train on: the score on 12 March is computed after every round and training stops when it has not improved for forty rounds (536 rounds at one hour, hence 1,608 trees). Gradient-boosted trees are the reference for tabular prediction: the M5 forecasting competition was won almost entirely by LightGBM on lagged and calendar features [81], and systematic comparisons on tabular benchmarks find trees at or near the top unless a neural model is trained with care.

2.8.3 Neural tabular learners

That care consists of a few known ingredients [82]. The first is the *piecewise-linear encoding* of every numeric feature [83]: the range of a feature is cut at T quantiles of the training data, $b_0 < b_1 < \cdots < b_T$, and the feature value x is replaced by T numbers,

$$\mathrm{PLE}(x)_t = \min\Big(1, \max\Big(0, \frac{x - b_{t-1}}{b_t - b_{t-1}}\Big)\Big), \qquad t = 1, \dots, T, \tag{2.19}$$

which are one for the bins below x, zero for the bins above it and a fraction in the bin that contains it; a linear layer on this encoding can represent any piecewise-linear function of x with knots at the quantiles, that is, the thresholds that a tree would find, which is exactly what a plain MLP on the raw feature cannot do. With $T = 16$ bins the 59 numeric features become 59×16 numbers, plus a flag per missing value, the hour as a sine and a cosine and the category as eight indicators: 674 inputs at one hour. The other ingredients are the ones of Section 2.5: shuffled mini-batches of 8,192 rows, the one-cycle learning-rate schedule, dropout and early stopping on 12 March. The MLP so trained has three layers of 512 units. *TabM* [84] turns it into a cheap *ensemble*: $k = 16$ members share the same three layers, but

each member multiplies the inputs of every layer by its own learned vector (initialised at ± 1) and has its own output head, so that the sixteen members are different functions at the cost of one; the prediction is the mean of their sixteen softmax outputs. Averaging several differently initialised classifiers is the most reliable known improvement of a neural classifier, and TabM obtains most of it with a tenth of the parameters of sixteen separate networks; it is among the best neural tabular learners on recent benchmarks [85]. Both learners are used in Section 5.2 for one purpose: to tell whether a gap between the trees and the stateful model is the learner or the training regime.

2.9 Evaluation under imbalance and dependence

A *sample* is one market at one moment at which a prediction is asked (Section 2.4); a validation day has about ten million of them. The three classes are of very unequal size: at one hour more than half of the samples are CONST, at one minute nine in ten. Plain accuracy, the share of samples predicted correctly, would therefore reward a model that always predicts CONST: it is right 89% of the time at one minute and knows nothing. The thesis uses *balanced accuracy* instead [86]. For each class, the *recall* is the share of the samples of that class that the model predicts as that class, how many of the real UP samples it called UP, and so on; balanced accuracy is the mean of the three recalls,

$$\mathrm{BA} = \frac{1}{3}\sum_{c=1}^{3} \mathrm{recall}_c, \qquad \mathrm{recall}_c = \frac{\#\{i : y_i = c,\ \hat{y}_i = c\}}{\#\{i : y_i = c\}}. \tag{2.20}$$

A model that always predicts one class has recall one on that class and zero on the other two, hence balanced accuracy $1/3$ whatever the class sizes, and random guessing scores the same. Balanced accuracy depends on the predicted classes, not on the probabilities, which is why the decision rule of Eq. (2.9) matters as much as the model and why every comparison of the thesis is made after the same calibration.

A confidence interval says how much a number would move if the experiment were repeated on fresh data. The usual interval treats every sample as an independent observation, and with ten million samples it would be a few hundredths of a point wide. Here it would be wrong, because the samples are far from independent: one market contributes thousands of samples a few seconds apart that are almost copies of one another, and all the markets of one hour share the same news, so there are far fewer independent observations than samples. The intervals are therefore computed with a *block bootstrap* [87]. The validation day is cut into its 24 hours; a new "day" is built by drawing 24 hours at random, with replacement, from those 24; the metric, or the difference between two models, is recomputed on it; this is repeated two thousand times, and the 2.5th and 97.5th percentiles of the two thousand values are the 95% interval. Drawing whole hours keeps the dependence inside an hour intact, which is what makes the interval account for it. A difference whose interval includes zero is reported as a tie. Every number is also given on the *deduplicated* subset of Section 3.6.1, one sample per market per fifteen minutes, on which the samples are closest to independent, and on the *extreme* and *central* price ranges separately; a result that holds on one view only is reported as such.

Chapter 3

The Dataset: Collection, Preparation and Exploration

This chapter follows the data from their three sources to the objects the models read, in the order of the pipeline: what was collected and from where (Sections 3.1 and 3.2), how large and how reliable it is (Sections 3.3 and 3.4), how the raw streams were preprocessed into a common time axis and per-block tensors (Section 3.5), and how the final dataset, its labels, samples, features and periods, was defined (Section 3.6). It closes with an exploratory and statistical analysis of the data (Section 3.7): the price process, the transaction graph and the feature space, read with the tools of exploratory analysis and classical data mining before any model is trained.

3.1 The four tables

The dataset covers twenty-one consecutive days, from 6 to 26 March 2026, and consists of four tables (Table 3.1). The *markets* table describes every market that existed in the period: question text, tags, the two outcome tokens, creation and end dates, and the final outcome. The *order book* table is the stream of price-level updates of the "Yes" side of every market's book: each row carries the best bid and ask, the changed price level, size and side, and the mid price, with two timestamps (creation and reception). The *snapshots* table holds full order-book states. The *transactions* table is the on-chain record of every fill: block number, timestamp, transaction hash, wallet, role (maker or taker), token, side, quantity, price and fee.

The order book is a stream of *intentions* (orders placed and cancelled); the transactions are the subset that found a counterparty, recorded one block later. The two streams are collected by different clocks, and Section 3.5.1 says how they are aligned.

What a row looks like. Tables 3.2 to 3.4 list the columns of the three tables that the models read, with the meaning of each column and three real rows, written one row per column so that they fit the page. The snapshots table has the same two timestamps and the same market identifier as the order book, and one text column with the whole book as JSON: the token the book refers to, the best quotes, and two lists of (price, size) pairs, the bids in increasing price up to the best bid and the asks in decreasing price down to the best ask. One snapshot of 10 March 2026 at 14:00:00 reads, with the long lists cut short:

Table 3.1: Data sources. Row counts of the DuckDB database.

Table	Rows	Content
markets	175,476	one row per market: text, tags, tokens, dates, outcome
order book	5,555,777,555	price-level updates of the Yes-side book
snapshots	51,674,425	full order-book states (JSON)
transactions	218,882,492	on-chain fills with wallet, token, side, size, price

```
{
  "market_id": "0x0245263b...",
  "token_id":  "82153428...",
  "side":      "YES",
  "best_bid":  "0.28",
  "best_ask":  "0.5",
  "bids": [["0.01", "1923.79"], ["0.02", "1007.14"], ...,
           ["0.25", "15.3"],    ["0.28", "205.04"]],
  "asks": [["0.99", "1409.85"], ["0.98", "6215.4"],  ...]
}
```

that is, a market whose Yes token has a best bid of 0.28 with 205 tokens resting there and a best ask of 0.50, with most of the resting size far from the best quotes.

3.2 Collection

The three streams come from three places.

3.2.1 Market metadata: the Gamma API

The *markets* table was built from Polymarket's public metadata service (the Gamma API): the markets closed in the window were paged through its keyset endpoint, ordered by start date, with the tag list attached, and the markets that appear in the transactions but not in that listing (tokens traded in the window on markets created outside it) were completed one by one through the order-book operator's markets-by-token endpoint. The tags are the source of the eight categories used throughout the thesis (crypto, sports, esports, politics, weather, economy and tech, culture, other), assigned by a fixed mapping from the tag list.

3.2.2 The order-book archive

The *order book* and *snapshots* tables come from a public archive of the operator's websocket feed, the *Polymarket Tick-Level Orderbook Dataset* published on Kaggle by Marvin Gozo under a CC BY-NC 4.0 licence [88]: one file per day, restricted to the Yes side of every book. Every row of the feed carries two timestamps: the time at which the operator created the message and the time at which the archive's collector received it; the thesis uses the creation time as the time of the event and keeps the reception time only to check the feed's latency.

3.2.3 The on-chain transactions: a blockchain scraper

I collected the *transactions* table with a modified version of `poly-trade-scan` [89], an open-source Python client that connects to a Polygon node over a WebSocket JSON-RPC endpoint, reads every block, filters the transactions addressed to Polymarket's exchange contracts and decodes their `matchOrders` calls. The original tool emits one row per transaction, the taker's order with its nominal amounts; my version decodes the whole call, the amounts actually

Table 3.2: The *markets* table: its columns, their meaning and three real rows (a one-hour Bitcoin market of 7 March 2026, a tennis match of 9 March and a long-dated question on the Fed), one row per column. Identifiers are cut short; times are UTC.

Column	Meaning	Row 1	Row 2	Row 3
`market_id`	the on-chain identifier of the market (its *condition id*)	`0x6b03a4da...1a91`	`0x020c64e6...d0c7`	`0xd4e77ba6...8527`
`question`	the question text	Bitcoin Up or Down - March 7, 7PM ET	Austin: Caty McNally vs Yulia Starodubtseva	Will no Fed rate cuts happen in 2026?
`slug`	the name of the market in its web address	`bitcoin-up-or-down-march-7-7pm-et`	`wta-mcnally-starodu-2026-03-09`	`will-no-fed-rate-cuts-happen-in-2026`
`tags`	the tags attached by Polymarket; the category of Section 3.2 is derived from them	1H, bitcoin, crypto, crypto-prices, hide-from-new, recurring, up-or-down	games, sports, tennis	business, economic-policy, economy, fed, fed-rates, finance, jerome-powell, rewards-200-4pt5-50
`outcome_1`, `outcome_2`	the names of the two outcomes	Up, Down	Caty McNally, Yulia Starodubtseva	Yes, No
`token_id_1`, `token_id_2`	the identifiers of the two outcome tokens, the key that joins the order book and the transactions to the market	`7094783845...`, `3450397751...`	`7364199465...`, `4686906466...`	`1240360292...`, `2129459220...`
`outcome_price_1`, `outcome_price_2`	the final prices after resolution, one and zero; for an open market, the last prices	0.0, 1.0 (Down won)	1.0, 0.0 (McNally won)	0.8135, 0.1865 (open)
`deploying_timestamp`	when the market's contract was deployed on-chain	2026-03-06 00:00:06	2026-03-09 15:07:57	2025-09-29 22:08:49
`accepting_orders_timestamp`	when the book started accepting orders	2026-03-06 00:01:21	2026-03-09 15:09:21	2025-09-29 22:24:25
`start_date`, `end_date`	the nominal start and end of the question	2026-03-06 00:02:27, 2026-03-08 01:00:00	2026-03-09 15:10:27, 2026-03-16 19:00:00	2025-09-29 22:24:45, 2026-12-31 00:00:00
`closed_time`	when trading closed; empty while the market is open	2026-03-08 02:34:13	2026-03-10 02:22:47	(open)
`resolution_source`	the source named for resolution	binance.com, BTC/USDT	wtatennis.com/scores	(none given)

Table 3.3: The *order book* table: columns, meaning and three consecutive rows of one market (a tennis match, 10 March 2026, the first milliseconds after midnight UTC), one row per column. Every row is one change of one price level: the first trims the size resting at the best ask, the second empties the level just behind it, the third posts a new ask inside the spread, and the best ask and the mid move with it.

Column	Meaning	Row 1	Row 2	Row 3
`timestamp_created_at`	when the operator created the message; the time of the event in the thesis	2026-03-10 00:00:00.069	2026-03-10 00:00:00.069	2026-03-10 00:00:00.069
`timestamp_received`	when the archive's collector received it; used only to check latency	2026-03-10 00:00:00.010	2026-03-10 00:00:00.011	2026-03-10 00:00:00.015
`market_id`	the market	`0x020c64e6...d0c7` (Austin: McNally vs Starodubtseva)	same	same
`token_id`	the outcome token whose book this is	`4686906466...` (Starodubtseva)	same	same
`change_side`	the side of the book that changed: BUY is the bid side, SELL the ask side	SELL	SELL	SELL
`change_price`	the price level that changed	0.33	0.34	0.22
`change_size`	the size now resting at that level; zero when the level is emptied	140.0	0.0	3000.0
`delta`	the change of that size with respect to the previous state of the level	−8.0	−36.0	+3000.0
`best_bid`, `best_ask`	the best quotes after the event	0.20, 0.33	0.20, 0.33	0.20, 0.22
`spread`	best ask minus best bid	0.13	0.13	0.02
`mid_price`	the average of the best quotes; the price whose direction the thesis predicts	0.265	0.265	0.21

Table 3.4: The *transactions* table: columns, meaning and the three legs of one fill (an esports market, 10 March 2026, 14:17:59 UTC, block 84,016,123), one leg per column. A taker buys 1,790.02 tokens of 3DMAX at 0.30. Only 9 of them come from a maker selling that token (order 1); the other 1,781.02 are minted: a maker buying the opposite outcome at 0.70 (order 0) and the taker together pay one dollar per token pair, and the pair is split between them (Section 2.2.1).

Column	Meaning	Taker	Maker 0	Maker 1
`block_number`	the Polygon block that settled the fill	84016123	84016123	84016123
`timestamp`	the time of the block	2026-03-10 14:17:59	2026-03-10 14:17:59	2026-03-10 14:17:59
`tx_hash`	the on-chain transaction; one transaction may contain several legs and several fills	`0x82d8545e...b519`	same	same
`order_index`	-1 for the taker's order, $0, 1, \ldots$ for the makers' orders matched against it	-1	0	1
`wallet`	the address of the trader of this leg	`0x7a464614...ab25`	`0xa6fd9229...30f0`	`0x73a293ad...3b68`
`role`	TAKER (the order that crossed the book) or MAKER (a resting order)	TAKER	MAKER	MAKER
`token_id`	the outcome token traded	`5334097798...` (3DMAX)	`9786745195...` (Natus Vincere)	`5334097798...` (3DMAX)
`token_direction`	$+1$ if the token is the market's first token, -1 if the second; the models correct it by the side the book refers to (Section 3.4)	$+1$	-1	$+1$
`side`	BUY or SELL, from the point of view of this wallet	BUY	BUY	SELL
`tokens`	the number of tokens filled	1790.02	1781.02	9.0
`price`	the price per token, in dollars	0.30	0.70	0.30
`total_usdc`	the collateral paid or received	537.006	1246.714	2.7
`fee_rate`	the fee rate charged on the leg	0.0	0.0	0.0
`market_id`	the market, obtained from the token through the markets table	`0x6bae5e4c...` (Counter-Strike: 3DMAX vs Natus Vincere)	same	same

filled and received by the taker, the filled amount and the fee of every maker order, and emits one row per wallet and per fill (block number, timestamp, transaction hash, order index, wallet, role, token, side, quantity, price, total collateral and fee rate), with the longer timeouts and the many retries that a free public node requires. The node was the public endpoint of dRPC (`wss://polygon.drpc.org`); the token identifier of every fill is then mapped to its market through the markets table. A transaction with one taker and n makers yields n taker rows and n maker rows, so the 219 million rows are *fill legs*; 79 million of them are taker legs.

3.3 Scale, coverage and storage

Daily activity is stable over the period: 9.2–11.8 million transaction rows, 122,000–161,000 distinct wallets and 15,000–21,000 active markets per day. Markets and wallets are mapped to dense indices (175,292 markets, 646,394 wallets). The data were stored as Parquet files and queried with DuckDB [90]; all derived tensors were produced with SQL and NumPy and checked by unit tests. The raw files are never modified and every derived dataset is rebuilt from them (Appendix C).

3.4 Data quality

Four properties of the data shape every later decision.

Fabricated mid on one-sided books. When one side of the book is empty, the feature builder substitutes 0.49 or 0.51 for the missing price; 2.4% of the rows have a fabricated mid and 2.1% a crossed book. Excluding those samples changes the headline metrics by less than 0.2 points, so they were kept, but the thesis flags them.

Token orientation. The order book, and therefore the label, refers to token 1 only in Yes/No markets and to token 2 in all other markets (Up/Down, Over/Under, team A/team B), whereas the raw transaction direction is defined with respect to token 1 everywhere. Without a correction, the sign of a buy is wrong relative to the label in 82% of the edges of a typical day (8.5% of the labelled 1h samples). Every model reads the corrected sign (Section 4.5.1).

Price level. About 40% of the samples have a mid below 0.05 or above 0.95 (Table 3.5); their dynamics are those of a bounded, one-tick process and they are where every model struggles most. Every result is therefore reported separately on the extreme and the central samples (Section 5.1).

Gaps. The two streams have one dip in common. Between 01:00 and 03:00 UTC on 20 March the hourly number of book events falls from about nine million to 1.3 million and the hourly number of fill legs from about 500 thousand to 79 thousand, and both recover within three hours (Fig. 3.4). The two streams are collected independently, by the archive's collector and by my node scraper, so the dip is in the activity of the venue, not in the collection; Polymarket's public status history lists no incident for that day. No model is trained or evaluated on 20 March.

3.5 Preprocessing

The data of Section 3.1 are three streams on three clocks. This section turns them into the objects the models read: a common time axis and per-block tensors.

3.5.1 Time axis: blocks and the synchronisation of the two streams

Polygon produces a block roughly every two seconds, and every transaction carries its block number. The block is therefore the natural time step of the on-chain stream and it is also the time step of the models: a "step" is one block, the set of transactions in it, and the order-book updates that arrived since the previous block. About 1,800 blocks per hour contain Polymarket transactions. Order-book updates arrive at their own times; a model that acts at block t sees every update with a timestamp strictly before t. Because the two streams are collected by different clocks (Section 3.1), every stateful model reads the book updates created strictly before a block time and the fills of the block itself; a sample at block t never sees an update created at or after t.

3.5.2 Derived tensors

Order-book features. For every market and every order-book update, a 52-dimensional feature vector is computed from the book state at that moment: best bid and ask, five levels of bid and ask prices and sizes, spread, mid price, volume and depth imbalances, recent trade statistics, and rolling statistics of the mid. The mid price is stored in column 40 of the vector; column 0 is the relative deviation of the best bid from the mid. The vectors are stored as hourly Parquet partitions (about 287,000 snapshots per hour, 27 GB in total). To these, the models append the long-lookback price features of Section 4.4.1. The order-flow imbalance features follow Cont, Kukanov, and Stoikov [21].

Market lifecycle. A table of 283,318 open/close events gives, for every market, the time at which its contract was deployed (known in advance, hence causal) and the time of its last trade (known only in hindsight). The deployment time defines when a market may enter the model. The last-trade time is used only to define the evaluation sample (a label is not computed when the market stops trading before the horizon), never as an input.

Static market features. Each market has a 390-dimensional static vector: a 384-dimensional sentence embedding of its question text, produced by a frozen MiniLM sentence encoder [91], and six numerical fields (initial prices, creation date). These features are fixed at market creation, so they are leak-free; they initialise the market's state (Section 4.4.1) through a fixed random projection. What the text contributes to prediction is measured in Section 5.2.2.

Identifiers. Markets and wallets are mapped to dense indices (175,292 markets, 646,394 wallets) so that their states can live in two matrices on the GPU.

3.6 The final dataset: labels, samples, features and periods

This section defines the dataset on which every model of the thesis is trained and evaluated: the label and the sample population, the hand-made features and the other feature tables, and the split of the twenty-one days into training, validation and test periods.

3.6.1 Labels

The target of a market m at block time t and horizon H is the direction of the change of its mid price between t and $t + H$ (Section 2.4):

$$y_{m,t} = \begin{cases} \text{UP} & \text{if } \text{mid}_m(t+H) > \text{mid}_m(t), \\ \text{CONST} & \text{if } \text{mid}_m(t+H) = \text{mid}_m(t), \\ \text{DOWN} & \text{if } \text{mid}_m(t+H) < \text{mid}_m(t), \end{cases} \tag{3.1}$$

where $\text{mid}_m(s)$ is the mid price of the last order-book update of m strictly before s. A price is CONST when the two mids are *exactly* equal: there is no tolerance band, the mid moves in ticks of 0.001, and a move of one tick or more in either direction is UP or DOWN. Three choices matter:

- **Three classes.** The price is a step function with ticks of 0.001; over one hour the mid of more than half of the samples does not move at all (Table 3.5). Collapsing "no change" into one of the two directions would make the task ill-posed; the three-class formulation keeps it well posed.
- **The last update before $t+H$, wherever it is.** The update that defines $\text{mid}_m(t+H)$ can live in any hour between t and $t + H$, so the lookup scans every hour of the interval: a market that moved in between and was quiet afterwards is labelled by that move, not CONST.
- **Unknown is unknown.** If $t + H$ is beyond the loaded data, the label is missing, not "whatever the last price was".

Samples. A sample is a (market, block) pair (m, t) in which the market received at least one order-book update in the block (Section 2.4): a prediction is asked whenever something new about a market is observed, and a market whose book did not change in a block has nothing new to read. Every model is asked for a prediction at every such pair, so markets that update often contribute many samples. Because consecutive samples of the same market within a few seconds are almost identical, every result is also reported on a *deduplicated* subset that keeps at most one sample per market per 15 minutes. At the 6h horizon the validation day has 9.3 million samples, 1.2 million after deduplication.

3.6.2 Feature engineering

The sixty-one hand-made features. For every validation sample of the 1h model (same time, market and label), and later for every tensor row, 61 causal features were computed from the clean data: the mid, its changes over 15 minutes and 1, 3, 6 and 12 hours and their signs, and the hour (*price history*, the inputs of the direct path); the spread, the level-1, 5-level and 10-level depth imbalances, the microprice, the sizes and the number of filled levels (*book shape*); the order-flow accumulators, event counts in the last 1, 5, 15 and 60 minutes, the time since the last snapshot and since the last mid change, the number of mid changes and the realised volatility of the last hour, and the position of the mid in its one-hour range (*activity and clock*); the signed taker flow of the on-chain trades over 5, 15 and 60 minutes (*trades*); and the category, the outcome type, the time to the end date, the duration and the age of the market (*static*). The full list, with the exact definition of each feature, is in Appendix A. These features feed the stateless learners of Section 4.3 and, from Section 4.4.1 on, the stateful model as well; Section 3.7.3 studies their structure before any learner is trained on them.

Sibling features. The contracts of the same event form a *market–market* graph; its information is measured through ten causal *sibling features* computed for every row from the last rows of the market's siblings within six hours: how many are active, their mean 15-minute and 1-hour returns, the largest of the latter, the sum of their mid prices (the consistency of the event), the time since the latest sibling move, their net trade flow and order-flow sign. Siblings are the contracts of the same game (from the market's slug) or of the same question template and end date (the candidates of one election, the strikes of one ladder, the two sides of one seat): 9,356 groups of two to sixty markets, covering 74% of the control's rows with about eleven active siblings each. If the trees find nothing in these features, a message-passing layer over the same edges has nothing to carry, and the lead–lag statistic of Section 3.7.1 says why.

News features. The *news flow*: every market is mapped to the entities named in its question and tags (proper-noun phrases with generic words removed; 129 thousand of the 175 thousand markets, 74% of the validation samples), and the GDELT Global Knowledge Graph files, one every fifteen minutes, give for each entity the number of articles naming it and their tone; twelve causal features per row follow (counts over one, six and twenty-four hours, bursts against the previous seven days, tone and its change, share of the news flow, time since the last mention), computed from the files published at least ten minutes before the row. Like the sibling features, the news features are measured in the stateless learner first (Section 5.5.2).

Wallet features. For the graph models, every transaction edge carries a seven-dimensional feature (side, role, signed size, price, fee, token orientation) and, in the enriched variants, a causal record of the wallet's past accuracy at the horizon ("skill"), a placebo skill (another wallet's record), and the wallet's past behaviour (activity, maker share, one-sidedness, size, breadth); Section 4.5.1 says how they enter.

Foundation-model features. Eleven numbers are appended to every window row, derived from a zero-shot Chronos-2 forecast [55] of the market's one-minute mid series: the 10%, 50% and 90% quantiles of the forecast at 15, 30 and 60 minutes ahead, expressed as scaled differences from the current mid, the sign of the 60-minute median change, and an availability flag. Forecasts are computed on a five-minute grid from the 512 minutes of history strictly before the grid time, and a sample at time t reads the last forecast made at most eight minutes before t; relative to the current mid, a slightly stale forecast is "the move still expected from now". The features enter the GRU and the direct path like the lookback features.

3.6.3 Training, validation and test periods

The twenty-one days are split by date. Models are trained on 6–12 March, and every design choice, hyper-parameter and number of Chapter 5 is taken on the validation day, 13 March; the period 24–26 March is a locked test set, never evaluated and never inspected, reserved for the final numbers of the thesis (Section 5.7). The exploratory analysis of Section 3.7 obeys the same split: the price process and the feature space are read on the training and validation days only, and the one part that uses no label and no model, the description of the transaction graph, covers the whole window. The evaluation protocol that goes with the split, the metric, the subsets, the bootstrap intervals and the calibration rule, is given with the experimental setup (Section 5.1).

3.7 Exploratory and statistical analysis

Before any model is trained, the data say a great deal about what can and cannot be predicted. This section reads the three streams with the tools of exploratory analysis and classical data mining, in three parts. The first (Section 3.7.1) is the *price process*: what the stream of book events looks like, how often a price moves over each horizon, how the price level bounds the dynamics, how regular the activity of a market is, what the kind of contract says, whether the contracts of the same event lead one another, and whether the wallets that trade carry information. The second (Section 3.7.2) is the *transaction graph*: how large, how dense and how connected the bipartite graph between wallets and markets is, how it grows, and how its map is organised. The third (Section 3.7.3) is the *feature space*: how many quantities the hand-made features of Section 3.6.2 contain, what organises the space they span, how much of the label is linear, local or made of thresholds, and which families of markets emerge from how they trade. Every number of the first and third parts is computed on the training and validation days of Section 3.6.3, never on the locked test period; the second part is purely descriptive, uses no label, and covers the whole window.

3.7.1 The price process

Before any model is trained, the data say a great deal about what can and cannot be predicted. This section collects the descriptive results that the methodology and the experiments rely on: what the stream of book events looks like, how often a price moves over each horizon, how the price level bounds the dynamics, how regular the activity of a market is, what the kind of contract says, whether the contracts of the same event lead one another, and whether the wallets that trade carry information. Every number is computed on the training and validation days of Section 5.1, never on the locked test period.

The event stream

An audit of one training day (12 March, 325 million events on 23,512 markets) shows how uneven this unit is. Only 4.2% of the events change the mid (1.9% at extreme prices, 5.2% at central ones); 55% add size and 44% remove it. Ninety percent of the events are away from the best quote (median distance 0.03 on the bid side and 0.10 on the ask side), and a full book has a median of about 650 bid and 750 ask levels, so most events never reach the ten stored levels and enter the vector only through the order-flow accumulators. The event clock is irregular: 30% of consecutive events share a timestamp, the median gap is 31 ms and the 90th percentile 4.3 s. One 20-event snapshot therefore spans 12 s in median but 196 s at the 90th percentile, and 73% of the snapshots contain no mid change; a 10-snapshot window spans 2.8 minutes in median and 34 minutes at the 90th percentile, with 44% of the windows containing no mid change. The consequences for the models are discussed in Section 5.2.2.

How often a price moves

The three-class label of Section 3.6.1 is dominated by CONST at every horizon below a few hours (Table 3.5 and Fig. 3.1). Over one minute nine samples in ten do not move; over fifteen minutes seven in ten; over one hour a little more than half; over twelve hours one in three. The two directions are close to balanced at every horizon, with DOWN slightly more frequent than UP at the short ones. Two consequences follow. The first is that the task changes with the horizon: at one minute it is almost entirely "will the mid move at all", and the direction of a move, once it moves, is nearly a coin toss (Section 3.7.1); at twelve hours the direction is the task. The second is that any classifier's balanced accuracy at a short horizon depends

Table 3.5: Share of the three classes among the labelled validation samples of 13 March, by horizon (samples of the stateful models: every market of every block at the block time; 1 h, 6 h and 12 h from the causal-baseline evaluation, 1 and 15 min from the short-horizon runs).

Horizon	DOWN	CONST	UP
1 min	0.057	0.891	0.052
15 min	0.164	0.690	0.146
1 h	0.236	0.549	0.215
6 h	0.335	0.354	0.312
12 h	0.377	0.275	0.348

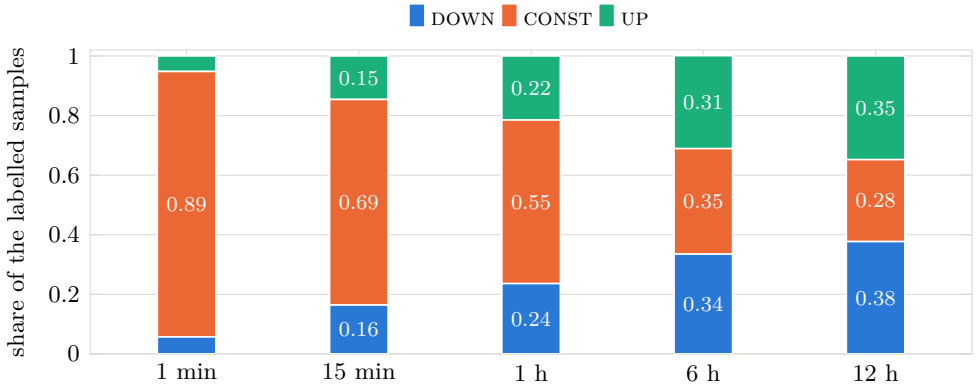


Figure 3.1: Class shares of the validation labels by horizon. Below one hour the task is dominated by CONST; at twelve hours the three classes are nearly balanced.

on where it places its decision thresholds far more than on what it knows: a model that under-calls the moves by a few per cent loses several points of balanced accuracy, which is why every comparison in Section 5.6 is made after the same post-hoc calibration.

The price level

About 40% of the samples have a mid below 0.05 or above 0.95. A Polymarket mid price is a bounded, discrete process observed through a thin book; most of its changes are one-tick moves produced by an order arriving at or leaving the best quote, they are not information and they revert, and a market that has not moved in the last hour is usually illiquid or settled near a boundary and will not move in the next hour either. At extreme prices the process is almost purely a one-tick oscillation: the share of events that change the mid falls from 5.2% at central prices to 1.9% there (Section 3.7.1). Every result of the thesis is therefore reported separately on the *extreme* samples (mid outside $[0.05, 0.95]$) and the *central* ones, and every learned model is weakest, relative to the simple rules, at the extremes. The literature adds that the extremes are also where prices are systematically biased: contracts bought below ten cents lose money on average and contracts above ninety cents earn a little [7], the favourite–longshot bias of Section 2.1, so the price level is not only a bound on the dynamics but a feature in its own right.

The activity clock

The most regular thing about a market is its own rhythm. The time since the mid last changed, the number of mid changes in the last hour and the number of book events in the

last minute are the three quantities that best separate the samples whose mid will move from those whose mid will not. Over a one-hour horizon the time since the last change separates CONST from a move with an area under the ROC curve of 0.85; over one minute the number of changes in the last hour reaches 0.94, the time since the last change 0.91 and the event count of the last minute 0.89 (Section 5.2.2). No quantity of the book's shape comes close. This is the *activity clock*, and it is the first fact any model of this data has to know: a market that has moved often lately will move again, one that has been still will stay still. It is also a fact that the event-clock representation of Section 3.5.2 hides, since a window of twenty book events can be ten seconds or ten minutes old and nothing in the window says which; Sections 5.2 and 5.3 measure what this costs.

The direction of a move

Given that the mid moves, which way is far less predictable. On the samples that moved within one hour, the sign of "mid minus its one-hour mean" gives the direction 57.3% of the time, the level-1 imbalance of the book 55.7%, the microprice 55.5%, the on-chain taker flow 52.6% (on the fifth of the samples with trades), the ten-level imbalance 50.8% and the all-level order-flow accumulators of the tensor 50.0–50.5%. The direction is in the first level of the book and in the position of the mid relative to its recent mean, which is the reversal of Section 3.7.1, and not in the depth of the book; this is the finding of Tran, Kanniainen, and Iosifidis [92] and Kolm, Turiel, and Westray [30] on equities reproduced on a prediction market. Over one minute the best univariate hit rate is 54.1% (the level-1 imbalance), every feature is within four points of a coin toss, and the twelve-hour return is the strongest single feature at twelve hours with a 57.9% directional hit as a reversal sign. The rule that follows from these numbers, "predict that the price will move back against its recent change, and no move if it did not move", is Mean Reversion-H, the reference of every comparison in the thesis (Section 4.2).

The kind of contract

Questions are templated: 3,528 templates among the active markets ("Will the temperature in X be above y on d", "Will X win the Y game", "Bitcoin Up or Down, t to $t + 15$ min"). The template explains 55% of the between-market variance of the probability of a move within one hour, from 4% for House-seat markets to 83% for daily-temperature markets, and a ridge regression on a sentence embedding of the question predicts that probability out of sample with $R^2 = 0.22$, no better than the template means. The contract type therefore conditions the dynamics, as Le [93] also find for the calibration of political markets, but it does so through a handful of categories rather than through the words of the question; the usable form of the text is a category or template indicator, and that is how it enters the models (Section 3.6.2).

Siblings co-move, nobody leads

Contracts of the same event, the two sides of a game or the candidates of an election, are the obvious place to look for one price leading another (Fig. 3.2). Taking the minute changes of the mid of every market (6.65 million changes over 49 million market-minutes of 6–13 March) and, for every change of the more active contract of a sibling pair, counting the changes of the quieter one at lags of -15 to $+15$ minutes against the count expected under independence, two contracts of the same game change in the same minute 2.7 times more often than chance, 2.3 times at one minute of distance and still 1.9 times at fifteen; the contracts of the same election or ladder, 2.0 and 1.5 times. But the histogram is symmetric

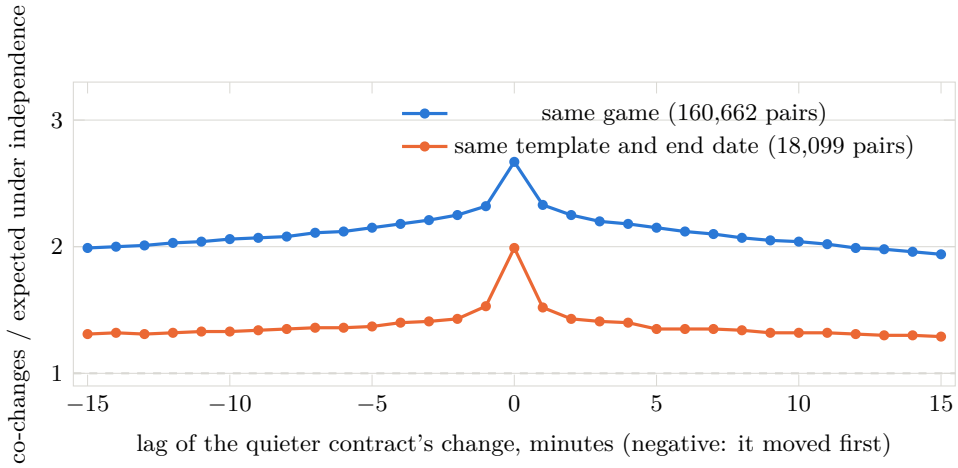


Figure 3.2: Lead–lag between sibling contracts, 6–13 March: for every minute change of the more active contract of a pair, the number of changes of the quieter one at each lag, divided by the number expected under independence. The peak is at lag zero and the two sides are symmetric: siblings co-move, nobody leads.

to the third decimal (asymmetry between $+k$ and $-k$ below 0.01 at every lag): the quieter contract does not follow the active one any more than it precedes it. The common cause, the event on the pitch or the headline, reaches both books within the same minute, which is below the resolution of any horizon this thesis uses. What a sibling did in the last hour is therefore already in the market's own book by the time the model reads it, and the sibling features of Section 5.4.3 lose rather than gain in the trees. The sign structure is weak and, for exclusive events, slightly concordant rather than opposite (43–47% of the simultaneous changes have opposite signs): a part of the co-movement is a market maker re-quoting the same side of several books, not probability mass moving from one candidate to another.

Wallets

Each day 122,000–161,000 distinct wallets trade. A wallet's past accuracy, the share of its trades on the side that the price then moved to over the horizon, is strongly persistent: the Spearman correlation between a wallet's hit rate on the training days and on the validation day is 0.5–0.6. Skill exists at the wallet level. Whether it is information about a *market* is another question: a market is traded by many wallets, their aggregate at any moment is close to that of random wallets with the same activity, and as a market-level predictor the skill of the wallets that traded in the last hour adds about one point over an identity-free placebo and does not beat Mean Reversion (Section 5.4.2). The trade of a skilled wallet is also a book event, seen by the order-flow features before any wallet feature can see it.

3.7.2 The transactions as a graph

Every on-chain fill is a meeting between a wallet and a market, so the transactions table of Section 3.1 is, read as a whole, a bipartite graph: wallets on one side, markets on the other, one edge for every pair that traded at least once, weighted by the number of fill legs between them. The graph is the input of the models of Section 5.4; this section describes it as an object, before any model reads it. The description is purely descriptive, uses no label and no model, and is therefore computed on the whole window of twenty-one days, the locked test days included. The numbers come from the exact graph (every fill leg of the window,

Table 3.6: The wallet–market transaction graph of 6–26 March 2026.

Quantity	Value
fill legs (edge events)	218,882,492 (78,931,041 taker fills)
wallets	646,394
markets	157,776
distinct wallet–market edges	32,273,710
bipartite density	3.2×10^{-4}
wallet degree (markets traded): median / mean / max	4 / 49.9 / 41,767; 22.5% of the wallets have degree 1
market degree (wallets seen): median / mean / max	29 / 204.6 / 16,816; no market below 2
edge multiplicity (fills per pair): median / p99 / max	2 / 107 / 101,367; 47.6% of the edges carry one fill
connected components, whole window	1 (all 804,170 nodes)
k-core: largest core / nodes in it / nodes with core ≥ 10	685 / 8,428 / 339,471
concentration	top 1% of wallets make 77.9% of the fill legs (Gini 0.963) top 1% of markets receive 16.6% (Gini 0.836)

Appendix C); the maps come from a two-dimensional layout described in Section 3.7.2.

Size, density and heavy tails

Table 3.6 gives the graph in numbers and Fig. 3.4(a) its degree distributions. The graph is sparse as a matrix, three edges in ten thousand possible pairs, and dense as a social object: every market has been traded by at least two wallets and by twenty-nine in median, and the whole graph is *one* connected component, with no isolated pair and no small island. Its three distributions have the heavy tails of every interaction network: the median wallet traded four markets and the most active 41,767; the median market was traded by 29 wallets and the most popular by 16,816; the median edge carries two fills and the heaviest a hundred thousand. Concentration is extreme on the wallet side, where one wallet in a hundred makes more than three quarters of the fill legs, and much milder on the market side, where the top one per cent of the markets takes one sixth of the activity: the activity is spread over many markets but made by few hands. The k-core decomposition locates those hands: a core of 8,428 nodes in which every node has at least 685 neighbours inside the core, and a third of all nodes with a coreness of at least ten.

The graph in time

The hourly slices of Fig. 3.4(b–e) show the graph as a process. An average hour has 434 thousand fill legs (79 thousand in the dip of 20 March, 607 thousand at the peak), 23 thousand active wallets, rising from about 20 to about 27 thousand over the month, and 4,200 active markets. The hourly graph has 87 thousand distinct edges, and 74% of them are seen for the first time in the window: most of what happens in an hour is a wallet trading a market it had never traded before, and 1,300 wallets per hour have never been seen at all. The hour-of-day profile in UTC is nearly flat (peak at 14 h, trough at 4 h, ratio 1.32), the signature of a venue whose public is spread over the time zones and whose busiest contracts, the crypto intervals, never close. The hourly graph is not connected, it has 51 components on average (13 to 103), but its giant component holds 99% of the hour's active nodes and never less than 95%: the small components are a few wallets on a few markets that nobody else touched that hour. Cumulatively (Fig. 3.4(f)) the graph never stops growing: 56 components after the first day, 9 after four, 2 after six, one from the eleventh day on, and a straight line of new wallets, new markets and new edges to the end of the window. A model that keeps a state per wallet, as the graph models of Section 5.4 do, therefore meets at every hour a population of which a part has no history at all.

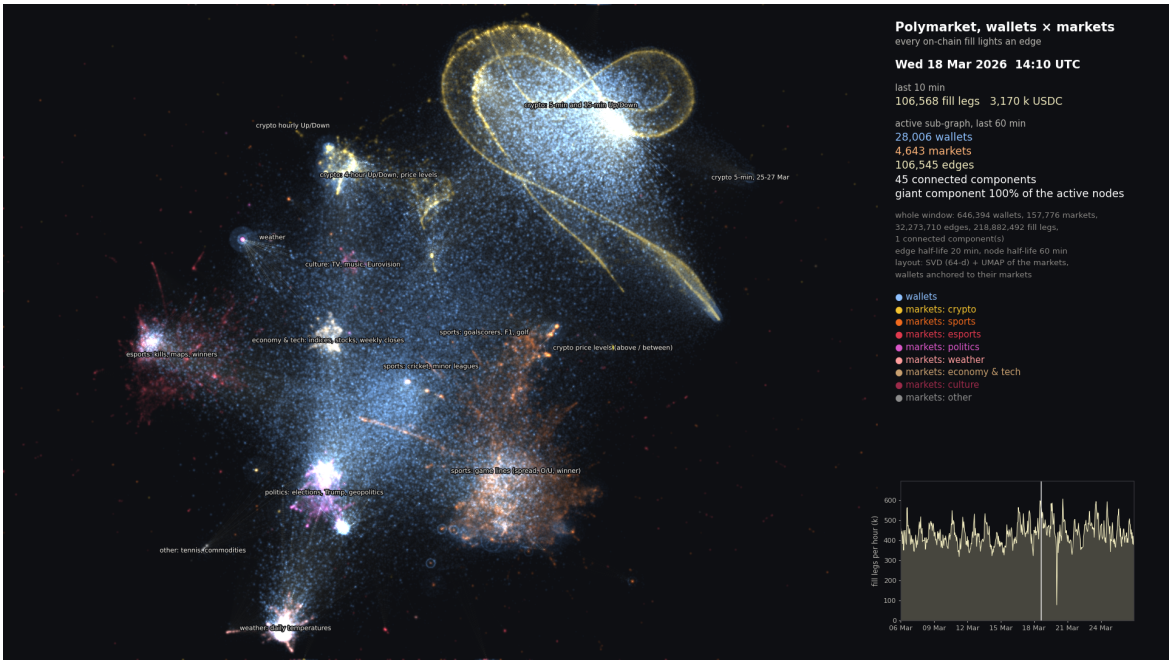


Figure 3.3: One frame of the animation of the transaction graph: Wednesday 18 March 2026, 14:10 UTC. Every fill of the last minutes lights the edge between a wallet and a market and fades with a half-life of twenty minutes; wallets are blue, markets are coloured by category (crypto yellow, sports orange, esports red, politics violet, weather pink, economy and tech brown, culture dark red); the largest islands of the map carry their label. The side panel gives the activity of the bin and the active sub-graph of the last hour. The ribbon at the top right is the crypto five- and fifteen-minute contracts; the bright blobs are the markets whose public is active in those minutes.

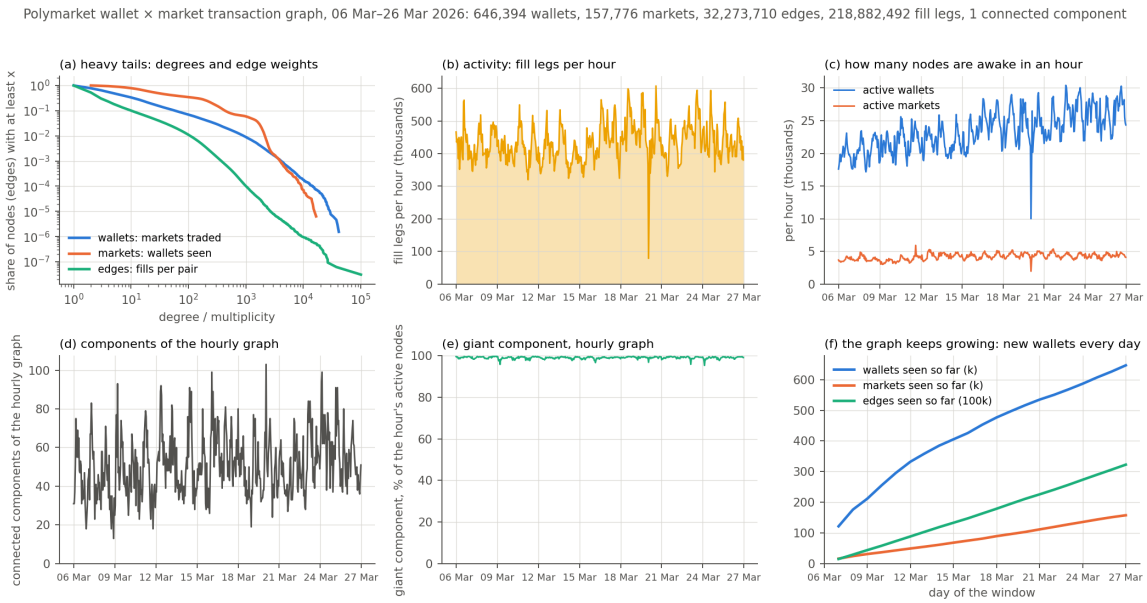
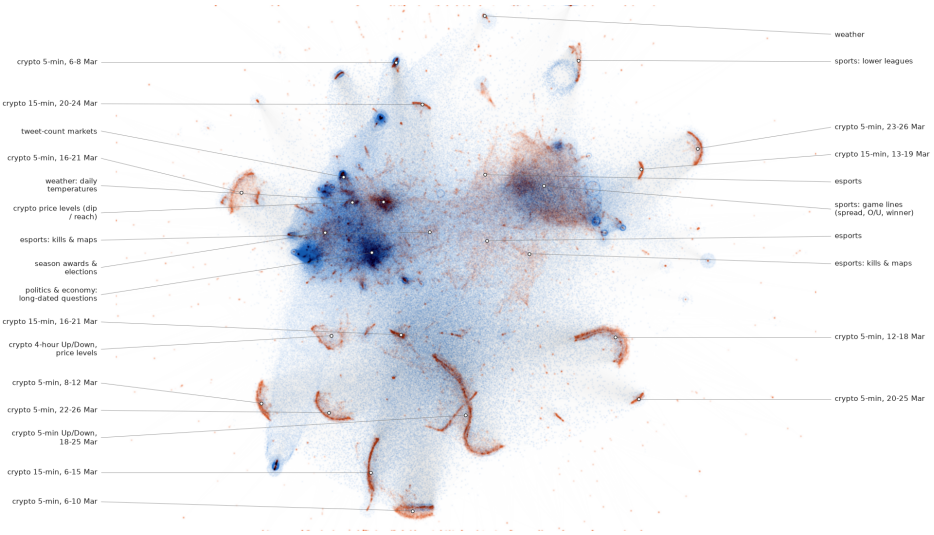


Figure 3.4: The transaction graph in six panels. (a) Complementary cumulative distributions of the wallet degree, the market degree and the edge multiplicity (log–log). (b) Fill legs per hour over the window. (c) Active wallets and active markets per hour. (d) Connected components of the hourly graph. (e) Share of the hour's active nodes in its giant component. (f) Cumulative number of wallets, markets and edges seen since the start of the window.

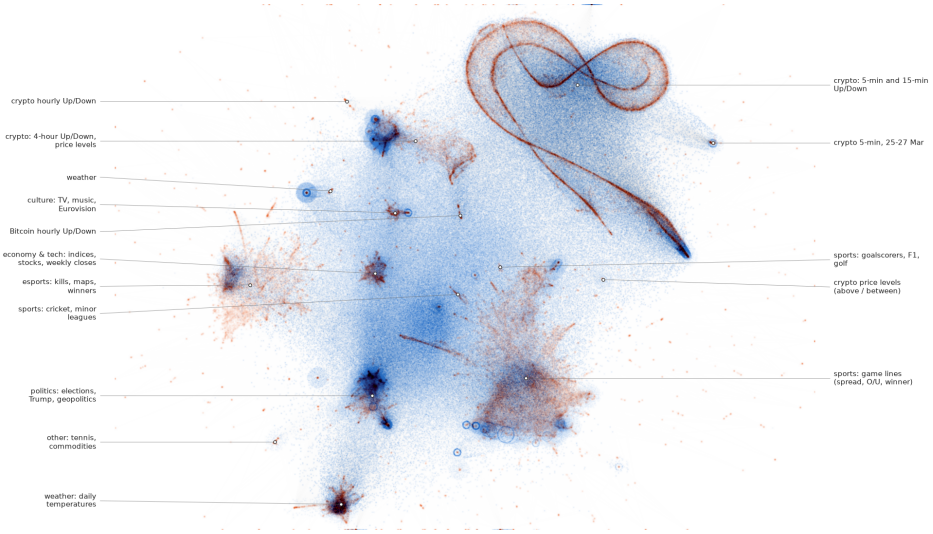
A map of the graph

A graph of 800 thousand nodes and 32 million edges cannot be drawn by a force-directed layout. The maps of Fig. 3.5 use a two-step large-graph technique. First, the biadjacency matrix, with log(1+fills) weights, an inverse-frequency scaling of the market columns and unit-length wallet rows, is reduced by a truncated singular value decomposition to 64 dimensions, which gives wallets and markets a common latent space in which two markets are close when they are traded by the same wallets. Second, the 157,776 markets are mapped to the plane by UMAP [94] on that space (cosine distance, fifteen neighbours), and every wallet is placed at the fill-weighted centroid of the markets it traded, pushed outwards in proportion to how spread those markets are, with a deterministic halo for the wallets that traded a single market. The second map adds to the latent vector of every market a block that encodes its category, so that two markets of different categories are pushed apart by a fixed amount on top of their similarity distance.

The islands of the first map, found by HDBSCAN on the market positions (minimum size 150), are 129 and hold 79% of the markets and 93% of the fills; the islands of the second map are 39 and hold 95% of the markets. Read with the question texts of their markets, the islands say how Polymarket's trading is organised. The first partition is the *category*: sports game lines (spreads, totals, winners) are the largest island of the first map with 24,624 markets, and on the second map every category is a continent of its own, sports with 48 thousand markets, crypto with 44 thousand and 148 million fill legs, two thirds of all the activity of the venue, esports with 19 thousand, then politics, weather, economy and culture. Inside crypto the second partition is the *contract length and the date*: the five- and fifteen-minute Up/Down contracts live a few minutes each, a new one opens as the previous one closes, and the wallets that trade them on a given day form the island of that day, so that the first map has one crypto island per two to five days and the second map a single ribbon whose



(a) Markets placed by trading similarity only.



(b) Markets placed by similarity and category.

Figure 3.5: Two maps of the whole transaction graph (646 thousand wallets in blue, 158 thousand markets in orange, six million of the edges as faint lines). (a) The markets are laid out by UMAP on the trading-similarity space alone; the islands are groups of markets traded by the same wallets, and the crypto five- and fifteen-minute contracts form one island per few days, because the wallets that trade them on a given day are largely the wallets of that day. (b) The category of the market is added to the distance, and the map separates into one region per category, with the date as the internal order of the crypto ribbon. Labels are the largest islands found by HDBSCAN on each map; wallets are placed at the centroid of the markets they trade, so the dense blue regions are the publics of the corresponding markets and the blue rings are the exclusive publics of single markets.

internal order is still the date. The longer crypto contracts (hourly, four-hour, price levels) are separate islands in both maps, as are the daily temperature markets, the tweet-count markets and the long-dated political questions. The organisation of the trading is thus the organisation of Section 3.7.1 seen from the wallets' side: who trades what is decided first by the category and then by the type of contract, and the families of markets defined by how they trade (Section 3.7.3) are a third partition that cuts across both.

Large taker orders

The animation from which Fig. 3.3 is taken shows, now and then, a blue disc lighting up at once: several hundred wallets that traded nothing else in the whole window, all filled within the same minutes. The disc shape is an artefact of the layout, which draws single-market wallets in a halo around their market; the burst itself is the footprint of one very large taker order. An order that crosses the book is filled against the resting orders on the other side, and when the resting side is made of many small orders from many wallets, one transaction creates hundreds of edges in one block. The largest such bursts (575 one-market wallets on a basketball spread in ten minutes on 20 March; 466 on a basketball total; 227 on a recession question) have exactly this anatomy: all the wallets are *makers*, filled by one or a few taker transactions that each consume about a hundred resting orders from a hundred distinct wallets, at the same prices and with a handful of distinct sizes, and most of those wallets have exactly one fill in the whole month. Such transactions are marginal in volume: those with at least fifty maker legs of which four fifths belong to wallets with at most two fills are 28 in the month, 105 thousand USDC of 3.4 billion, on 16 markets; but 20% of all wallets have one or two fills on a single market, so the resting side of a book is often many small orders from wallets that are never seen again, and a wallet identity is a weaker thing than it looks.

What the graph says for the models

Three properties of the graph bear on the models of Section 5.4. The graph is one component with a dense core, so a message-passing model has no locality to exploit: within a few hops everything is reachable from everything. Most wallets are sparse: the median wallet has four markets and a fifth of them one, the wallets of an hour are three quarters new edges and five per cent new nodes, so a per-wallet state is, most of the time, either empty or stale. And the organisation of the trading is by category, contract type and date, which the market's own metadata already encode; what the wallets add is the identity of a public, and Section 3.7.1 has shown that a public's skill, though persistent, is close to that of random wallets with the same activity. The negative result of Section 5.4 is, in this light, a property of the data rather than of the model.

3.7.3 The structure of the feature space

The sixty-one hand-made features of Section 3.6.2 (listed in Appendix A) were built to feed the stateless control of Section 5.2, and the experiments use them only through the accuracy of what is trained on them. They are defined in Section 3.6.2 and taken as given here, because what is asked of them does not need a trained forecaster. This section looks at the features themselves, with the tools of classical data mining, and asks four questions that do not need a trained forecaster: how many independent quantities the features really contain; whether the label is organised geometrically in the feature space, or whether the space is organised by something else; how much of the separability is linear, how much is local and how much needs interactions; and whether the markets form natural families that cut across the categories of

Section 3.7.1. The answers set expectations for every model family and explain, before any training, several of the results of Sections 5.2, 5.3 and 5.6.

Everything in this section uses the samples of the validation day, 13 March, with the training rows of 7–12 March wherever a method is fitted, and never the test set (Section 5.1). The methods are the standard implementations of scikit-learn [95] and umap-learn [94]; the script is `scripts/data_audit/datamining.py` (Appendix C). The horizon of 15 minutes is used as the reference throughout, because its classes are the least unbalanced of the short horizons (Table 3.5); the same study was run at every horizon and the differences are reported where they matter.

Preprocessing

Distance-based methods need the features on a common scale and without the heavy tails that order-book quantities have. Every feature is imputed with its training median and then mapped by rank through the inverse normal distribution (a rank-gauss transform, fitted on one million training rows): a monotone map, so that trees and rules are unaffected, which gives every feature the same marginal distribution. Fifty-nine features enter the analysis: the hand-made features of Section 3.6.2 without the category of the market, one of eight classes derived from the tags that Polymarket attaches to each market (Section 3.2), which is kept as a label to colour by and never as an input, and without the time since the last book event, which is constant on the training rows.

How many quantities the features contain

Principal components. On 200,000 validation samples at 15 min, four principal components hold half of the variance, ten hold 80% and twenty-three hold 95%; the participation ratio of the spectrum, the number of equally loaded components that would give the same spread, is 9.7. The first component (23% of the variance) is the direction of the recent history: it loads on the sign of the last move and on the signs of the 3, 6 and 12-hour changes. The second (14%) is activity: the sixty-minute realised volatility, the number of moves, the dollars traded and the position in the sixty-minute range. The third (10%) opposes the sign of the last move to the long signs, which is a reversal axis, and the fourth (7%) opposes activity to trades. By group, the price and the flow-and-clock features take almost all of the first six components; the book-shape features (G2) take between 0.3% and 9% of each. The book shape is a low-variance direction of the space, because its quantities are bounded ratios, and a method that looks for variance will not find it; whether it carries information is a different question, answered below by the mutual information.

Correlation blocks. Figure 3.6 shows the Spearman correlation matrix. The participation ratio of its spectrum, an effective number of independent features, is 16.8. At $|\rho| \geq 0.7$ the features fall into fifteen blocks of two to five: each return with its sign; the event counts over 1, 5, 15 and 60 minutes with the time since the previous row; the two order-flow imbalances with their standardised copies; the dollars and the number of on-chain trades with the time since the last trade; the sixty-minute number of moves, realised volatility and time since the last move; the imbalances at five and ten levels with the depth skew; the hours to the end with the duration of the market. At $|\rho| \geq 0.9$ fourteen pairs remain. The fifty-nine columns are, in short, about seventeen quantities, and a model that reads all of them is reading each of these quantities three or four times.

Random projection and multidimensional scaling. A Gaussian random projection to twenty dimensions preserves the pairwise distances of 3,000 samples within a ratio of 0.75 to

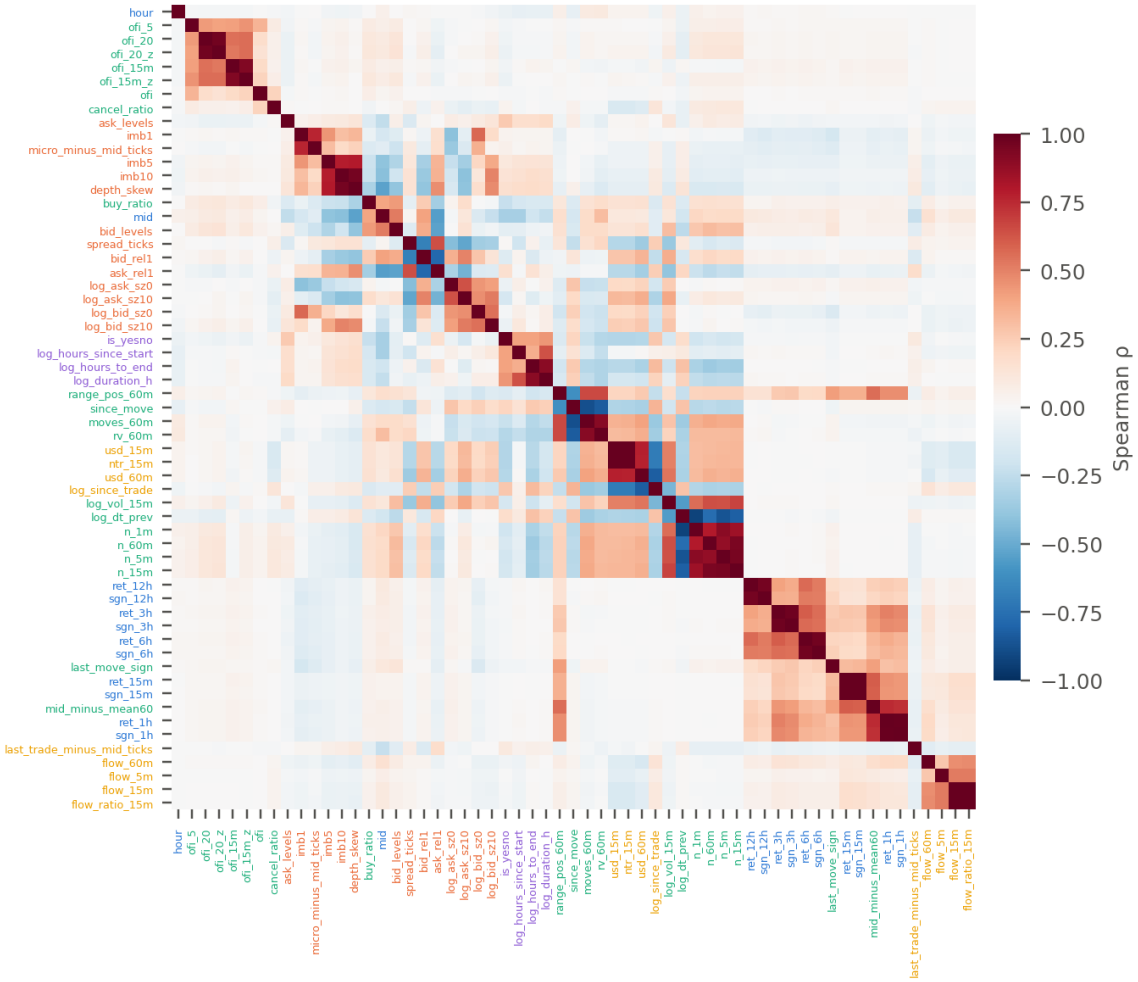


Figure 3.6: Spearman correlation of the fifty-nine features on 200,000 validation samples at 15 min, ordered by average-linkage clustering of $1 - |\rho|$. Tick labels are coloured by group: blue G1 price, orange G2 book, aqua G3 flow and clock, yellow G4 trades, violet G5 static. The blocks are the returns with their signs, the event counts, the order-flow imbalances with their standardised copies, the on-chain trade quantities, and the sixty-minute activity triple.

1.36 (5th and 95th percentile; standard deviation 0.18), and to forty dimensions within 0.82 to 1.18; a nearest-neighbour classifier on the twenty-dimensional projection loses 0.65 points of balanced accuracy against the same classifier on the full space (Section 3.7.3). The intrinsic dimension is therefore well below fifty-nine but well above two: a metric multidimensional scaling of 2,000 samples to the plane has a Kruskal stress of 0.31, against 0.48 for the first two principal components on the same rows, and both are far from a faithful planar map.

Maps of the samples

Figure 3.7 is a UMAP of 50,000 samples and Table 3.7 measures what the map, and the original space, are organised by: for each sample, the share of its fifteen nearest neighbours that carry the same label, divided by the share expected by chance. The lift of the prediction label is 1.24 in the map and 1.26 in the fifty-nine-dimensional space: the neighbours of a sample that is about to move are only a quarter more likely than chance to move as well. The lift of the activity quartile is 1.7 to 1.9, that of the price-level bucket 1.4 to 1.6, that

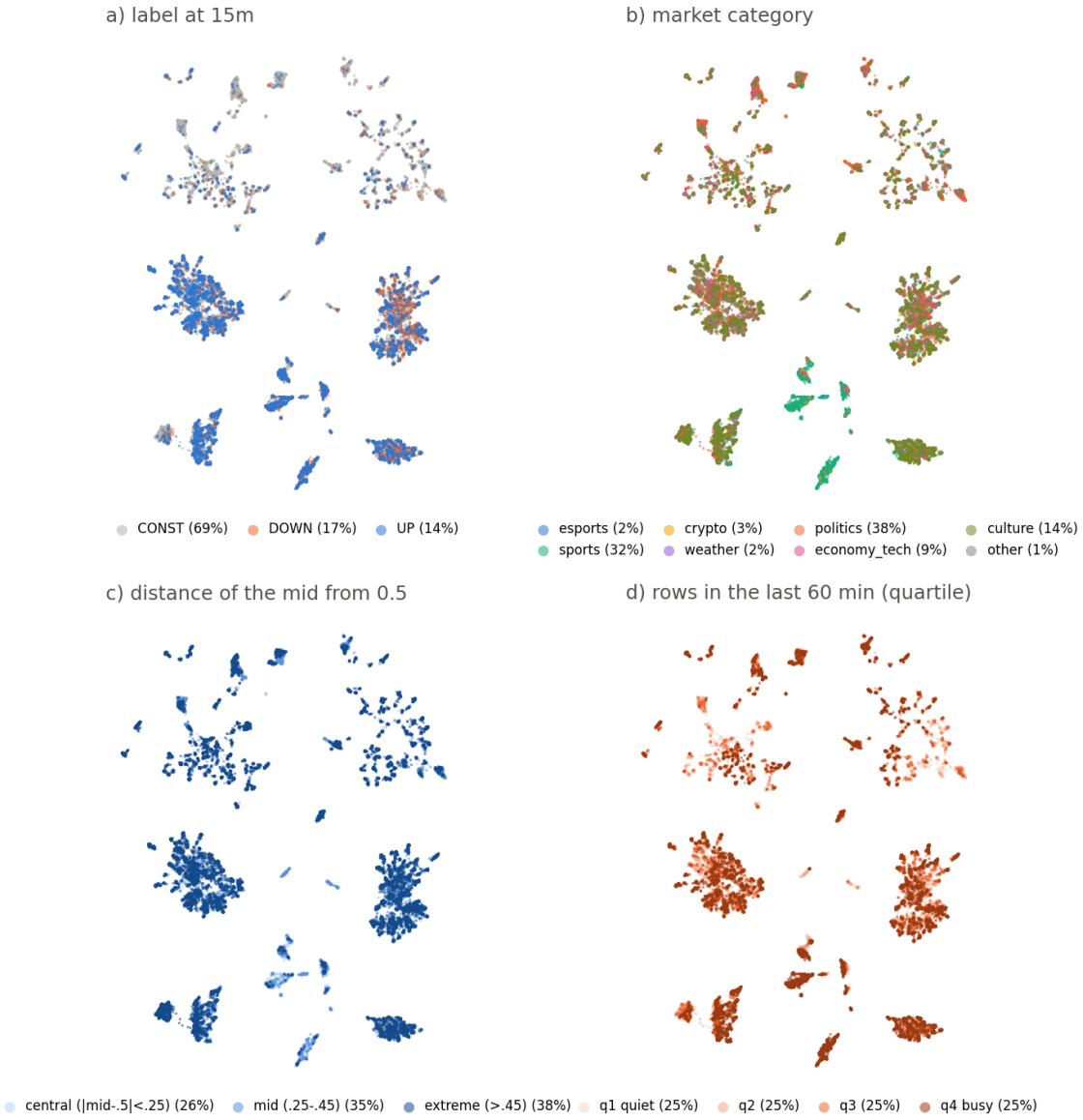


Figure 3.7: UMAP of 50,000 validation samples at 15 min (fifty-nine rank-gauss features, thirty neighbours), the same points coloured four ways: (a) the label at 15 min, (b) the category of the market, (c) the distance of the mid from one half, (d) the quartile of the number of book rows in the last sixty minutes. The islands are coherent in activity and, largely, in category; the label is spread inside each island.

Table 3.7: Neighbourhood purity in the maps and in the original space at 15 min: share of the fifteen nearest neighbours with the same label, the share expected by chance ($\sum_c p_c^2$) and their ratio. UMAP on 50,000 samples, t-SNE on 20,000.

	UMAP			original space		
Label	same	chance	lift	same	chance	lift
prediction label (3 classes)	0.649	0.524	1.24	0.662	0.524	1.26
market category (8)	0.404	0.273	1.48	0.462	0.273	1.69
price-level bucket (3)	0.488	0.341	1.43	0.556	0.341	1.63
activity quartile (4)	0.423	0.250	1.69	0.487	0.250	1.95
market identity (11,294)	0.026	0.0003	104	0.052	0.0003	202

Table 3.8: Separability of the three classes at 15 min on the validation day, balanced accuracy on all and on deduplicated samples. Linear models and trees are fitted on one million training rows; kNN on the same rows and evaluated on 300,000 validation samples, with the vote of the neighbours weighted by the inverse class frequency. LightGBM is the G5 control of Section 5.2, uncalibrated.

Model	all	dedup
Mean Reversion-H	0.537	0.525
LDA, uniform priors	0.561	0.495
Logistic regression, balanced weights	0.568	0.505
Decision tree, depth 3, balanced	0.548	0.549
Decision tree, depth 6, balanced	0.571	0.562
kNN, $k = 5$	0.532	0.517
kNN, $k = 50$	0.570	0.551
kNN, $k = 200$	0.574	0.553
kNN, $k = 1000$	0.572	0.551
kNN, $k = 50$, 20-D random projection	0.564	0.547
LightGBM G5 (control)	0.616	0.600

of the category 1.5 to 1.7, and that of the market identity is 100 to 200: one neighbour in twenty comes from the same market, out of eleven thousand. The feature space is organised by the market and by its activity, not by what the price is about to do; the label is a weak modulation on top of a geometry made of market regimes. UMAP and t-SNE keep between 85% and 95% of the lifts of the original space, so the maps are faithful to this extent, and the two methods agree with each other on every count.

The picture explains two results of the experiments. The same-market lift is why the per-market state of Section 5.3 has something to learn, and why deduplication changes every number: consecutive samples of a market are neighbours in the feature space and carry nearly the same information. The weak label lift is why no family reaches balanced accuracies far above 0.6 at this horizon: in the space of what is observable at the sample time, samples that will move and samples that will not are mixed at the scale of fifteen neighbours.

Linear, local and interaction separability

Table 3.8 places the classical learners between Mean Reversion and the control. Three things stand out.

The linear signal is "whether", and it lives in the busy markets. Linear discriminant analysis and a balanced logistic regression gain 2.3 and 3.0 points over Mean Reversion on

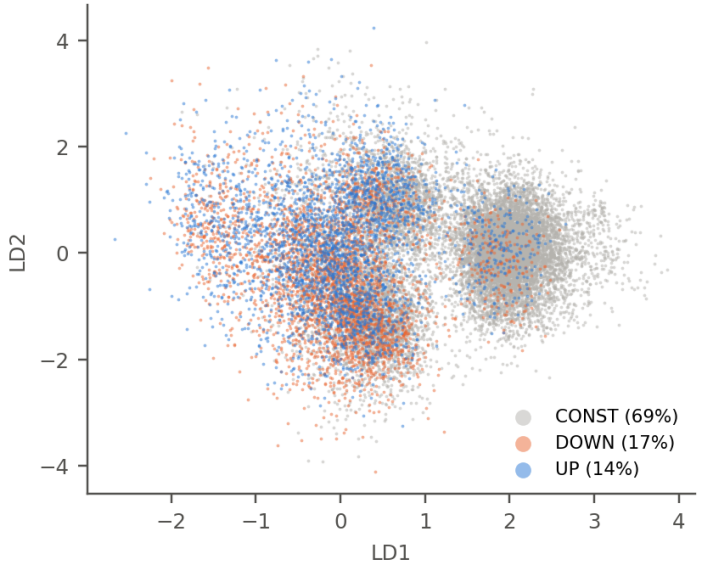


Figure 3.8: The two discriminant directions of LDA at 15 min on 20,000 validation samples. LD1 separates CONST (right) from the moving classes (left); the two directions overlap.

all samples and *lose* 3 and 2 points on the deduplicated samples. The leading discriminant direction is made of the number of trades and of book events in the last minutes, the time since the last move and the number of moves in the hour, with the sign structure of an activity axis: it separates CONST from the two moving classes, and it does so on the stream of consecutive samples of busy markets, which deduplication removes. Figure 3.8 shows it: the CONST samples form a cloud on the right of LD1, the moving samples a wider cloud on the left, and DOWN and UP overlap almost completely. A linear function of the features cannot tell the direction; it can tell, imperfectly, the activity.

Thresholds and a few interactions do most of the work. A depth-three tree, twelve numbers in all, reaches 0.548 on all samples and 0.549 deduplicated, above every linear model on the deduplicated samples; a depth-six tree reaches 0.571 and 0.562. The depth-three tree is worth reading in full, because it is the simplest statement of what the short-horizon problem is:

```
since_move ≤ 1048 s (the price moved in the last 17 minutes)
  moves_60m ≤ 8.5:  last_move_sign ≤ 0 → UP, else DOWN
  moves_60m > 8.5:  since_move ≤ 121 s → UP, else DOWN
since_move > 1048 s → CONST on every branch
```

The first split is the activity clock of Section 3.7.1: if the price has not moved in the last seventeen minutes, predict that it will not move. Below it, in the quieter markets, the direction is the reversal of the last move, which is Mean Reversion on the last tick; in the busier markets the tree uses the recency of the last move instead, a split that balanced weighting rewards and that the full learners do not keep. The gap from this tree to the control, 6.8 points on all samples and 5.1 deduplicated, is what six hundred boosted trees of sixty-three leaves add: finer thresholds on the same quantities and interactions between the activity, the price level and the book.

Local information is there but it is not enough. A nearest-neighbour classifier with a balanced vote reaches 0.574 at $k = 200$ and is flat between $k = 50$ and $k = 1000$; with a

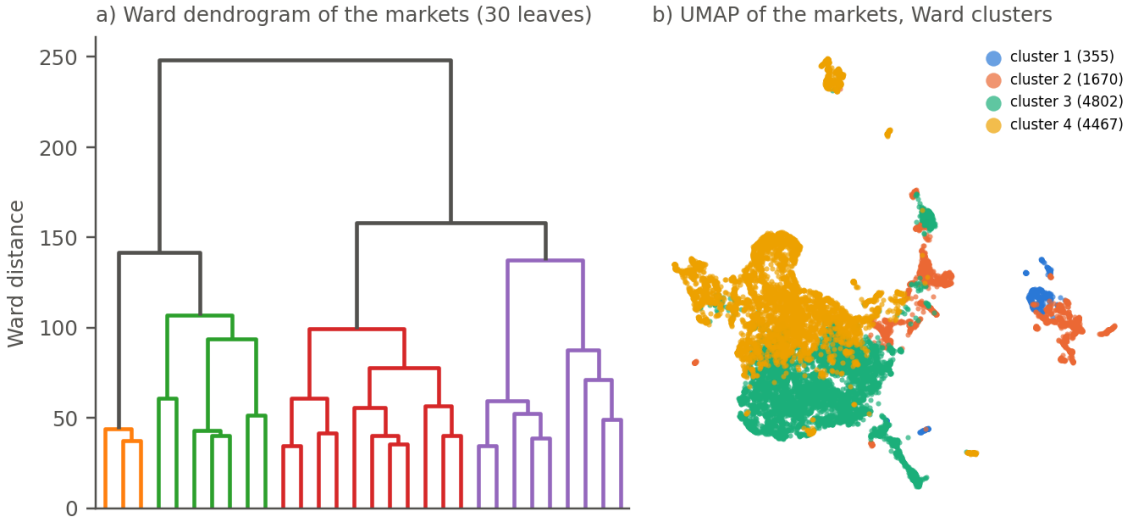


Figure 3.9: Clustering of the 11,294 markets with at least a hundred validation samples at 15 min, on thirteen standardised per-market aggregates (activity, spread, depth, realised volatility, distance of the mid from one half, share of moving samples, duration, time to the end, dollars traded, cancellation ratio, contract type). (a) Ward dendrogram cut at four clusters; (b) UMAP of the markets coloured by cluster.

plain majority vote it is at 0.45–0.49, below Mean Reversion, because the CONST class takes every neighbourhood. Against the control on the same 300,000 samples the best kNN is −4.0 points [−4.3, −3.8]. The feature space has usable local structure, but a local average cannot represent the thresholds that the trees find, and the optimum at a large k says that the structure is smooth rather than clustered.

Where the information is, feature by feature. The mutual information of each feature with the label, on 300,000 samples, ranks the sixty-minute activity first: realised volatility (0.229 nats, against a label entropy of 0.834), number of moves (0.221), time since the last move (0.210), then the distance from the sixty-minute mean (0.187), the position in the range (0.182), the microprice offset (0.166), the 15-minute and 1-hour returns (0.162, 0.158), the duration of the market (0.148) and the first-level imbalance (0.142). By group, flow and clock sum to 1.46 nats, price 1.16, book 1.10, trades 0.75, static fields 0.37. The book-shape group, invisible to the principal components, is third in information: its features vary little but what varies is informative, which is the usual reason why variance-based reduction and supervised importance disagree.

Families of markets

The markets are the natural unit of this data and Section 3.7.1 grouped them by their tags. Figure 3.9 groups them instead by how they trade, on thirteen per-market aggregates of the validation day, with Ward's hierarchical clustering; k-means and HDBSCAN on a five-dimensional UMAP give the same families. The silhouette is highest at two clusters (0.36) and flat afterwards (0.13–0.16): the first split is sharp and the rest is a continuum. The four-cluster cut of Table 3.9 is the readable one.

Cluster 1 is the live games: 355 markets, almost all sports and esports, one hour from their end, three-tick spreads, three hundred book rows and four hundred dollars an hour, and a price that moves in 82% of the 15-minute windows. Cluster 2 is the wide-spread markets:

Table 3.9: The four Ward clusters of the markets at 15 min: profile (means of the raw aggregates; the logged quantities are back-transformed), composition by category, and balanced accuracy of Mean Reversion and of the control inside the cluster.

	1 live games	2 wide-spread	3 long-dated, mid	4 long-dated, tight
markets	355	1,670	4,802	4,467
samples (thousands)	748	753	6,333	1,891
median spread (ticks)	3.5	33.4	9.4	0.9
rows in 60 min (geometric mean)	320	25	50	16
share of moving samples	0.82	0.42	0.27	0.10
hours to the end (median)	1.1	25	1,100	2,300
dollars in 60 min (geometric mean)	460	2	0.7	1.5
Yes/No contracts	15%	59%	100%	100%
sports / esports	288 / 52	859 / 153	1,357 / 21	1,731 / 39
politics	3	120	2,213	1,857
economy and tech / crypto	1 / 2	129 / 128	569 / 174	339 / 278
weather / culture / other	0 / 9 / 0	179 / 94 / 8	46 / 360 / 62	17 / 184 / 22
Mean Reversion-H	0.513	0.511	0.524	0.529
LightGBM G5	0.550	0.586	0.606	0.612

1,670 contracts with a median spread of thirty-three ticks, a day from the end, a mix of sports, weather, esports, crypto and economy, with almost no on-chain trading. Clusters 3 and 4 are the long-dated markets, political and economic and the pre-game sports contracts, which differ by their spread: nine ticks against one, and by how often they move: 27% against 10%. The adjusted mutual information between the clustering and the category is 0.08 (Ward), 0.07 (k-means) and 0.10 (HDBSCAN): the families cut across the categories, and sports in particular is present in all four. How a market trades is a different partition from what it is about.

The last two rows are the point of the exercise. Mean Reversion is within two points of the same value in every family, but the control is not: 0.550 in the live games, 0.586 in the wide-spread markets, 0.606 and 0.612 in the long-dated ones. Predictability at 15 minutes grows as activity falls, and the live games, where almost every window moves and the direction is the whole problem, are the hardest family even though they are the richest in data. This is the per-category table of Section 5.6.5 seen from the other side: what makes sports hard is not the sport, it is the live-game regime, and the pre-game sports contracts in clusters 3 and 4 are as predictable as politics.

Across the horizons

The study was repeated at every horizon on the samples of the corresponding runs. What depends on the features alone does not change: at every horizon four, ten and 22–23 components hold 50, 80 and 95% of the variance, the effective number of features is 16.8–18.3, the same fifteen or sixteen correlation blocks appear, a random projection to twenty dimensions keeps the distances within the same spread and the planar MDS has the same stress of 0.30. The dimensionality is a property of the feature set and of the validation day, not of the label. What depends on the label changes in three ways.

The label becomes geometric as the horizon grows. Table 3.10 repeats the purity of Table 3.7 at every horizon. The lifts of the category, the price level, the activity and the market identity are the same at every horizon, as they must be, since the samples are nearly the same. The lift of the label is 1.06 at 1 min, 1.26 at 15 min, 1.39 at 1 h and 1.42–1.45 at 6 and 12 h. At one minute the neighbours of a sample carry almost no information on its label:

Table 3.10: Neighbourhood purity lift (same label among the fifteen nearest neighbours over chance) in the original space / in the UMAP, by horizon. Only the prediction label depends on the horizon.

Lift of	1 min	15 min	1 h	6 h	12 h
prediction label	1.06 / 1.05	1.26 / 1.24	1.39 / 1.36	1.45 / 1.39	1.42 / 1.35
market category	1.68 / 1.48	1.69 / 1.48	1.69 / 1.48	1.64 / 1.41	1.64 / 1.41
price-level bucket	1.63 / 1.43	1.63 / 1.43	1.64 / 1.44	1.61 / 1.41	1.60 / 1.38
activity quartile	1.90 / 1.66	1.95 / 1.69	1.95 / 1.70	1.95 / 1.66	1.96 / 1.66
market identity	216 / 114	202 / 104	202 / 106	198 / 101	200 / 102

Table 3.11: Separability by horizon: balanced accuracy on all and deduplicated samples of the same learners as Table 3.8. kNN with the best k on 300,000 samples; Δ is the difference from the control on the same samples, hour-block bootstrap 95% interval.

	1 min		15 min		1 h		6 h		12 h	
Model	all	dedup	all	dedup	all	dedup	all	dedup	all	dedup
Mean Reversion-H	0.536	0.501	0.537	0.525	0.531	0.518	0.507	0.500	0.507	0.500
LDA	0.462	0.362	0.561	0.495	0.563	0.546	0.541	0.517	0.539	0.516
Logistic regression	0.504	0.376	0.568	0.505	0.562	0.525	0.544	0.518	0.543	0.519
Decision tree, depth 3	0.558	0.474	0.548	0.549	0.534	0.503	0.513	0.492	0.499	0.497
Decision tree, depth 6	0.575	0.479	0.571	0.562	0.554	0.535	0.531	0.512	0.531	0.523
kNN, best k	0.508	0.400	0.574	0.553	0.566	0.549	0.543	0.528	0.546	0.533
best k	20		200		200		1000		1000	
Δ vs control	−10.2 [−11.2, −9.5]		−4.0 [−4.3, −3.8]		−3.6 [−4.0, −3.2]		−3.0 [−3.3, −2.6]		−2.5 [−2.8, −2.2]	
LightGBM G5 (control)	0.609	0.474	0.616	0.600	0.602	0.583	0.572	0.567	0.572	0.568

whether the price moves in the next minute is a matter of the next book event, not of where the sample sits in the feature space. At twelve hours the neighbourhood is informative, and the information is about the direction, because the CONST class has become a third of the samples.

The short horizons are a threshold problem, the long ones a linear one. Table 3.11 and Fig. 3.10 order the simple learners differently at each end of the range. At 1 min the linear models are *below* Mean Reversion (0.462 and 0.504 against 0.536) and the nearest-neighbour classifier reaches 0.508 at its best $k = 20$, ten points under the control; the only simple learners that work are the trees, and a depth-three tree with its first split at "no move in the last 105 seconds" is at 0.558, one point from the depth-six tree and five from the control. Every learner, the control included, falls to 0.47–0.48 on the deduplicated samples at this horizon. At 15 min the trees are still ahead of the linear and local learners. At 1 h the order reverses: linear discriminant analysis, at 0.563 on all samples and 0.546 deduplicated, is the best simple learner, above the depth-six tree (0.554 / 0.535) and level with kNN, and it matches to the third decimal the LOB-only stateful model with the time-first encoder of mid-October, evaluated on the same samples (0.564 / 0.541, Section 5.3.4). At 6 and 12 h the linear models and kNN are together at 0.54, the trees below them, and the margin of the control shrinks to 2.5–3 points. The gap between the control and the best simple learner goes from ten points at 1 min to two and a half at 12 h: the boosted trees earn their advantage where the problem is made of thresholds on the activity clock, and the threshold itself scales with the horizon: the first split of the depth-three tree is at 105 seconds since the last move at 1 min, 17 minutes at 15 min, 3.1 hours at 6 h and 5.5 hours at 12 h.

The information moves from the clock to the book. Table 3.12 is the clearest statement of the horizon dependence in the thesis. At 1 and 15 min the most informative

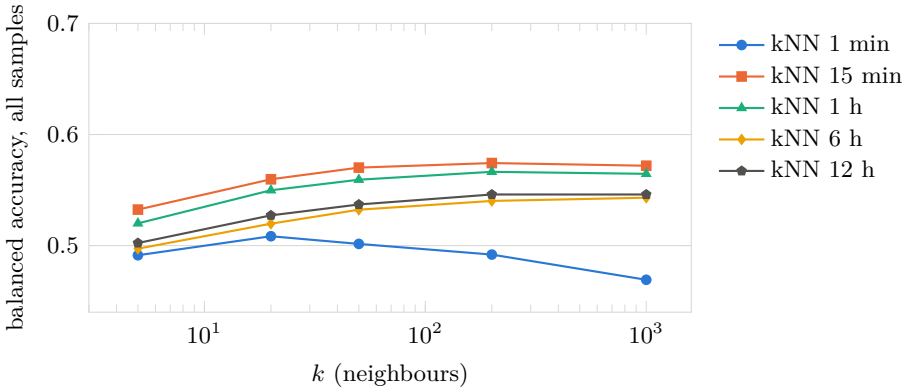


Figure 3.10: Balanced accuracy of the balanced-vote kNN against k at every horizon, on 300,000 validation samples. The optimum moves from $k = 20$ at 1 min to $k = 1000$ at 6 and 12 h.

features are the activity clock: the sixty-minute number of moves, the realised volatility, the time since the last move, the position in the range; the flow-and-clock group sums to 1.15 and 1.45 nats and the book-shape group to 0.38 and 1.10. From 1 h the order turns: the microprice offset and the first-level imbalance lead, followed by the sizes at the first level, and the book-shape group sums to 1.68 nats at 1 h, 2.51 at 6 h and 2.81 at 12 h, while the clock falls to 0.73. The price group is stable at 1.1–1.3 nats and the duration of the market is in the top six from 6 h onward. The reading is the one of Section 3.7.1: at short horizons the question is whether the price will move and the clock answers it; at long horizons almost every price moves, the question is the direction, and the shape of the book, which is strongly conditioned by the price level (Section 3.7.1), is what carries it. The same features, the same day: what the label asks of them changes with H, and so does which model family can read them.

The families of markets, by horizon. The clustering of Section 3.7.3 was repeated on the samples of each horizon; since the share of moving samples is one of the aggregates, the partition is re-derived each time and the clusters are not identical across horizons. The live-game family is recovered at 1 min (304 markets), 15 min (355) and 1 h (356), always sports and esports an hour from the end; at 6 and 12 h it merges with the wide-spread sports contracts into a family of 648 and 566 markets. The control inside the live games is 0.573 at 1 min, 0.550 at 15 min, 0.548 at 1 h, and 0.476 and 0.488 at 6 and 12 h, where Mean Reversion in the same family is at 0.42 and 0.38: in a game that is ending, the price trends, and the reversal rule is wrong more often than right. At the other end, a family of deep and tight long-dated markets (368 markets at 1 min, 527 at 12 h; spreads under one tick, first-level depth of thousands of shares) is the *worst* family for the control at 1 min, 0.412 against 0.527 for Mean Reversion, because its price moves in 2% of the minutes and the control cannot pick them, and the *best* family at 12 h, 0.614: the deep markets are the ones whose direction over a day is readable in the book. The adjusted mutual information between the families and the category stays between 0.04 and 0.10 at every horizon.

3.7.4 What the data say before a model is trained

The findings of this chapter set the agenda of the methodology and the experiments and are, in the terms of Section 1.3, a first answer to the first research question.

Table 3.12: Mutual information between a feature and the label, in nats, by horizon, for the fifteen features that are among the six most informative at some horizon (bold where they are), and the sum over each group. 300,000 validation samples, nearest-neighbour estimator. Label entropy 0.42, 0.83, 1.00, 1.10, 1.09 nats.

Feature (group)	1 min	15 min	1 h	6 h	12 h
`moves_60m` (G3)	**0.149**	**0.221**	**0.212**	0.135	0.104
`rv_60m` (G3)	**0.142**	**0.229**	**0.225**	0.154	0.122
`since_move` (G3)	**0.123**	**0.209**	**0.211**	0.167	0.153
`range_pos_60m` (G3)	**0.132**	**0.182**	0.171	0.110	0.090
`mid_minus_mean60` (G3)	0.104	**0.187**	**0.190**	0.127	0.099
`usd_15m` (G4)	**0.112**	0.114	0.099	0.062	0.055
`ntr_15m` (G4)	**0.105**	0.088	0.070	0.032	0.028
`ret_15m` (G1)	0.097	0.162	0.143	0.080	0.060
`ret_1h` (G1)	0.087	0.158	0.165	0.112	0.089
`micro_minus_mid_ticks` (G2)	0.044	**0.166**	**0.270**	**0.410**	**0.457**
`imb1` (G2)	0.041	0.143	**0.233**	**0.362**	**0.408**
`log_bid_sz0` (G2)	0.039	0.116	0.184	**0.290**	**0.334**
`log_ask_sz0` (G2)	0.035	0.100	0.160	**0.261**	**0.303**
`log_bid_sz10` (G2)	0.024	0.085	0.139	**0.217**	**0.242**
`log_duration_h` (G5)	0.095	0.148	0.178	**0.223**	**0.262**
sum over G1 (price)	0.63	1.16	1.31	1.16	1.10
sum over G2 (book)	0.38	1.10	1.68	2.51	2.81
sum over G3 (flow and clock)	1.15	1.45	1.38	0.87	0.73
sum over G4 (trades)	0.60	0.75	0.76	0.51	0.46
sum over G5 (static)	0.28	0.37	0.42	0.45	0.49

The price process. Predictability at short horizons is first of all *whether* the price moves, and that is written in the market's own recent activity; the direction, when it moves, is weakly in the first level of the book and in the reversal of the recent change; the price level and the kind of contract condition everything; the neighbours of a market, whether the wallets that trade it or the contracts of its event, do not carry information that the market's own book does not already show. A model that reads the activity clock, the first level, the long returns together with the price level and the contract type has most of what there is; the question for the stateful and graph families is whether a state or a neighbourhood adds anything on top, and the answer turns out to depend on the horizon.

The graph. The transaction graph is one connected component with a dense core and a long tail of sparse wallets; its organisation is by category, contract type and date, which the market's metadata already encode; a wallet's identity is often a single fill. A model with a state per wallet will find most states empty, and a message-passing model will find no locality.

The feature space. The features are about seventeen quantities read several times, so the capacity of a learner is not the issue and redundancy is not a problem for trees but is one for distance-based and linear methods. The space is organised by market and by activity, not by the label, which is why a per-market state has material to work with and why deduplication is a necessary second view of every result. The separable part of the label is first of all *whether* the price moves, which is linear and local; the direction is neither, and it is where the boosted trees earn their margin over every simpler learner and where, in Section 5.3, the stateful model earns its margin over the trees at the shortest horizon. The information that the label asks of the features moves with the horizon, from the activity clock at minutes

to the shape of the book at hours, and the learner that reads it best moves with it, from thresholds to linear directions. And the families of markets defined by how they trade are not the categories: the live-game regime, not the sports category, is the hard case, and any model that is to be judged by category should also be judged by regime.

Chapter 4

Methodology

This chapter sets out the rationale of the study and the approaches that were compared. Section 4.1 states the objective, the criteria by which an approach is judged and the rules that decided which approaches were included and which were left out. Sections 4.2 to 4.6 then describe the five families as they were built: causal rules on the price; stateless learners on hand-made features; sequence models of the order-book stream, a continuous-time stateful model built for this thesis, the same model given the trees as a base, and a pretrained forecaster; graph models on the wallet–market and market–market graphs; and sources of information outside the book. Each family closes with its advantages and limits a priori, and Section 4.7 puts them side by side; what each did on the data is the subject of Chapter 5. All of them read the inputs of Section 3.6 and are evaluated under the protocol of Section 5.1.

4.1 Objective, criteria and rules of inclusion

4.1.1 Objective

The object of the study is the direction of the mid price of a Polymarket market over a horizon H, read as the three-class label of Section 3.6.1, DOWN, CONST or UP, at every block in which the market updated its book, for five horizons from one minute to twelve hours. The task serves the four questions of Section 1.3: the exploratory analysis of Section 3.7 answers the first, and the comparison of approaches built here is designed to answer the other three, which publicly observable information predicts the direction, which families of models can read it and what a sequential state or a neighbourhood is worth over a stateless learner on the same inputs, and how the answers change with the horizon. The task is the prediction of the *price while the market is open*, not of the outcome of the event; the measure is balanced accuracy, because the three classes are of very unequal size and change size with the horizon (Section 2.9); and the unit of every comparison is the paired difference between two models on the same samples, with an interval.

4.1.2 Criteria of success

A model is said to read information that another does not when its balanced accuracy on the validation day exceeds the other's *on the same samples*, after the same post-hoc calibration, by a margin whose 95% hour-block bootstrap interval excludes zero, both on the full stream and on a deduplicated subset of independent samples (Section 5.1). The reference competitor is first the best causal rule found on the data, "predict that the price will move back against its recent change", and then, once it exists, the best stateless learner on hand-made features. The final numbers on a locked test period are produced once, after every choice has been made (Section 5.7). A difference whose interval includes zero is reported as a tie, and a result

that holds on one of the two views of the samples only, the full stream or the deduplicated subset, is reported as such.

4.1.3 Rules of inclusion and exclusion

Six rules decided which approaches were built and which were left out.

1. **Causal inputs only.** An approach is admissible only if every input it reads at time t is computed from data strictly before t and every label from data after it. Approaches that would need the outcome, the resolution time or the last trade of a market as an input are excluded; the last-trade time defines the sample population and is never an input (Section 3.5.2).
2. **Public information that the dataset contains.** The inputs are the three streams and the public market metadata, plus two public external sources that could be aligned to them, the news flow of the GDELT project and a pretrained time-series forecaster. External reference prices, the spot price of Bitcoin for the crypto contracts or the score of the game for in-play sports, are not in the data; their absence is measured in the results by kind of market (Section 5.6.5), not modelled.
3. **One model for all markets.** Every learned approach is one model pooled over all markets, never one model per market: the predictive relation between a book and its next move is to a large extent universal (Section 2.3), and most markets have too few samples of their own.
4. **Every approach is measured against the best simpler one on the same inputs.** A learned model is admitted only if it can be compared, on the same samples and with the same calibration, with the best causal rule and, once it exists, with the best stateless learner; where possible it contains the rule as a special case (Section 4.4.1), and where a deep model and a tabular learner read the same table, a second tabular learner is trained to separate the effect of the learner from that of the training regime.
5. **Families that span the questions.** The families were chosen so that each question of Section 1.3 has a paired comparison: a learner with a sequential state against one without, on the same inputs; the same stateful model with and without the transaction edges; the same stateless learner with and without each external source. Approaches that would not add such a comparison are outside the scope: outcome forecasting with language models, the subject of most prior work on Polymarket (Section 2.2.5), trading strategies and their profitability, and per-market models.
6. **A fixed compute budget.** A full-length run of the stateful model takes nine to seventeen hours on one GPU of a shared server, one run per GPU (Appendix B). Variants were therefore screened with two-day short tests and confirmed with full-length runs, every claim rests on the latter, hyper-parameters were set by a small number of screening runs rather than by a systematic search, and the combinations that the budget did not cover (the residual construction at fifteen minutes and at the long horizons, the hand-made features at six and twelve hours, a wallet memory at a slower time scale) are reported as not run rather than guessed.

4.1.4 Principles

Three methodological principles run through the thesis.

1. **Causality first.** Every feature and every label is computed from data strictly before the prediction time, and the evaluation protocol is chronological.
2. **Strong baselines, the same calibration for everybody.** The reference competitor is not a random classifier but the best simple rule found on the data and, once it exists, the best stateless learner on the same inputs. Every model is reported as a difference

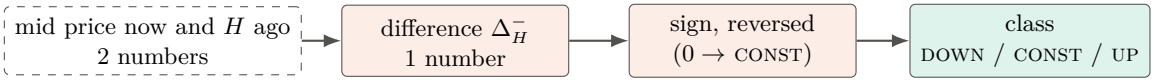


Figure 4.1: Mean Reversion-H as a block diagram. Two numbers enter, one class comes out.

from the reference with a confidence interval obtained by resampling validation hours, and every model receives the same post-hoc calibration before it is compared, because at short horizons a decision threshold is worth more than a model.

3. **Negative results are results.** Several components that are reasonable a priori (the transaction graph, a mean-reversion initialisation, wallet-skill features, zero-shot foundation models, news features, sibling features) do not help, and the thesis reports them with the same care as the components that do, with the reason the data give for each.

4.2 Baselines: causal rules on the price

All baselines are causal rules on the mid price of the market itself. With $\Delta_L^- = \text{mid}(t) - \text{mid}(t-L)$ the change over the past lookback L:

- **Constant**: always CONST. Balanced accuracy 1/3 by construction.
- **Momentum-L**: predict $\text{sign}(\Delta_L^-)$.
- **Mean Reversion-L**: predict $-\text{sign}(\Delta_L^-)$, and CONST if the price did not move.

With $L = H$, Mean Reversion is the strongest rule at every horizon (Table 5.1); it is the reference of every comparison in the thesis. Its strength has the reading of Section 3.7.1: on a bounded, discrete price most moves are one-tick oscillations that revert, and a market that did not move in the last H is likely not to move in the next H.

Figure 4.1 draws the rule in the notation used for every model of this chapter: dashed boxes are the inputs, orange boxes are computations or trained modules, blue boxes are quantities kept or produced inside the model, green boxes are the output, and every box states the size of what passes through it.

Advantages and limits. A rule needs no training, cannot leak, costs nothing and is readable; on a bounded, discrete price that mostly oscillates by one tick it is also a strong competitor, which is why it is the first reference of every comparison. It reads one number: it cannot combine the recent change with the price level, the activity of the market or the shape of the book, and it has no way to say *whether* the price will move other than by the price not having moved.

4.3 Stateless learners on hand-made features

The strongest test of whether a recurrent state is where the information lives is a model that has none. The inputs are the sixty-one causal features of Section 3.6.2, in five groups: price history (the inputs of the direct path), book shape, activity and clock, on-chain trades, and the static fields of the market. A LightGBM classifier [80] with class-balanced weights was trained on one row in ten of 7–11 March and early-stopped on 12 March, adding one group at a time (Table 5.2); the LightGBM parameters (learning rate 0.05, 63 leaves, 500 rows per leaf, feature and row subsampling 0.8, 600 rounds at most, early stopping on 12 March) are the same at every horizon.

On the same table, two neural tabular learners are trained as a tabular model should be (Section 2.8): an MLP of three layers of 512 units on a piecewise-linear encoding of every

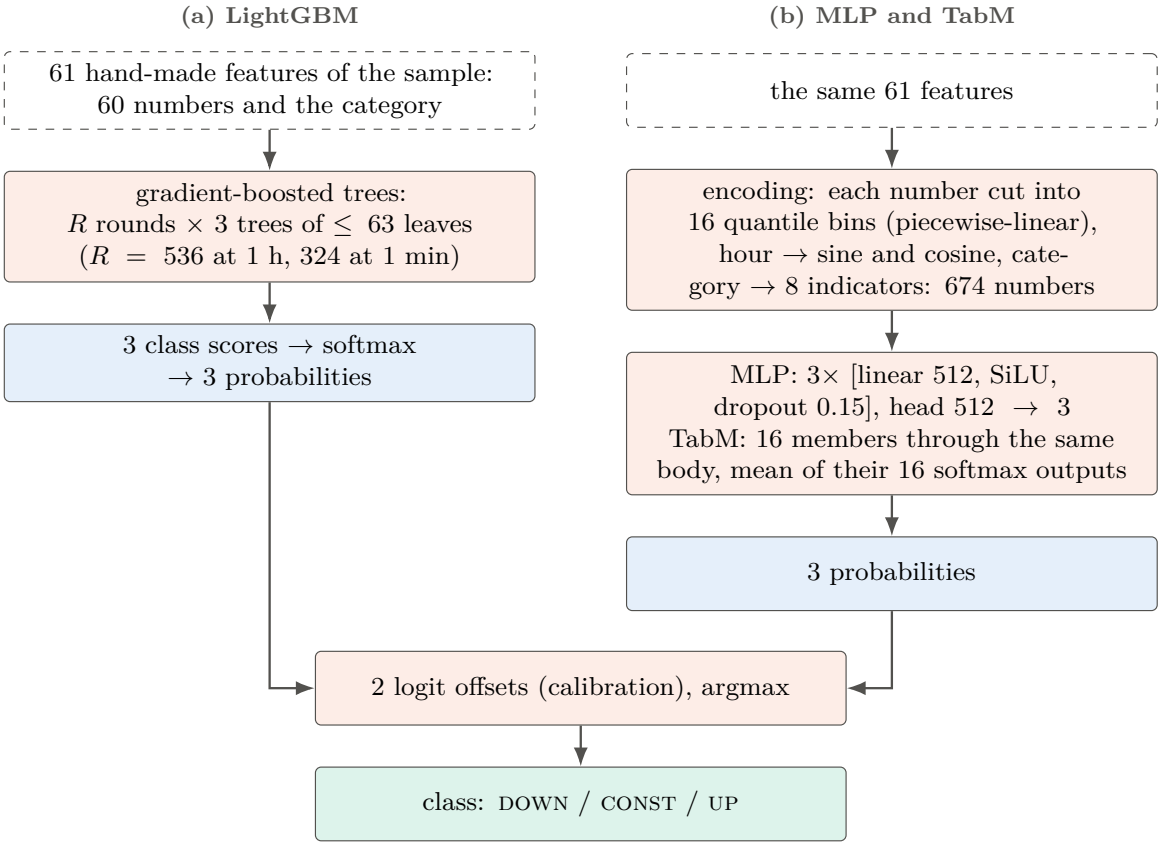


Figure 4.2: The stateless learners as block diagrams. (a) LightGBM: the 61 features of the sample enter as they are, one tree per class is added at every round, and the three class scores pass through a softmax and the two calibration offsets. (b) MLP and TabM: the same features are first expanded into 674 numbers (at 1 h; 706 at 1 min), then pass through three layers of 512 units; TabM runs sixteen members through the same body and averages their outputs.

numeric feature (sixteen quantile bins), with shuffled mini-batches of 8,192 rows, a one-cycle schedule over sixteen epochs and early stopping on 12 March, and the TabM ensemble of sixteen members on the same backbone; both with the class-balanced cross-entropy of the trees, three seeds each. They serve one purpose: to tell whether a difference between the trees and the stateful model is the learner or the training regime.

Reading the diagram (Fig. 4.2). Both learners read the same row: sixty numbers and the category of the market, which LightGBM treats as a categorical column and the networks as eight indicators. In LightGBM every round adds three trees, one per class, each with at most 63 leaves and at least 500 rows per leaf; early stopping on 12 March chooses the number of rounds (536 at one hour, hence 1,608 trees; 324 at one minute, 280 at six hours, 260 at twelve), and the sum of the trees of a class is that class's score. A softmax turns the three scores into probabilities, two logit offsets fitted on the validation day shift DOWN and UP against CONST, and the largest probability is the answer. The networks first expand the row: every number is cut into sixteen quantile bins and encoded piecewise-linearly, the hour becomes a sine and a cosine, the category eight indicators, and a flag marks each missing value, 674 numbers at one hour. The MLP is three layers of 512 units with SiLU activations and dropout 0.15 and a linear head to three scores (0.87 million parameters). TabM keeps

the same body and runs sixteen members through it, each with its own learned scaling of the input (initialised at ± 1) and its own head; the prediction is the mean of the sixteen softmax outputs (0.91 million parameters). Both are trained on shuffled mini-batches of 8,192 rows for at most sixteen epochs, with the class-balanced cross-entropy of the trees, and their output is calibrated and read exactly as the trees' is.

Advantages and limits. A stateless learner reads whatever can be computed at the sample time, including the quantities that a window of book events hides, the time since the last move, the number of recent moves, the price level, the age of the market; gradient-boosted trees handle thresholds and interactions between them without any scaling, train in minutes on a CPU, and can be trained the way tabular models are best trained, on shuffled rows with early stopping. The family's limit is its blindness to order: it sees a summary of the past and never the sequence of events that produced it, and what is not computed by hand is not seen at all. Its rows are also millions of near-duplicates of one another, which is why every result is read on the deduplicated samples as well.

4.4 Sequence models of the order-book stream

The third family keeps a state per market, updated from the stream of book events, and asks the state for the prediction. Its main member is a continuous-time stateful model built for this thesis; the family also contains the same model given the trees as a base, and a pretrained time-series forecaster used without training.

4.4.1 The continuous-time stateful model

The model, called CT-DGNN in the code, processes the stream block by block. At block time t it receives the markets that updated their book since the previous block, with a window of their last K book states ($K = 10$ in the reference configuration), and the transactions of the block as edges between wallets and markets. It keeps a state vector of dimension $d = 128$ for every market and every wallet. One step has four parts (Fig. 4.3).

States and initialisation

Market states are initialised by a fixed random projection of the 390-dimensional static features (Section 3.5.2); wallet states start near zero. The projection is not trained: the states are detached at every truncation window, so a gradient could only ever reach it through the first window. States live outside the model, in two matrices on the GPU, and are updated in place; a *sparse* engine keeps, for the nodes touched in the current window, an overlay of their new values and the time of their last event, so that a step costs time proportional to the number of active nodes rather than to the 820,000 nodes of the graph.

Continuous-time decay

Between two events a state decays as $\mathbf{h} \leftarrow e^{-r\,\Delta t}\,\mathbf{h}$, with a single learned rate r shared by all dimensions and node types (Section 2.4). Because $e^{-ra}e^{-rb} = e^{-r(a+b)}$, applying the decay lazily at a node's next event is identical to applying it at every block. In every trained model r converges to about 0.04 per second, a half-life of 17 seconds: the fast state is, by its own choice, a very short memory. Two alternatives were tested: no decay, and a fixed multi-scale decay with groups of dimensions at half-lives of one minute, one hour, one day and infinity (Section 5.3.3).

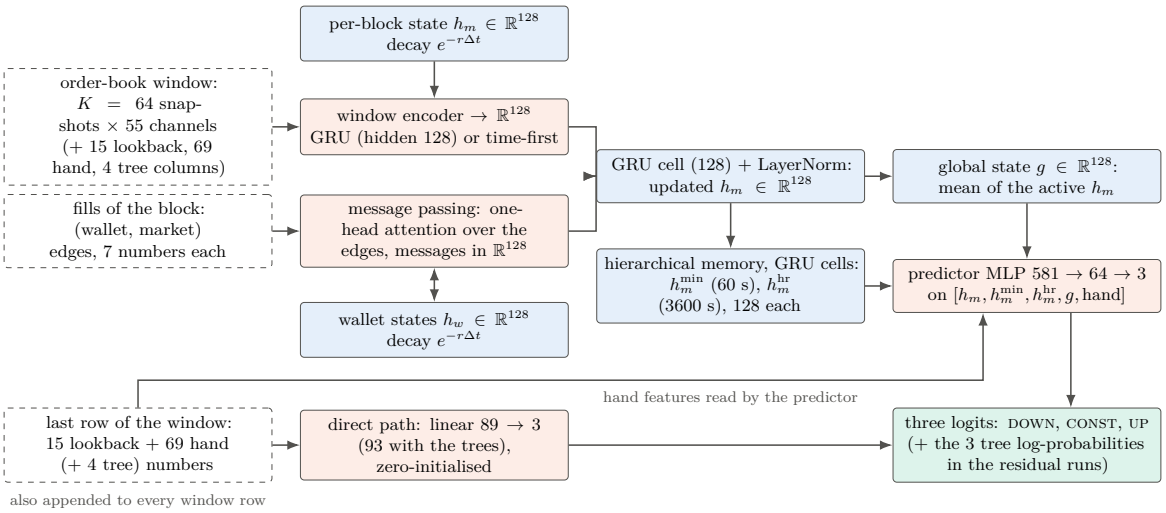


Figure 4.3: One block step of the stateful model, with the sizes of the two-pass model on the hand features (Table 4.1 gives the sizes of the other configurations). Dashed boxes are inputs of the step, blue boxes are states kept between steps, orange boxes are trained modules, the green box is the output. The edges and the wallet states exist only in the graph variant; the hand features and the tree log-probabilities only in the runs that use them.

Order-book update

For each active market the window of K book states (each 52 features plus the appended price features of Section 4.4.1) is encoded by a GRU over the K snapshots; its last output is the input of a GRU cell that updates the market state, followed by layer normalisation:

$$\mathbf{h}_m(t) = \mathrm{LN}\Big(\mathrm{GRUCell}\big(\phi(\mathbf{W}_{m,t}),\ e^{-r\,\Delta t}\,\mathbf{h}_m(t^-)\big)\Big), \tag{4.1}$$

where $\mathbf{W}_{m,t}$ is the window, ϕ the window encoder (the GRU here, or the time-first encoder of the next paragraph), t^- the time of the market's previous event, $\Delta t = t - t^-$, and LN layer normalisation. This is the only part of the model that reads the order book, and it is the same in the graph and in the LOB-only variants.

Window encoders: space first, or time first

The GRU above reads the window one snapshot at a time: at every step it sees all 55 channels of one book state (the 52 features and three momentum columns computed within the window) and condenses them into one vector, and only the sequence of these condensed vectors is a time series. Space is read before time. The alternative encoder, called *time first* in the code, inverts the order, in the manner of the channel-independent models of Section 2.3. Each channel c of the window is read as a univariate series $x_c \in \mathbb{R}^K$; it is normalised within the window by subtracting its last value and dividing by its standard deviation over the real snapshots (the zero-padded rows at the start of a young market's history are masked), and mapped by one linear layer $\mathbb{R}^K \to \mathbb{R}^{32}$, shared by all channels, to which a learned channel embedding and a projection of three scalars (the last value, the log standard deviation, the fraction of real snapshots) are added. The 55 channel tokens then pass through one multi-head self-attention layer over the channels and a linear map to the state dimension; this vector replaces the last GRU output as the input of the state-update cell. The columns that are constant along the window (lookback and forecast features, Sections 3.6.2 and 4.4.1) enter through a separate linear map from the last row. Nothing downstream changes.

The shared time map means that the encoder learns one notion of "shape of a recent series" for all 55 quantities and lets the attention decide which quantities matter together; a GRU over 64 snapshots of 55 channels, by contrast, does not fit on the available GPU memory with thousands of active markets and five-block truncated BPTT. The experiments of Section 5.3.4 compare the reference GRU over 10 snapshots, the same GRU over 32 snapshots, and the time-first encoder over 64.

Hierarchical memory

The per-block state is a 17-second memory. To give the model a longer view without changing the fast dynamics, each market gets two additional states, a *minute* state and an *hour* state, each with its own GRU cell. Once every 60 seconds the minute state reads the per-block state; once every 3600 seconds the hour state reads the minute state:

$$\mathbf{h}_m^{\min} \leftarrow \mathrm{GRUCell}(\mathbf{h}_m, \mathbf{h}_m^{\min}) \text{ every 60 s}, \qquad \mathbf{h}_m^{\mathrm{hr}} \leftarrow \mathrm{GRUCell}(\mathbf{h}_m^{\min}, \mathbf{h}_m^{\mathrm{hr}}) \text{ every 3600 s}. \tag{4.2}$$

Refreshes are event-driven (a market's slow state is refreshed at its first event after its period has elapsed) and each market has a random phase, so that refreshes are spread over time. Message passing and the book update only ever touch the per-block state; the predictor reads the three states side by side. This is the clockwork principle [53] adapted to irregular events.

Global state

The predictor also reads the mean of the per-block states of all markets active in the block, $\bar{\mathbf{h}}(t) = \frac{1}{|A_t|}\sum_{m\in A_t}\mathbf{h}_m(t)$ with A_t the set of markets active in block t, a market-wide context in the spirit of the virtual global node of Feldman et al. [74].

Long-lookback price features and the direct path

The window of K book states spans a few minutes. To let the model see what the baselines see, every row of the window is extended with, for each lookback $L \in \{15\text{ min}, 1, 3, 6, 12\text{ h}\}$, three numbers: $10\,\Delta_L^-$, $\mathrm{sign}(\Delta_L^-)$, and a flag that is zero when the price L ago is not observable (market younger than L). These features go through the GRU like the others, and they also enter a *direct path*: a linear layer from the fifteen numbers (plus their five absolute signs) to the three logits, added to the predictor's output. The layer is initialised to zero, so at the start the model is exactly the one without the path; but with weights a on the sign of Δ_H^- and $-a$ on its absolute value it reproduces Mean Reversion-H exactly. The model therefore *contains the heuristic it has to beat*, the recommendation of Feldman et al. [74], and can learn how far to depart from it. Initialising the layer *at* the heuristic instead of at zero was also tested (Section 5.3.3).

Foundation-model features

The eleven forecast features of Section 3.6.2 enter the window rows and the direct path like the lookback features; the encoder reads them as constant columns from the last row (Section 4.4.1).

Predictor and training

The predictor is a two-layer MLP with dropout on the concatenation of the per-block, minute, hour and global states; its output plus the direct path gives the three logits,

$$\mathbf{z}_{m,t} = \mathrm{MLP}([\mathbf{h}_m, \mathbf{h}_m^{\min}, \mathbf{h}_m^{\mathrm{hr}}, \bar{\mathbf{h}}, \mathbf{x}_{m,t}]) + \mathbf{D}\,\mathbf{u}_{m,t}, \qquad \hat{\mathbf{p}}_{m,t} = \mathrm{softmax}(\mathbf{z}_{m,t} + \boldsymbol{\delta}), \tag{4.3}$$

where $[\cdot]$ is concatenation, $\mathbf{x}_{m,t}$ are the hand-made features (present only in the runs of Section 4.4.1), $\mathbf{u}_{m,t}$ collects the inputs of the direct path (the fifteen lookback numbers, their five absolute signs and, when present, the hand-made features), $\mathbf{D}$ is the zero-initialised matrix of the direct path and $\boldsymbol{\delta}$ the calibration offset of Section 5.1. The loss is the focal loss with $\gamma = 2$ and class weights 2/1/2 for DOWN/CONST/UP; alternatives (automatic weights, undersampling of CONST, regression of the signed change with a calibrated threshold) are in Section 5.3.2. Gradients flow through five consecutive blocks (truncated BPTT), the optimiser is AdamW with learning rate 10^{-4}, and one pass over the seven training days is one epoch; the reference results use a single epoch, and the *two-pass* runs of Section 5.3.6 make a second pass with the states reset and the optimiser kept. Every run writes its validation predictions and probabilities to disk so that the models can be compared on exactly the same samples.

Hand-made features as inputs of the stateful model

The sixty-one features of Section 3.6.2 can also be given to the stateful model. They are computed for every tensor row (117.7 million rows over 6–13 March) and, at every block, the standardised features of the market's last row (69 columns once the category is one-hot encoded and the time-dependent ones are recomputed at the block time) are appended to the window. They enter in three places: the window rows read by the encoder, the zero-initialised direct path, and the predictor, which reads them next to the states so that an interaction such as "time since the last move × price level" does not have to survive the recurrent update. Nothing else changes.

What passes through the model, and the runs by name

Figure 4.3 carries the sizes of the two-pass model on the hand features, the configuration of the main results. At every block the model receives, for each active market, a window of the last $K = 64$ stored book snapshots; every snapshot is a row of 55 channels (ten levels of bid and ask prices and sizes, 40 numbers; the mid, spread, best quotes, order-flow and depth statistics, 12 numbers; three momentum columns computed within the window), and the same constant columns are appended to every row of the window: the 15 lookback numbers of Section 4.4.1 and the 69 columns of the hand-made features (the sixty-one features of Appendix A with the hour as a sine and a cosine, the category as eight indicators and an availability flag), 139 numbers per row. The window encoder condenses the 64×139 window into one vector of 128 numbers: the time-first encoder maps each of the 55 channel series to 32 numbers with one shared linear layer, adds a learned embedding of the channel and a projection of its last value, scale and fill, mixes the 55 tokens with one self-attention layer (4 heads, width 32) and projects the 55×32 result to 128, adding a linear map of the 84 constant columns of the last row; the GRU encoder of the reference configuration reads the $K = 10$ rows one after the other with a hidden size of 128 and keeps its last output. A linear map and a GRU cell of size 128 turn that vector and the market's previous state into the new per-block state, normalised by LayerNorm; the minute and hour states are two more vectors of 128 numbers, each updated by its own GRU cell from the state one level below; the global state is the mean of the per-block states of the markets active in the block, 128 numbers. The predictor concatenates the four states and the 69 hand columns of the last row, 581 numbers, and passes them through one hidden layer of 64 units (SiLU, dropout 0.1) to the three logits; the direct path is one linear layer from the 89 numbers of the last row (15 lookback, their five absolute signs, 69 hand) to the same three logits, initialised at zero and added. The model has 0.60 million trained parameters (0.75 in the checkpoint, 0.15 of which belong to the message-passing module that the LOB-only runs build but never call);

Table 4.1: The configurations of the stateful model that appear in the result tables, with the name used for them. All share the state of 128 numbers per market, the three-rate memory, the global state, the five lookbacks and the direct path; "columns" is the width of one row of the order-book window (55 book channels plus the constant columns), and the sizes are those of the direct path and of the predictor.

Name in the results	Encoder, K	Extra inputs	Columns	Direct path / predictor	Trained
GRU reference	GRU, 10	none	70	$20 \to 3$ / $512 \to 64 \to 3$	1 pass; 1 h, 6 h, 12 h
GRU reference with the transaction graph	GRU, 10	edges, 7 numbers each	70	same	1 pass; 1 h, 6 h, 12 h
GRU reference + Chronos-2 features	GRU, 10	11 forecast columns	81	$31 \to 3$ / same	1 pass; 1 h
Time-first, no hand features	time-first, 64	none	70	$20 \to 3$ / same	1 pass; 1 h, 6 h
Stateful, 1 pass	time-first, 64	69 hand columns	139	$89 \to 3$ / $581 \to 64 \to 3$	1 pass; 1 h
Stateful, 1 pass, weight averaging	time-first, 64	69 hand columns	139	same	1 pass, averaged weights; 1 h
Stateful, 2 passes	time-first, 64	69 hand columns	139	same	2 passes; 1 min, 15 min, 1 h
Stateful, 2 passes, residual of the trees	time-first, 64	69 hand + 4 tree columns	143	$93 \to 3$ / $585 \to 64 \to 3$	2 passes; 1 min, 1 h

the states, two matrices of 175,292 × 128 and 646,394 × 128 numbers, are not parameters. Table 4.1 lists the configurations that appear in the result tables, with the name each carries there and the sizes that change.

4.4.2 The stateful model as the residual of the trees

The combination of two models that know different things suggests a trained one. LightGBM with the control's hyper-parameters is fitted on the 55 inputs that the stateful model receives through the hand-made features, and its class log-probabilities are computed at every tensor row of the eight days, *cross-fitted by day*: the rows of a training day are predicted by a model fitted on the other six training days, the rows of 13 March by the model fitted on all seven, so that no row is ever predicted by trees that saw its day. The stateful model then receives the three log-probabilities of the last row before the block as three more inputs and adds them to its output logits, in the manner of a boosting stage,

$$\mathbf{z}_{m,t} = \log \mathbf{p}^{\text{tree}}_{m,t} + \text{MLP}(\cdots) + \mathbf{D}\,\mathbf{u}_{m,t}, \tag{4.4}$$

with the same MLP and direct path as in Eq. (4.3): at initialisation it is the tree model, and training moves it only where the state and the book history change the answer. The number that measures the state is then the gain over the tree base. In sizes, the residual run differs from the two-pass model by four columns in every window row (the three log-probabilities and an availability flag), by 585 inputs to the predictor and 93 to the direct path instead of 581 and 89, and by the three log-probabilities added to the three output logits.

4.4.3 A pretrained forecaster used zero-shot

The last member of the family needs no training. Chronos-2 [55], a time-series foundation model (Section 2.6.6), is given the last 512 mids of a market on a one-minute grid (a five-minute grid at the longest horizon), ending at the sample time, and asked for the quantiles of the mid at $t + H$. The forecast is turned into a decision in three ways: the sign of the median, the sign of the median with a dead zone of half a tick for CONST, and the position of the current mid in the forecast's quantile bands. The same quantiles, computed for every active market on a five-minute grid, are the eleven forecast features of Section 3.6.2 that the stateful model can read.

Reading the diagram (Fig. 4.4). Nothing in this member is trained: Chronos-2 is used with its published weights, on one series at a time, with a context of 512 values and a forecast of H steps ahead. The half-tick dead zone is ±0.0005 around the current mid, half of the tick of 0.001; the band rules answer UP when a lower quantile is above the current mid and

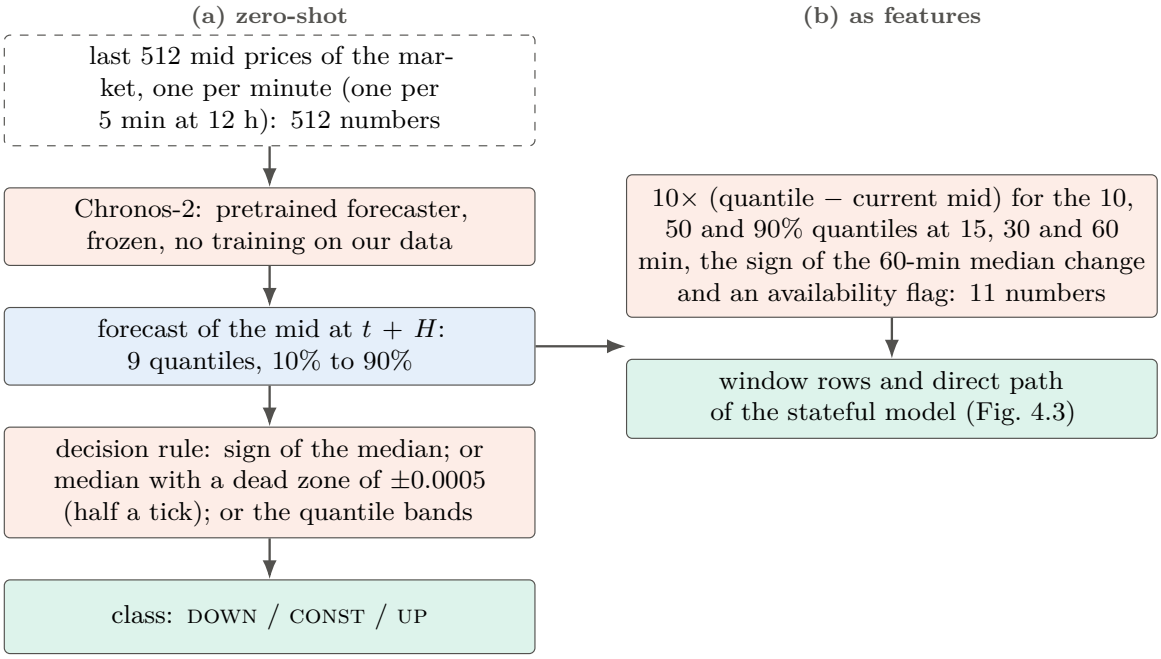


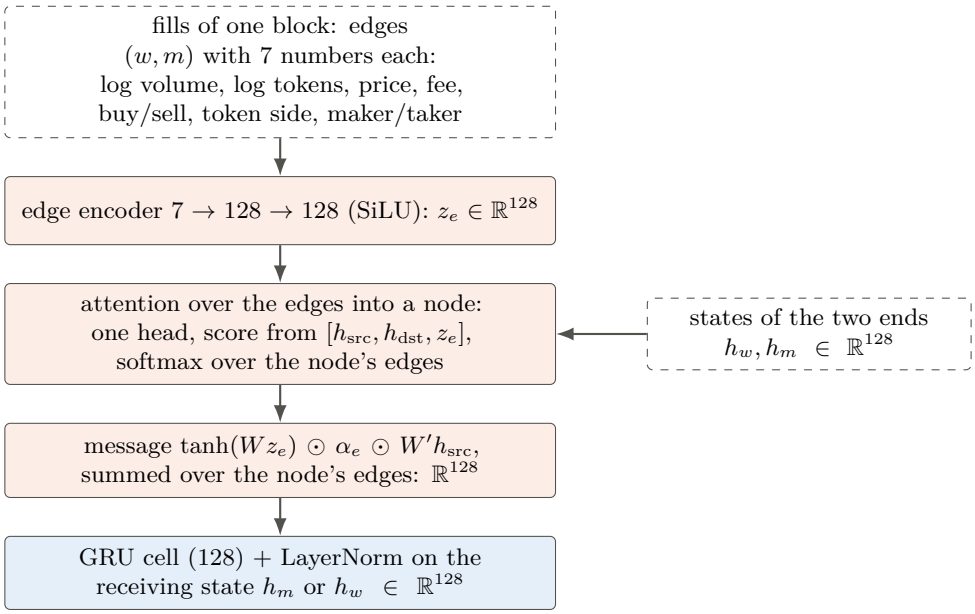
Figure 4.4: The pretrained forecaster as a block diagram. (a) Zero-shot: 512 mid prices enter, nine quantiles of the mid at $t+H$ come out, and a rule turns them into a class. (b) The same quantiles, computed on a five-minute grid for every active market, become eleven inputs of the stateful model.

DOWN when an upper one is below it. As features, the three quantiles at three horizons are expressed as ten times their distance from the current mid, so that they are on the scale of the lookback changes, and a forecast is reused for at most eight minutes.

Advantages and limits. A model with a state reads the raw stream and can, in principle, keep what a summary throws away: the order in which depth appeared and disappeared, the pattern of the last minute of a quiet market; its memories at three rates give it a view from seconds to hours, and the direct path lets it start from the rule it has to beat. It pays for this in three ways. It is trained once through the stream, in time order, with the gradient truncated to a few blocks, a regime under which the same learner on the same table is known to do worse than when it is trained on shuffled rows (Section 2.8); a full-length run costs hours on a GPU, so few variants can be tried; and its window is on an event clock, so that it is never told how much wall-clock time its last twenty events span unless the time is given to it as a feature. The pretrained forecaster has the opposite profile: no training and a long context, but weights fitted to other domains, a per-window normalisation that treats a one-tick oscillation as a signal, and no notion of "no change".

4.5 Graph models

The fourth family is the one the project started from: the bipartite graph between wallets and markets, read by message passing at every block of the stateful model, and, in a second view, the graph between the contracts of the same event.



first pass: wallets → markets; second pass: markets → wallets, with the same weights; the updated h_m then enters the state update of Fig. 4.3

Figure 4.5: Message passing on the transactions of one block. Each fill is an edge between a wallet and a market with seven numbers; the receiving state is updated by a GRU cell, first on the market side, then on the wallet side.

4.5.1 Message passing on transactions

The transactions of the block form the set of edges E_t (Section 2.4), each a fill $e = (w, m, t, \mathbf{a}_e)$ with seven attributes $\mathbf{a}_e \in \mathbb{R}^7$: the logarithms of the collateral and of the number of tokens, the price, the fee rate, buy or sell (± 1), the token orientation (± 1) and maker or taker. Messages go from wallets to markets and from markets to wallets through attention over the incident edges [65], and the receiving state is updated by a GRU cell. For a market m with incoming edges $N_t(m)$, w being the wallet of edge e,

$$\mathbf{e}_e = \mathrm{MLP}_e(\mathbf{a}_e), \qquad \alpha_e = \mathrm{softmax}_{e \in N_t(m)}\Big(\mathrm{LeakyReLU}\big(\mathbf{q}^\top [\mathbf{h}_w, \mathbf{h}_m, \mathbf{e}_e]\big)\Big), \tag{4.5}$$

$$\mathbf{h}_m \leftarrow \mathrm{LN}\Big(\mathrm{GRUCell}\Big(\sum_{e \in N_t(m)} \alpha_e \, \tanh(\mathbf{V}\mathbf{e}_e) \odot \mathbf{U}\mathbf{h}_w, \; \mathbf{h}_m\Big)\Big), \tag{4.6}$$

with $\mathbf{q} \in \mathbb{R}^{3d}$ and $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{d \times d}$ trained, and $\odot$ the element-wise product; the same update, with the same weights, then runs from markets to wallets. The *token orientation* attribute is $+1$ when the traded token is the one the order book refers to and -1 otherwise (Section 3.4), so that a buy of the No token counts as a sell of the Yes token. Several edge enrichments were tested: a causal record of the wallet's past accuracy at the horizon ("skill"), a placebo skill (another wallet's record), and the wallet's past behaviour (activity, maker share, one-sidedness, size, breadth). Setting `-ablation` removes the edges entirely and gives the LOB-only model.

Reading the diagram (Fig. 4.5). The seven numbers of an edge (log collateral, log tokens, price, fee rate, buy or sell, the side of the traded token, maker or taker) are mapped by two linear layers to an edge vector of 128 numbers. For every node that received edges in the block, one attention head scores each incoming edge from the states of its two ends and the edge vector, normalises the scores over the node's edges, and sums the messages, each the

product of a gated edge vector, its attention weight and a linear map of the sender's state. A GRU cell of size 128 with LayerNorm writes the sum into the receiving state. The module runs twice per block with the same weights, wallets to markets and then markets to wallets, and the updated market state is the one the book update of Fig. 4.3 continues from. The graph adds 0.15 million parameters to the model and a state of 128 numbers for each of the 646,394 wallets.

4.5.2 The two graphs

Two graphs are considered. The first is the bipartite *wallet–market* transaction graph of Section 4.5.1: the stateful model with its message passing active is the graph model, and the same model with `-ablation` (no edges) is its LOB-only counterpart, so that the contribution of the graph is a paired difference at every stage of the design. The second is the *market–market* graph between the contracts of the same event. It was not built as a message-passing layer; its information was measured first in the stateless learner, through the ten sibling features of Section 3.6.2. If the trees find nothing in those features, a message-passing layer over the same edges has nothing to carry, and the lead–lag statistic of Section 3.7.1 says why.

Advantages and limits. The graph is the only family that can use what makes this dataset unusual, the identity of every counterparty: a wallet's history across markets, its past accuracy, and the co-movement of the contracts of one event are visible to no other family. Because the graph variant and the LOB-only variant are the same model with and without edges, its contribution is a paired difference at every stage of the design. Its limits are in the data more than in the method: most wallets trade once and have no history to carry (Section 3.7.2); a trade is a book event before it is an edge, so the order-flow features see it first; the shared decay erases a wallet's state within a minute of its last trade; and keeping a state for 820,000 nodes is what makes the sparse engine of Section 4.4.1 necessary.

4.6 Information outside the book

Three sources were tried, none of which changes the learner. The *foundation-model forecasts* of Section 3.6.2, used zero-shot as a forecaster of the mid in their own right and then as inputs of the stateful model. The *news flow* about the entities of a market, as the twelve features of Section 3.6.2, measured in the stateless learner. And the *text* of the market, as the sentence embedding that initialises the market state (Section 4.4.1) and as a category indicator among the static features (Section 3.7.1).

Advantages and limits. These sources are the only way to reach what moves a price from outside the book, and each is measured first in the stateless learner, where adding a group of features costs minutes rather than hours. Their limits are coverage and alignment: three samples in four have a market with a named entity, the news flow counts articles rather than reading them, the forecaster is noise at extreme prices (Section 2.6.6), and the text enters through a frozen embedding and a category.

4.7 The families at a glance

Table 4.2 summarises the families, what each reads, where its state lives and what it is measured against; Table 4.3 what enters and what leaves each model, with sizes; Chapter 5 reports what each did.

Table 4.2: The five families of approaches compared in the thesis.

Family	Members	Reads	State	Reference
Causal rules	Constant, Momentum-L, Mean Reversion-L	the market's own mid	none	—
Stateless learners	LightGBM, MLP, TabM	the 61 hand-made features at the sample time	none	Mean Reversion-H
Sequence models	CT-DGNN (LOB-only), residual model, Chronos-2 zero-shot	the window of book states, the lookbacks, the hand-made features, the tree log-probabilities	per market, at three rates	the stateless learner
Graph models	CT-DGNN with edges, sibling features, smart money	the above plus the transaction edges and the siblings' rows	per market and per wallet	the LOB-only model
Outside the book	forecast, news and text features	the above plus forecasts, news counts, the question text	that of the host model	the host model without the source

Table 4.3: What enters and what leaves each model, with sizes. A sample is a market at a block time; the output of every learned model is three class scores, read through the same calibration (two logit offsets fitted on the validation day) and the largest score.

Model	Input per sample	Inside	Output
Mean Reversion-H	the mid now and H ago (2 numbers)	a difference and a sign	class
LightGBM	the 61 hand-made features (60 numbers and the category)	536 rounds $\times$ 3 trees of ≤ 63 leaves at 1 h, the number of rounds chosen by early stopping	3 scores
MLP	the 61 features expanded to 674 numbers	3×512 units, SiLU, dropout 0.15; 0.87 M parameters	3 scores
TabM	the same 674 numbers	the same body shared by 16 members with their own input scaling and heads; mean of 16 softmaxes; 0.91 M parameters	3 scores
Stateful model (two passes, hand features)	a 64×139 window of book snapshots, the 84 constant columns of its last row, the market's three states (128 each)	encoder $\to$ 128; GRU cell 128; minute and hour cells 128; global mean 128; predictor $581 \to 64 \to 3$; direct path $89 \to 3$; 0.60 M trained parameters	3 logits
... as the residual of the trees	the same, plus 4 tree columns (log-probabilities and a flag)	the same with 585 and 93 inputs; the tree log-probabilities added to the logits	3 logits = trees + correction
GRU reference	a 10×70 window and the market's states	GRU encoder (hidden 128); predictor $512 \to 64 \to 3$; direct path $20 \to 3$; 0.42 M trained parameters	3 logits
... with the transaction graph	the same, plus the fills of the block (7 numbers per edge)	edge encoder $7 \to 128$, one-head attention, GRU cells on markets and wallets; 0.57 M parameters; wallet states $646{,}394 \times 128$	3 logits
Chronos-2 zero-shot	512 mid prices on a one-minute grid	a pretrained forecaster, frozen; 9 quantiles of the mid at $t + H$	class, by a rule

Chapter 5

Experiments and Results

This chapter reports the experiments. Section 5.1 gives the setup under which every number is produced: the periods, the metric, the subsets, the intervals, the calibration rule and the compute. The families are then assessed in the order of Chapter 4, mostly at one hour, the horizon at which the design was done: causal rules and stateless learners (Section 5.2), the stateful model from its first version to its last (Section 5.3), the graph models (Section 5.4) and the sources outside the book (Section 5.5). Section 5.6 reads the same families across the five horizons, by kind of market and for robustness; Section 5.7 reserves the locked test set for the final numbers; and Section 5.8 discusses the results against the four questions of Section 1.3.

5.1 Experimental setup

Models are trained on 6–12 March and validated on 13 March, with 24–26 March locked for the final numbers (Sections 3.6.3 and 5.7); hyper-parameters and design choices were selected on the validation day. All numbers in the thesis are balanced accuracies [86] on the validation day, unless stated otherwise. Every result is reported on all validation samples and on three subsets: the *deduplicated* samples (at most one per market per fifteen minutes, 1.2–1.3 million), the *extreme* ones (mid outside $[0.05, 0.95]$) and the *central* ones. Differences between models are computed on exactly the same samples and carry a 95% hour-block bootstrap interval in brackets (Section 2.9); "MR" is Mean Reversion-H. Three rules make the comparisons fair.

- **The same samples.** The stateless learners are trained and evaluated on the samples of the stateful runs: at one hour the control was built from the run's predictions; at one and fifteen minutes it had to be rebuilt on the run's samples, because a control that samples tensor rows at the row time is a different population (27 of 598,872 rows in common at one minute; Section 5.6.1).
- **The same calibration.** The decision rule of a classifier trained with a balanced loss is not balanced-accuracy-optimal on a different class mix. Every model is therefore given two logit offsets (on DOWN and UP) chosen on the even validation hours and applied to the odd ones and vice versa, on a grid of ± 0.6 in steps of 0.1 at one hour and ± 2 in steps of 0.2 at the short horizons, and models are compared calibrated against calibrated. The offsets are worth $+0.3$ points to the trees at one hour and $+5.4$ at one minute; a comparison without them measures thresholds, not models.
- **Full-length runs.** Where a short experiment was needed to screen many variants, it was trained on 11–12 March; every claim rests on full-length runs, and the cases in which a short test predicted the wrong sign are reported (Section 5.6.6).

Table 5.1: Causal rules on the validation day, balanced accuracy, all samples and deduplicated samples (one per market per ten minutes). Momentum-H and Mean Reversion-H use the change over the past H; Momentum last-tick uses the sign of the last mid change. At 1 min and 15 min only Mean Reversion was evaluated, on the samples of the short-horizon runs.

	1 min		15 min		1 h		6 h		12 h	
Rule	all	dedup	all	dedup	all	dedup	all	dedup	all	dedup
Constant, prior, uniform	0.333	0.333	0.333	0.333	0.333	0.333	0.333	0.333	0.333	0.333
Momentum last-tick	—	—	—	—	0.375	0.387	0.365	0.374	0.362	0.370
Momentum-H	—	—	—	—	0.472	0.445	0.436	0.422	0.424	0.420
Mean Reversion-H	**0.536**	**0.501**	**0.537**	**0.525**	**0.527**	**0.506**	**0.501**	**0.488**	**0.507**	**0.500**

Compute and the order of the experiments. Every stateful run trains on one GPU of a shared server, one run per GPU, for nine to seventeen hours depending on the horizon and the load (Appendix B); the stateless learners train in minutes on a CPU. Because of that cost the design was done at one hour, variants were screened with two-day short tests and confirmed with full-length runs, the other horizons were run with the final configurations only, and seeds were repeated where a gain was small. Every run is identified by a label that encodes its configuration and writes its validation predictions to disk, so that any two runs can be compared on exactly the same samples (Appendix B).

5.2 Rules and stateless learners

The families without state come first: a causal rule on the market's own price, or a learner that reads a few dozen numbers computed from the clean data at the sample time and nothing else. It is the family that tells where the information is.

5.2.1 Causal rules

Mean Reversion is above 0.5 at every horizon and it is the rule to beat (Table 5.1). Momentum is far below it and, on the sign of the last tick, close to the constant rule: on this data the sign of the last move is *anti*-informative, and the whole of the simple predictability is a reversal. The Mean Reversion values quoted in later sections differ in the third decimal from these because every comparison is made on the samples of the run it belongs to (Section 5.1).

Why mean reversion is strong

A Polymarket mid price is a bounded, discrete process observed through a thin book. Most of its changes are one-tick moves produced by an order arriving at or leaving the best quote; they are not information and they revert. A market that has not moved in the last hour is usually illiquid or settled near a boundary and will not move in the next hour either. Mean Reversion-H encodes exactly these two regularities, and it is why a rule that uses one number is above 0.5 balanced accuracy. Its strength is largest at extreme prices (below 0.05 or above 0.95), where the process is almost purely a one-tick oscillation, and it is there that every learned model is weakest relative to it.

5.2.2 Trees on hand-made features: the no-state control

The sixty-one features of Section 3.6.2 were given to LightGBM one group at a time, on the validation samples of the best one-hour stateful model (the time-first model of Section 5.3.4), so that the two can be compared on the same samples.

Table 5.2: Gradient boosting without state against the deep models, 1h, on the same validation samples (balanced accuracy; last column: difference from the time-first model in points with 95% hour-block bootstrap intervals).

Model	#	All	Dedup.	Extreme	Central	Δ vs time-first
Mean Reversion		0.5311	0.5180	0.5230	0.5348	−3.32
GRU K=10 model		0.5547	0.5382	0.5385	0.5602	−0.97 [−1.28, −0.63]
Time-first K=64 model		0.5643	0.5410	0.5377	0.5748	
G1 price history	12	0.5617	0.5509	0.5449	0.5685	−0.26 [−0.67, +0.17]
G2 + book shape	26	0.5831	0.5718	0.5620	0.5918	+1.87 [+1.55, +2.19]
G3 + activity and clock	47	0.5929	0.5757	0.5722	0.6015	+2.86 [+2.54, +3.16]
G4 + on-chain trades	56	0.5997	0.5812	0.5788	0.6083	+3.54 [+3.22, +3.88]
G5 + static fields	61	**0.6015**	**0.5827**	**0.5804**	**0.6107**	+3.72 [+3.33, +4.12]

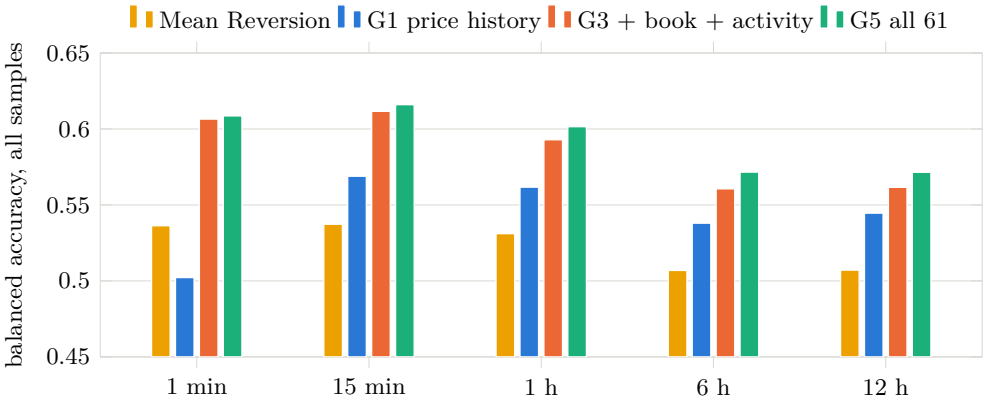


Figure 5.1: The stateless control by feature group at every horizon, against Mean Reversion (all samples). Price history alone (G1) is below the rule at one minute and above the stateful model at six and twelve hours; the activity clock (G3) carries most of the gain at the short horizons.

Price history alone reproduces the deep model. The book shape adds two points, the activity clock one more, the trades two thirds of a point and the static fields a fifth; the full model is 3.7 points above the best deep model on the full stream, 4.2 on the deduplicated samples, and ahead in every one of the 24 validation hours and in every market category. The gain importance is dominated by the time since the last mid change (32%), the number of mid changes in the last hour (14%) and the realised volatility (10%); the level-1 imbalance and the category get 1% each and every order-flow accumulator of the tensor less than that. Univariately, on the samples that moved, the sign of "mid minus its one-hour mean" gives the direction 57.3% of the time, the level-1 imbalance 55.7%, the microprice 55.5%, the on-chain flow 52.6% (on the fifth of the samples with trades), the 10-level imbalance 50.8% and the tensor's all-level order-flow imbalance 50.0–50.5%; for separating CONST from a move, the time since the last mid change has an AUC of 0.85.

The other horizons

The same control was run at the other horizons (Table 5.3 and Fig. 5.1): at 6h and 12h on the exact validation samples of the deep models, at 1 minute, where no deep model has saved predictions, on one tensor row in twenty of the validation day labelled like the training rows, against Mean Reversion alone. At 6h and 12h the full model is 6.4–6.5 points above

Table 5.3: The no-state control at the other horizons, all samples and deduplicated samples (balanced accuracy; Δ in points with 95% hour-block bootstrap intervals). At 1 minute the samples are one tensor row in twenty of the validation day and the only comparator is Mean Reversion.

Horizon	Subset	MR	Best deep	G1	G3	G5	Δ G5 − MR [95% CI]
6h	All	0.5069	0.5396 (TF)	0.5380	0.5606	**0.5716**	+6.47 [+6.02, +6.91]
	Dedup	0.5002	0.5370 (TF)	0.5360	0.5588	**0.5665**	+6.63
12h	All	0.5071	0.4914 (GRU)	0.5446	0.5616	**0.5715**	+6.44 [+5.53, +7.33]
	Dedup	0.4995	0.5168 (GRU)	0.5427	0.5598	**0.5680**	+6.85
1 min	All	0.5862	—	0.5901	0.6579	**0.6594**	+7.32 [+6.22, +8.65]
	Dedup	0.5244	—	0.4636	0.5604	**0.5610**	+3.67

the rule: 3.2 above the time-first model at 6h and 8.0 above the reference at 12h, where the deep model is below the rule. Unlike at 1h, price history alone is already clearly above the deep model at these horizons (+1.7 at 6h, +5.3 at 12h): the five returns read by trees together with the price level carry most of the long-horizon signal, and the 12h return is the strongest single feature (reversal sign, 57.9% directional hit). The activity clock keeps a third of the gain importance, and the time to the end of the market, its age and its duration rise to about 9% together. At 1 minute the direction of a move is nearly unpredictable (the best univariate hit rate is 54.1%, the level-1 imbalance), and the whole gain is in *whether* the mid will move: the number of mid changes in the last hour separates CONST from a move with an AUC of 0.94, the time since the last change 0.91, the event count of the last minute 0.89, and the activity group alone is 7.2 points above the rule. Price history alone is in fact *below* the rule at 1 minute on the deduplicated (−6.1) and extreme (−7.1) samples: trees on returns learn the move-or-not prior of the full stream, which is wrong on independent samples. The ordering of the groups is the same at every horizon, activity clock first, then book shape, then price history, with trades and static fields marginal; the tensor's order-flow accumulators stay below 1% of the importance everywhere.

Two caveats. The boosted model optimises balanced accuracy directly through its class weights, whereas the deep model is trained with the focal loss and predicts UP for 15% of the samples against 21.5% in the labels, so part of the gap may be calibration (the deep model's probabilities were not saved, so a prior correction could not be tried). And the control was trained on a tenth of five days; the deep model on seven full days. Neither caveat is large enough to change the sign.

Where the information is

The control of Section 5.2.2 is the most informative negative result of the thesis. A model with no memory, reading a few dozen numbers that anyone would compute from the clean data, beats the best recurrent model by almost four points. Three things follow. First, the recurrent state is not where the information lives: the price history alone (five returns, the mid and the hour, fed to trees instead of to a linear direct path) already matches the deep model, so what the state adds on top of the heuristic is small. Second, the information the deep model misses is not hidden in the order book; it is the *activity clock*. The single best predictor of whether a price will move in the next hour is how long ago it last moved, followed by how many times it moved in the last hour; neither is a column of the tensor, and the event-clock window (Section 3.5.2) hides them, because a window of twenty events can be ten seconds or ten minutes old and the model is never told which. Third, the way the book is compressed matters more than the architecture that reads it, which is the lesson of Kolm,

Turiel, and Westray [30] and Lucchese, Pakkanen, and Veraart [31] reproduced here: the direction is in the first level (imbalance, microprice) and in the position of the mid relative to its recent mean, not in ten levels of depth nor in an order-flow accumulator that sums changes at levels the mid will never reach.

The other horizons sharpen the picture rather than change it. At 6h and 12h the control is six and a half points above the rule and the price history alone already beats the deep model, so the long-horizon signal is a slow reversal of the 6h and 12h returns, modulated by the price level and by where the market is in its life (time to the end, age), which trees read directly and the direct path, linear in the returns, cannot bend to the level. At 1 minute nothing predicts the direction, every feature is within four points of a coin toss on the samples that moved, and the whole task is to predict *activity*: the gain of the control is the activity clock alone, and returns without it are worse than the rule on independent samples. The horizon thus changes what the task is, from "will the price move in the next minute" to "which way will it revert over the next half day", and the same architecture cannot be the right one at both ends.

5.2.3 Neural tabular learners: trees against networks

Whether a gap between the trees and a deep model is the learner or the training regime can be settled on the control's table alone. A wider tabular network (three layers of 512 units on the piecewise-linear encoding, shuffled mini-batches, a one-cycle schedule over sixteen epochs with early stopping) reaches 0.5983 with a single seed and 0.5991 with three; the parameter-efficient ensemble TabM [84], sixteen members sharing one backbone, reaches 0.6012, 0.6015 and 0.5996 with three seeds, indistinguishable from LightGBM (-0.04 [-0.21, $+0.11$] for the seed average), and its combination with the trees adds nothing ($+0.07$): the two learners know the same things. The claim that trees beat networks on this table does not survive a properly trained network; the claim that the online, time-ordered regime of the stateful model costs between one and a half and two points does (one pass 0.5842, two passes 0.5907, tabular 0.6011; Section 5.3.6).

At the short horizons the same learner on the same table gives the same answer (Section 5.6): TabM is $+0.33$ over the calibrated trees at one minute and $+0.22$ at fifteen, and their combination adds nothing beyond TabM at either. Trees and a properly trained network know the same things on this table.

5.2.4 What a stateless model cannot see

A stateless learner reads a summary of the past at the sample time: how long since the last move, how many moves in the last hour, the shape of the book now, the returns over fixed lookbacks. What it cannot read is the *sequence* of book events, the order in which depth appeared and disappeared, the pattern of the last sixty seconds of a quiet market. Whether that sequence carries information beyond its summary is exactly what the stateful family measures, and the answer of Sections 5.3 and 5.6 is: not at one hour, where the summary is complete, and a little at one minute, where it is not.

5.3 Stateful models

The sequence models keep a state per market, updated at every block from the stream of book events, and ask the state for the prediction. This section follows the stateful model of Section 4.4 from its first version to its last, with, at each step, what changed, what it gained and why. Unless stated otherwise the numbers are those of the LOB-only model, with no edges; the contribution of the edges is the subject of Section 5.4.

Table 5.4: Models before the direct path (seed 0; the lookback rows are the mean of three seeds).

Model	1h	6h	12h
Mean Reversion-H	0.527	0.501	0.507
CT-DGNN, graph	0.496	0.471	0.430
CT-DGNN, LOB-only	0.492	0.466	0.430
+ lookback 1/6/12h, graph (3 seeds)	0.5035	0.4769	0.4389
+ lookback 1/6/12h, LOB-only (3 seeds)	0.5037	0.4759	0.4358

Table 5.5: Loss and imbalance variants, LOB-only model with lookback 1/6/12h, seed 0.

Variant	1h	6h	12h
Focal, weights 2/1/2 (reference)	0.504	0.476	0.437
Focal, automatic weights	0.487	0.475	0.479
Undersampling of CONST	0.486	0.467	0.488
Regression + calibrated threshold	0.410	0.428	0.410

5.3.1 First models

Table 5.4 gives the first numbers: the dynamic model, with or without the graph, beats the blind baselines and momentum but loses to Mean Reversion by 3–8 points. Adding the lookback features of Section 4.4.1 to the window (without the direct path) gains about one point at every horizon, and three seeds put the seed-to-seed standard deviation at 0.001–0.002. The graph contributes nothing at 1h, +0.1 at 6h with mixed sign across seeds, and +0.3 at 12h with the same sign on all seeds.

5.3.2 Loss and imbalance variants

With the LOB-only lookback model as reference, Table 5.5 compares the fixed class weights with automatic (inverse-frequency) weights, with undersampling of the CONST class during training, and with regression of the signed change followed by a threshold calibrated on the last training day. At 1h and 6h the fixed weights are best; at 12h, where CONST is a third of the samples instead of a half, undersampling gains five points. Regression is far worse at every horizon: the Huber loss is dominated by the few large moves and the threshold calibrated on training data does not transfer. The continuous-time anti-symmetric update (CTAN, [73]) in place of the GRU gives no gain and is much worse without the decay; the wallet behaviour features give ±0.3 with no consistent sign.

5.3.3 Memory, direct path and lookbacks: the chain of improvements

Starting from the lookback model, three additions were made one at a time and evaluated at full length (Table 5.6 and Fig. 5.2). The hierarchical memory gains +1.2 [+0.9, +1.5] at 1h and +3.4 [+3.1, +3.7] at 12h. The direct path with the global state gains +2.9 [+2.6, +3.3] at 1h and +0.2 [0.0, +0.4] at 12h. Five lookbacks instead of three gain +1.0 [+0.8, +1.1] and +1.9 [+1.7, +2.0]. The final LOB-only model is +2.4 [+1.9, +2.8] above Mean Reversion at 1h and −1.6 [−2.1, −1.1] below it at 12h on all samples; on the deduplicated subset it is above at both horizons (+2.0 and +1.7). At 6h (Table 5.7) it is +1.5 [+0.9, +2.1] above on all samples and +3.3 above deduplicated.

Table 5.6: Chain of improvements, LOB-only, seed 0, all samples. Each Δ is the gain over the previous row with its 95% hour-block bootstrap interval.

Configuration	1h	Δ	12h	Δ
Mean Reversion-H	0.5311		0.5071	
lookback 1/6/12h	0.5038		0.4368	
+ hierarchical memory (60 s, 1 h)	0.5161	+1.23 [+0.92, +1.50]	0.4706	+3.38 [+3.08, +3.67]
+ direct path + global state	0.5452	+2.91 [+2.61, +3.25]	0.4727	+0.21 [+0.01, +0.39]
+ five lookbacks (0.25/1/3/6/12 h)	0.5547	+0.95 [+0.81, +1.08]	0.4914	+1.87 [+1.73, +2.01]
best − Mean Reversion		+2.36 [+1.93, +2.76]		−1.57 [−2.13, −1.09]

Table 5.7: Best configuration at 6h (seed 0).

Subset	Mean Reversion	LOB-only	graph
all (9.3 M)	0.5069	0.5214 (+1.45 [+0.89, +2.06])	0.5279 (+2.10 [+1.69, +2.51])
deduplicated (1.2 M)	0.5002	0.5333 (+3.31)	0.5278 (+2.76)
extreme prices	0.4879	0.5136	0.5154
central prices	0.5199	0.5260	0.5355

Variants that did not help. Removing the decay (0.475 at 1h) or replacing it with the fixed multi-scale decay (0.509, 0.427) is worse than the learned decay. Initialising the direct path *at* Mean Reversion instead of at zero is −2.1 [−2.7, −1.6] at 1h; log-odds price features have no effect. The Mean-Reversion initialisation is also the clearest case in which a two-day short test predicted the wrong sign (+0.7 and +2.0 in the short test, negative at full length), which is why every claim in this section rests on full-length runs.

Seeds. Three seeds of the best LOB-only configuration give 0.5547, 0.5546, 0.5549 at 1h (deduplicated 0.538–0.544): the gain over Mean Reversion is stable to the third decimal. At 12h they give 0.4914, 0.4843, 0.4695 (standard deviation 0.009), all below Mean Reversion on all samples and all above it on the deduplicated subset: the 12h result is real on independent samples and fragile on the full stream.

Why the direct path matters

Before the direct path, the model had to *learn* mean reversion through a GRU over window rows that contained the lookback change as one feature among fifty-five, and it never fully did: it stayed three points below the rule. With the direct path the rule is a two-weight solution reachable by gradient descent, and the recurrent part only has to learn the *residual*: when not to revert. The +2.9-point jump at 1h is the size of what the recurrent part had been failing to express. Initialising the path at the rule instead of at zero is worse, which is instructive: starting from the rule puts the model in a region where the gradient of the focal loss pushes the recurrent part toward agreeing with the rule, and the residual is never learned; starting from zero lets both parts grow together.

Why the hierarchical memory matters, and why the decay stays short

The learned decay rate converges to a 17-second half-life in every run. This is not a failure: at the two-second time scale of the stream, the per-block state is best used as a sharp description of the last few updates, and a slow decay would blur it. What the fast state cannot do is remember the morning; the minute and hour states do, each with its own GRU, and the

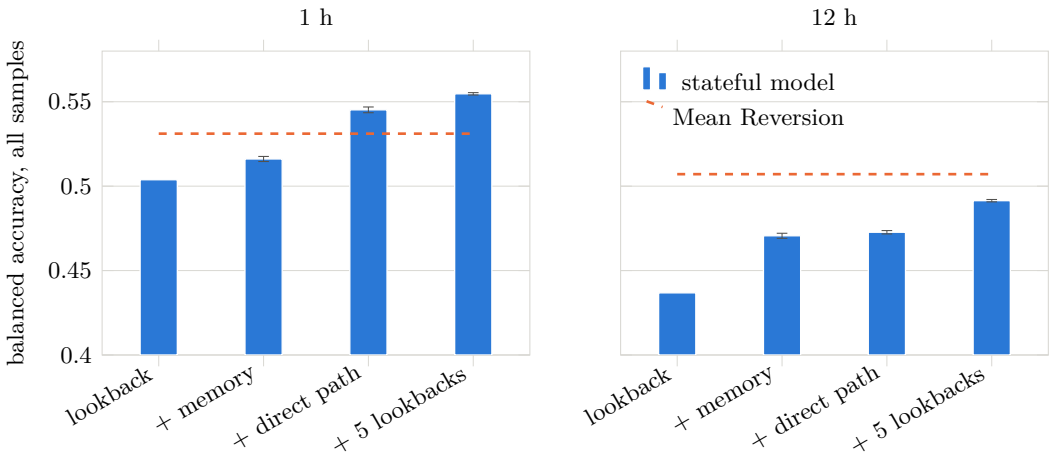


Figure 5.2: The chain of improvements of the LOB-only stateful model at 1 h and 12 h (seed 0, all samples; error bars are the 95% hour-block bootstrap intervals of each step's gain). The direct path is the large step at 1 h, the hierarchical memory at 12 h; at 12 h the model stays below Mean Reversion on the full stream.

gain is largest at 12h (+3.4 points), the horizon at which the morning matters most. A fixed multi-scale decay inside one state (the alternative tested) is worse because it forces one set of weights to serve all time scales, which is the problem the clockwork design avoids [53].

Why five lookbacks beat three

The three lookbacks (1, 6, 12 h) describe the day; the two added ones (15 min and 3 h) describe the recent past, where the dynamics of a prediction market are fastest. A 15-minute change is close to the natural time scale of the one-tick oscillation and lets the direct path revert *that*, while reading the longer changes as trend. The gain is consistent across seeds.

5.3.4 Window encoders: space first, or time first

The order-book window of the best 1h model was encoded in three ways (Section 4.4.1): the reference GRU over the last 10 snapshots, the same GRU over 32 snapshots (the longest that fits in memory) as a window-length control, and the time-first encoder over 64 snapshots. Table 5.8 compares the three runs on the same validation samples. The longer GRU window gains a quarter of a point; the time-first encoder gains three quarters of a point more, to 0.5643, the best one-hour balanced accuracy of the study, one point above the reference with an interval that excludes zero by a wide margin and thirty times the seed spread of the reference. The gain is concentrated in the central price range (+1.5 points over the reference); at extreme prices and on the deduplicated samples the three encoders are within their intervals of each other, and the longer GRU is in fact the best on the deduplicated samples. The time-first encoder costs 14 milliseconds per block in the order-book stage against 10 for the longer GRU and 5 for the reference. A second seed and the 6h horizon (Table 5.9) confirm the result: seed 1 reproduces seed 0 to within a tenth of a point on every subset (two-seed means 0.5639 against 0.5547), and at 6h the time-first encoder is 1.8 points above the reference and 3.3 above Mean Reversion, this time on every subset, the deduplicated samples and the extreme prices included, and above the best 6h graph run of Section 5.4.1.

Two caveats belong with this result. The three momentum columns of the window are computed over its K snapshots, so they change meaning with K; and the snapshots are on

Table 5.8: Three window encoders in the best 1h configuration, seed 0 (balanced accuracy; differences in points with 95% hour-block bootstrap intervals).

Subset	n	MR	GRU K=10	GRU K=32	Time-first K=64	Δ vs K=10	95% CI
All samples	9,864,125	0.5311	0.5547	0.5571	0.5643	+0.97	[+0.63, +1.28]
Deduplicated	1,255,651	0.5180	0.5382	0.5435	0.5410	+0.29	[−0.06, +0.62]
Extreme prices	3,787,202	0.5230	0.5385	0.5420	0.5377	−0.07	[−0.51, +0.34]
Central prices	6,076,923	0.5348	0.5602	0.5619	0.5748	+1.46	[+1.15, +1.75]

Table 5.9: Time-first encoder against the reference GRU: seed 1 at 1h and seed 0 at 6h (balanced accuracy; differences in points with 95% hour-block bootstrap intervals).

Horizon	Subset	n	MR	GRU K=10	Time-first K=64	Δ [95% CI]
1h, seed 1	All samples	9,864,125	0.5311	0.5546	0.5635	+0.89 [+0.54, +1.22]
	Deduplicated	1,255,651	0.5180	0.5437	0.5450	+0.13 [−0.19, +0.46]
	Extreme prices	3,787,202	0.5230	0.5409	0.5403	−0.06 [−0.48, +0.34]
	Central prices	6,076,923	0.5348	0.5588	0.5725	+1.37 [+1.04, +1.70]
6h, seed 0	All samples	9,321,125	0.5069	0.5214	**0.5396**	+1.83 [+1.41, +2.21]
	Deduplicated	1,234,892	0.5002	0.5333	0.5370	+0.38 [+0.13, +0.64]
	Extreme prices	3,744,385	0.4879	0.5136	0.5285	+1.49 [+0.94, +2.05]
	Central prices	5,576,740	0.5199	0.5260	0.5468	+2.08 [+1.81, +2.36]

an event clock, one every 20 book updates, with no channel for the time elapsed between them, so a 64-snapshot window covers seconds for a busy market and hours for a quiet one. The time gap between snapshots is the obvious channel to add next.

Why reading time before space helps

The reference GRU condenses each book state into one vector before it sees any sequence, so the temporal pattern of a single quantity (the ask depth draining over the last minute, say) must survive a 55-to-128 compression at every step before the recurrence can notice it. The time-first encoder gives each quantity its own series and a shared map from series to features, so such a pattern is represented directly, and the attention across channels then decides which patterns matter together. This is the channel-independence argument of Nie et al. [33] applied to an order book, and the evidence that it is the order of operations and not the window length is the control: a GRU over three times as many snapshots gets a quarter of the gain. The gain lives where the order-book series carry information, the central price range, and vanishes at the extremes, where the book is one tick wide and the series are noise; the deduplicated samples are dominated by the extremes (Section 5.2.1), which is why they show little. The shared linear time map also makes the encoder a cousin of the linear model of Zeng et al. [32]: per channel it *is* a linear model on the normalised window, and the attention only mixes its outputs.

5.3.5 The state given the control's inputs

If the control of Section 5.2.2 wins because of what it reads and not because of what it is, the stateful model should recover most of the gap once it reads the same things. The sixty-one features were therefore given to the best 1h model, the time-first encoder with five lookbacks, as described in Section 4.4.1. Table 5.10 gives the result on the model's own validation samples and against the control on the control's samples.

The features are worth two points on the full stream and three on the deduplicated and extreme samples, the largest single gain of the study and the first one that is larger on the

Table 5.10: The best 1h model with and without the control's features, seed 0 (balanced accuracy; differences in points with 95% hour-block bootstrap intervals).

Subset	MR	Time-first	+ **features**	Δ vs time-first	LightGBM	Δ vs LightGBM
All samples	0.5311	0.5643	**0.5842**	+1.99 [+1.65, +2.37]	0.6015	−1.73 [−2.28, −1.15]
Deduplicated	0.5180	0.5410	**0.5708**	+2.97 [+2.78, +3.16]	0.5827	−1.20 [−1.57, −0.79]
Extreme prices	0.5230	0.5377	**0.5667**	+2.90 [+2.51, +3.29]	0.5804	−1.36 [−1.84, −0.79]
Central prices	0.5348	0.5748	**0.5910**	+1.62 [+1.28, +2.00]	0.6107	−1.97 [−2.58, −1.29]

independent samples than on the full stream: what the model lacked at the extreme prices was the activity clock and the price level, and both are now in front of it. The gap to the trees is halved, not closed. Calibration is not where the remaining 1.7 points are. The model trained with the focal loss predicts a move for 53% of the samples against 45% in the labels, but the post-hoc logit adjustment by the training class prior [43], which is consistent for the balanced error when the probabilities are calibrated, is destructive here (0.527): the focal weights already shift the probabilities towards the rare classes, and the adjustment counts the shift twice. The cross-fitted version, two logit offsets fitted on the even validation hours and applied to the odd ones and vice versa, comes out at $(-0.1, -0.1)$ and changes nothing (0.5837). The decision rule of the model is already balanced-accuracy-optimal to within a tenth of a point; what is missing is in the function it has learned.

Two follow-up runs, each paired against the run with the features, exclude the two obvious explanations of the residual. Training with the control's objective, a class-balanced cross-entropy in place of the focal loss, loses 0.47 points [−0.74, −0.22] on the full stream and 1.95 [−2.13, −1.78] on the deduplicated samples. A predictor head four times wider and one layer deeper (256 units, two layers), motivated by the 581 inputs that the default head maps through 64 units, loses 0.37 [−0.70, −0.08] and 1.90 [−2.12, −1.69]. Both lose exactly where the features had gained most, the deduplicated and the extreme samples, and tie on the central prices; both predict fewer moves than the focal-loss run, a class mix within a point of the labels', and on the sparse subsets that is what balanced accuracy punishes. With two logit offsets fitted crosswise on the validation hours the three runs become indistinguishable (0.5837, 0.5848, 0.5849): the loss and the head moved the decision bias of the model, not the function it learned.

The experiment of Section 5.3.5 confirms the first reading and sharpens the third. Given the control's inputs, the recurrent model recovers half of the gap at once (two points on the full stream, three on the independent samples), so most of what the trees knew was indeed a matter of inputs. The half that remains is not a matter of calibration, since the decision rule is already optimal, and it is not obviously a matter of capacity either: it is a difference between a model that sees every training row once, in time order, through a 64-unit head, and six hundred rounds of boosting over the same seven million rows. Which of the two it is, the training regime or the head, is the question the follow-up runs are meant to answer.

5.3.6 The cost of the online regime

A learner comparison on the control's own table, its 61 features, its training rows and its validation samples (Table 5.11), separates the candidate causes. An MLP of the shape of the wide head, trained as a tabular model is trained (shuffled mini-batches, four passes, early stopping on 12 March), is 0.77 points below the trees; with a piecewise-linear encoding of each numeric feature over its training quantile bins [83] it is 0.41 below. The same MLP trained as the deep model is trained, one pass over the rows in time order at the same learning rate, is 2.77 below, and the deep model sits between the two. Neither the learning rate nor the number of steps explains it: a ten times larger rate in time order makes the one-pass MLP

Table 5.11: The control's table (61 features, training rows of 7–11 March, validation samples of 13 March) under different learners, and the combination of LightGBM with other models (geometric mean of the probabilities, weight cross-fitted on the validation hours). Balanced accuracy; differences from LightGBM in points with 95% hour-block bootstrap intervals.

Learner	All	Δ	Dedup.	Δ
LightGBM	0.6015		0.5827	
MLP, shuffled, four passes, early stopping	0.5938	−0.77 [−0.99, −0.54]	0.5757	−0.70
MLP + piecewise-linear encoding	0.5974	−0.41 [−0.65, −0.17]	0.5783	−0.44
MLP, one pass in time order (the deep model's regime)	0.5738	−2.77 [−3.27, −2.18]	0.5623	−2.04
MLP, one pass in time order, rate ×10	0.5675	−3.40 [−3.79, −2.97]	0.5355	−4.72
MLP, one pass shuffled	0.5818	−1.97 [−2.34, −1.59]	0.5649	−1.79
MLP, two passes in time order	0.5802	−2.13 [−2.62, −1.61]	0.5701	−1.26
Deep model + features	0.5842	−1.73 [−2.27, −1.13]	0.5708	−1.20
LightGBM ⊕ deep model + features	**0.6035**	+0.19 [+0.12, +0.27]	**0.5850**	+0.23
LightGBM ⊕ MLP + piecewise-linear encoding	0.6020	+0.05 [−0.02, +0.11]	0.5829	+0.02

worse, while one shuffled pass at the same rate recovers 0.8 points and a second pass in time order 0.6. The regime that the state imposes, one pass in time order, costs about two points on this table; the learner itself costs less than one. The combination of LightGBM with the deep model's probabilities, a geometric mean with the weight chosen on the even validation hours and applied to the odd ones and vice versa, is the first number of the study above the trees, +0.19 [+0.12, +0.27] on all samples and +0.23 on the deduplicated ones, whereas the same combination with the tabular MLP gives nothing (+0.05 [−0.02, +0.11]). What the state adds is small, but it is there, and it is not what another learner on the same table would add.

Two passes and weight averaging. The two regime levers that the comparison suggests were then tried on the deep model itself. A second pass over the seven training days (states reset between the passes, optimizer kept) gives 0.5907, 1.08 [−1.56, −0.59] below the trees on all samples and 0.46 [−0.75, −0.14] below on the deduplicated ones: the largest standalone improvement after the features, and in the direction the tabular comparison predicted. An exponential moving average of the weights along the pass (decay 0.998 and 0.999 per optimizer step, horizons of about 500 and 1,000 steps out of 3,000), swapped in before validation, does not help on its own: 0.5819 and 0.5795 on all samples, with a decision bias toward DOWN that two cross-fitted offsets only partly remove (0.5876 and 0.5859), while on the deduplicated samples both tie with the trees (0.5805, −0.22 [−0.68, +0.28]). What the averaged weights change is their complementarity: combined with LightGBM as above they give +0.68 [+0.57, +0.81] and +0.72 [+0.62, +0.84] on the deduplicated samples, three times the +0.23 of the run with features, and the two-pass run combined gives +0.26 [+0.16, +0.37] on all samples. The sequential state is worth most exactly where the trees are weakest, on the sparse markets, and the smoother weights carry that part better than the last ones.

5.3.7 The state as the residual of the trees

The residual model of Section 4.4.2 stays paired with the run with features (same loss, same seed, same samples). The cross-fitted trees reproduce the control on its own samples once their fit rows follow the control's rule (markets of the model's universe, last event at or after $t + H$): 0.6020 on all samples and 0.5811 on the deduplicated ones against 0.6015 and 0.5827, with 94.6% of the decisions in common; fitted on every tensor row they were 1.6 points below, a reminder that the population of the fit matters as much as the learner.

Table 5.12: The deep model as the residual of the trees (1 h, validation samples of the control). Balanced accuracy; differences from LightGBM in points with 95% hour-block bootstrap intervals; "offsets" = two logit offsets (DOWN, UP) fitted on the even validation hours for the odd ones and vice versa.

Subset	LightGBM	Residual	Δ	Residual, 2 passes	Δ	2 passes + offsets
All	0.6015	0.6024	+0.08 [−0.17, +0.34]	0.6024	+0.09 [−0.24, +0.46]	0.6048
Dedup.	0.5827	0.5862	+0.35 [+0.18, +0.51]	**0.5945**	+1.18 [+0.94, +1.42]	0.5933
Extreme	0.5804	0.5817	+0.13 [−0.13, +0.40]	**0.5854**	+0.50 [+0.21, +0.81]	0.5865
Central	0.6107	0.6112	+0.05 [−0.25, +0.37]	0.6093	−0.14 [−0.52, +0.33]	0.6123

Table 5.12 gives the result. The residual model is the first run of the study at or above the control on every subset: 0.6024 on all samples (+0.08, inside the interval) and 0.5862 on the deduplicated ones (+0.35 [+0.18, +0.51]); against the run with features the gain is +1.5 to +2.0 points on every subset, paired, more than every other lever combined. A second pass over the training days moves the gain where the trees are weakest, +1.18 [+0.94, +1.42] on the deduplicated samples and +0.50 [+0.21, +0.81] on the extreme ones, at no cost on all samples; with two cross-fitted logit offsets the two-pass residual model is at or above the control on every subset (0.6048 on all samples, +0.33 [−0.01, +0.64]). This is trees plus state, not a network beating the trees, and the number that measures the state is the gain over the tree base (from 0.6020 / 0.5811 to 0.6024 / 0.5945), concentrated on the sparse and extreme markets where a tabular model has least to work with. The post-hoc combination of the residual model with LightGBM adds a further +0.27 [+0.10, +0.43].

The same calibration for everybody

The offsets are a post-hoc calibration that any model can have, and the comparison is only fair when every model has it. Applied to LightGBM they are worth +0.29 on all samples and +1.15 on the deduplicated ones (0.6044 and 0.5943): most of what the residual runs gained on the sparse markets was a better-placed decision threshold. Calibrated against calibrated, the two-pass residual model ties the trees on every subset (+0.04 [−0.18, +0.24] on all samples, −0.10 [−0.26, +0.05] on the deduplicated ones), the one-pass residual model is at −0.05 and −0.38, and the two-pass model without trees at −1.42. At a one-hour horizon, then, the sequential state does not add measurable balanced accuracy on top of calibrated trees; it adds a combination gain of the same size as the calibration gain, which may be the same thing. This is the result the thesis reports at one hour, and the reason Section 5.5 looks for information the book does not contain rather than for a better learner of the same table; Section 5.6 shows that the answer is different at one minute.

5.3.8 What the state is worth

Read together, the section gives a precise account of the stateful family. Its own improvements are real and explicable: a direct path that contains the rule, a memory at three rates, five lookbacks and an encoder that reads time before space take the LOB-only model from eight points below Mean Reversion to three above it at one hour. Its inputs were its main handicap: the activity clock and the price level, which the event-clock tensor hides, are worth two to three points once they are put in front of it. Its training regime, one pass in time order, costs about two points against the same learner trained as a tabular model, and a second pass recovers a third of that. And on top of calibrated trees that read the same inputs, at one hour, the state adds nothing measurable on its own and a small combination gain. Where a state should matter is where a summary of the recent past is not enough, the next minute of

Table 5.13: Graph contribution at each stage (LOB-only / graph, all samples, seed 0).

Configuration	1h	Δ	12h	Δ
lookback	0.5038 / 0.5037	−0.01	0.4368 / 0.4400	+0.32
+ hierarchical memory	0.5161 / 0.5184	+0.23	0.4706 / 0.4559	−1.47
+ direct path + global state	0.5452 / 0.5461	+0.09	0.4727 / 0.4724	−0.03
+ five lookbacks (best)	0.5547 / 0.5536	−0.11 [−0.21, 0.00]	0.4914 / 0.4814	−0.99 [−1.15, −0.81]

a quiet market, and Section 5.6 finds it exactly there.

5.4 Graph models

The graph models read a dynamic graph between the wallets and the markets they trade, by message passing at every block. It is the clearest negative result of the thesis, and this section gives the result at every stage of the design, the two direct tests of what wallets know, the market–market view, and the reasons.

5.4.1 The wallet–market graph at every stage

Table 5.13 isolates the contribution of the wallet–market edges at every stage. It is never above +0.3 at 1h, it changes sign across configurations at 12h, and in the best configuration it is −0.1 [−0.2, 0.0] at 1h and −1.0 [−1.2, −0.8] at 12h; at 6h it is +0.65 [+0.20, +1.11] on all samples but −0.54 [−0.83, −0.27] on the deduplicated subset. The edge-orientation fix moves the graph effect on the affected (token-2) markets from $-0.6/-0.3$ to $+0.05/+0.4$ points at 1h/12h in a paired short test, with no overall gain because those markets are 4–9% of the samples. Wallet-skill features never beat an activity-matched placebo (−0.4 at 1h, 0.0 at 12h).

5.4.2 Smart money

A direct test of "smart money" outside the model confirms the picture. For every wallet the past hit rate at the horizon, the share of its trades on the side the price then moved to, is computed on the training days and compared with the validation day: the Spearman correlation is 0.5–0.6, so skill is strongly persistent at the wallet level (Section 3.7.1). As a market-level predictor, the activity-weighted skill of the wallets that traded a market in the last hour adds only about one point over an identity-free placebo (random wallets with the same activity) and does not beat Mean Reversion. Inside the model, the same record given as an edge feature never beats the placebo skill (−0.4 at 1h, 0.0 at 12h), and the wallet's past behaviour (activity, maker share, one-sidedness, size, breadth) gives ±0.3 with no consistent sign.

5.4.3 The market–market view: sibling contracts

The ten sibling features of Section 3.6.2 were added to the control's table. They lose −0.10 [−0.18, −0.02] on all samples and −0.25 on the political markets, tie on the sports markets of the same game (−0.06 [−0.28, +0.15]), and the best of them, the sum of the siblings' mids, is the twenty-fifth feature by gain. At one hour, after the market's own returns and activity clock, what its siblings did in the last hour does not change the decision. This is also the answer to whether a market-to-market graph would help the deep model at this horizon: the information such edges would carry is the information these features carry, and the trees found none; and the lead–lag statistic of Section 3.7.1 shows that at the minute scale, where

the siblings of a game might move first, they do not: the common cause reaches both books within the same minute.

5.4.4 Why the graph does not help

The bipartite transaction graph is the clearest negative result of the thesis. Four observations explain it.

1. **The temporal signal dominates.** Every improvement in Section 5.3.3 came from temporal components, and once they are strong the graph's small positive effects at 12h in weaker configurations disappear.
2. **Wallet memory is short.** The decay is shared by wallets and markets, so a wallet state is erased within a minute of its last trade; most wallets trade far less often than that, and their state carries no history when they reappear. A separate slow decay for wallets was proposed and not tested; the smart-money test suggests it would not change the conclusion.
3. **Wallet skill is real but not market-level information.** Past accuracy persists strongly at the wallet level, but a market is traded by many wallets and their aggregate is close to the placebo of random wallets with the same activity.
4. **Most of the signal a wallet brings is already in the book.** A trade is a book event; the order-flow features see it before the edge does.

In sum, the temporal component weighs far more than the graph on this data. The same conclusion emerges from the literature on spatiotemporal benchmarks [75]: per-node baselines are hard to beat and a graph must prove its value against them.

What would have to be different for a graph to help is also clear from the data. A wallet memory at a time scale of days rather than seconds, so that a wallet's state survives between its trades; an edge feature that is not already a book event; or a market–market graph on a question where the common cause reaches one book before the other, which the lead–lag statistic did not find at the minute scale in this data. A learned, sparse market–market graph restricted to the markets of the same event, in the spirit of Cini, Zambon, and Alippi [76], remains the version of the idea worth testing.

5.5 Information outside the book

The last family does not change the learner; it changes what the learner is shown. Once the stateless control and the stateful model read the same sixty-one features and tie at one hour (Section 5.3.7), the remaining question is whether anything the book does not contain predicts the price: a pretrained forecaster's view of the price series, the news flow about the market's subject, or the text of the market itself.

5.5.1 Time-series foundation models

Zero-shot. Chronos-2 was given, for 60,000 random validation samples per horizon, the last 512 mids of the market on a one-minute grid (1h) or five-minute grid (12h), ending at the sample time, and asked for the quantiles of the mid at $t + H$. Table 5.14 turns the forecast into a decision in several ways. The sign of the median almost never predicts CONST (17% of the samples at 1h against 55% in the labels) and scores 0.470; with a half-tick dead zone it reaches 0.505, still -2.6 [-3.4, -1.9] below Mean Reversion at 1h and -0.4 [-1.4, $+0.5$] at 12h. Where Mean Reversion takes a side, Chronos agrees with it 65–68% of the time; where they disagree and the price moves, Chronos is right 52.9% of the time at 1h and 48.0% at 12h. The foundation model re-discovers mean reversion in context.

Table 5.14: Zero-shot Chronos-2 on 60,000 validation samples per horizon.

Rule	1h	12h
Our model (same samples)	0.559	0.495
Mean Reversion-H	0.531	0.508
Chronos-2, sign of the median	0.470	0.454
Chronos-2, median with half-tick dead zone	0.505	0.504
Chronos-2, quantile bands	0.34–0.43	0.34–0.42

Table 5.15: Best 1h model with and without the Chronos-2 forecast features, two seeds (balanced accuracy; differences in points with 95% hour-block bootstrap intervals; the last column is the difference of the seed-1 pair).

Subset	n	Best	+ features	Δ (seed 0)	95% CI	Δ (seed 1)
All samples	9,864,125	0.5547	0.5576	+0.29	[+0.05, +0.53]	+0.57
Deduplicated	1,255,651	0.5382	0.5380	−0.02	[−0.18, +0.14]	+0.14
Extreme prices	3,787,202	0.5385	0.5309	−0.76	[−1.02, −0.50]	−0.16
Central prices	6,076,923	0.5602	0.5687	+0.85	[+0.53, +1.17]	+0.93

As features. The same quantiles were then cached for every active market on a five-minute grid over 6–13 March (2.6–3.1 million forecasts per day, context of 512 one-minute mids ending strictly before the grid time) and given to the best 1h model as eleven extra inputs: the ten-fold distance of the 10/50/90% quantiles at 15, 30 and 60 minutes from the *current* mid, the sign of the 60-minute median change and an availability flag; a forecast is used for at most eight minutes. They enter both the order-book window and the direct path (Section 3.6.2). Table 5.15 compares the two runs on the same validation samples. The gain on all samples is +0.29 points, ten times the seed spread of the base model and with an interval that excludes zero, but it vanishes on the deduplicated samples and is the net of two opposite effects: +0.85 in the central price range and −0.76 at the extremes, where the zero-shot study had already shown the forecasts to be noise. The direct path reads one forecast feature for direction, the sign of the 60-minute median (the largest directional weight in the layer, read *with* the forecast), and uses the width of the quantile band for the CONST logit; the lookback weights are unchanged to two decimals. A second seed of both runs gives the same picture (+0.57 [+0.33, +0.82] on all samples, +0.93 central, −0.16 extreme, +0.14 deduplicated): the gain on the full stream and in the central range is reproducible, the loss at the extremes keeps its sign but not its size.

Why the foundation model re-discovers mean reversion

Chronos-2 standardises each context window and forecasts its continuation. On a one-tick process the standardised window is noise around a level, and the model's best in-context guess is a return to the level: its median moves against the last change 68% of the time Mean Reversion does. It is worse than the rule for two reasons: it never predicts "no change" (a dead zone recovers most of the gap), and at extreme prices the per-window standardisation amplifies one-tick noise into forecasts that are 8 points below the rule. This is the finance literature's finding [56, 57] reproduced on prediction markets: value, if any, must come from a learned correction on top of the frozen forecast. That is the experiment of Section 3.6.2, and its outcome (Table 5.15) fits the same picture. Given to the best model as inputs, the forecasts add about 0.9 points where prices are central (both seeds) and cost between 0.2 and

0.8 where they are extreme, for a net +0.3 to +0.6 on the full stream and nothing reliable on independent samples. The direct path learns to use a single bit of the forecast, the sign of the 60-minute median change, with the largest directional weight in the layer; the rest of the information it extracts is the width of the quantile band, which it reads as the probability of "no change". In other words, the foundation model is used as a smoothed second opinion on the direction and as a volatility estimate, both of which the recurrent state already has in the central range and both of which are unreliable at the boundaries. The natural fix, masking the forecast outside the central range or giving the direct path the price level so that it can do the masking itself, was not run.

5.5.2 News volume

The twelve news features of Section 3.6.2, added to the control's table, change nothing: −0.10 [−0.20, +0.00] on all samples, −0.06 [−0.16, +0.04] on the political markets, which are 38% of the samples, and −0.27 on the rows whose market has no entity at all, the signature of noise features displacing useful splits; the twelve features take 3% of the trees' gain. At one hour the news flow about a market's entities is either already in the book or unrelated to the contract's question, and a finer alignment of articles to questions is a different project. The deep model was therefore not run with these inputs.

5.5.3 The text of the market

The sentence embedding of the question, as used to initialise the market state, has no effect (Section 3.5.2); what the text could contribute is the *type of contract* (Section 3.7.1). Questions are templated (3,528 templates among the active markets), and the template explains 55% of the between-market variance of the probability of a move within one hour, from 4% for House-seat markets to 83% for daily-temperature markets; a ridge regression on the embedding predicts that probability out of sample with $R^2 = 0.22$, no better than the template means. The cheap way to use the text is a learned embedding of the category or template fed to the predictor; the control gets a fifth of a point from the static fields, on top of everything else.

5.5.4 What is outside the book

The three sources give one answer at one hour. A frozen forecaster of the price series knows what the market's own history knows, mean reversion and a volatility estimate, and is useful only as a smoothed second opinion in the central price range; the news flow about a market's entities is either already in the book or unrelated to the contract's question; and the text matters through a handful of contract types. Information outside the book that would move a one-hour prediction, the reference price of a crypto interval market [14], the score of a game, the content rather than the volume of the news, was not available to this study, and Section 5.6.5 shows where its absence is felt.

5.6 Across horizons, kinds of market and seeds

The previous sections built and assessed the families mostly at one hour. This section reads the same families across the five horizons of the study, one minute, fifteen minutes, one hour, six and twelve hours, because the horizon changes what the task is and which family can do it; then by kind of market; then for robustness, across seeds, calibration and run length. The locked test set and the return to the four questions follow in Sections 5.7 and 5.8.

Table 5.16: Short horizons against one hour: the no-state control, the tabular network on the control's table and the stateful model, every one with its own two cross-fitted logit offsets (DOWN, UP), on the runs' validation samples of 13 March. Balanced accuracy after calibration; Δ in points against the calibrated LightGBM with 95% hour-block bootstrap intervals. Labels DOWN / CONST / UP: 5.7/89.1/5.2% at 1 min, 16.4/69.0/14.6% at 15 min.

Horizon	Model	All	Δ [95% CI]	Dedup.	Δ [95% CI]
1 min	LightGBM G5 (offsets 1.6, 1.6)	0.6623	—	0.6143	—
	TabM, 3 seeds	0.6656	+0.33 [+0.26, +0.39]	0.6133	−0.10 [−0.31, +0.10]
	Stateful, 2 passes	0.6685	+0.62 [+0.46, +0.82]	0.6317	+1.74 [+1.31, +2.22]
	Stateful, 2 passes, residual	**0.6760**	**+1.37** [+1.16, +1.58]	**0.6346**	**+2.04** [+1.76, +2.30]
15 min	LightGBM G5 (offsets 0.4, 0.4)	0.6213	—	0.6197	—
	TabM, 3 seeds	**0.6235**	+0.22 [+0.07, +0.36]	**0.6248**	+0.51 [+0.35, +0.68]
	Stateful, 2 passes	0.6206	−0.07 [−0.34, +0.19]	0.6158	−0.39 [−0.63, −0.13]
1 h	LightGBM G5 (offsets 0.2, 0.3)	0.6044	—	0.5943	—
	TabM, 3 seeds	0.6033	−0.11 [−0.29, +0.06]	0.5922	−0.21 [−0.45, +0.00]
	Stateful, 2 passes	0.5902	−1.42 [−1.83, −1.00]	0.5797	−1.46 [−1.76, −1.14]
	Stateful, 2 passes, residual	**0.6048**	+0.04 [−0.18, +0.24]	0.5933	−0.10 [−0.26, +0.05]

5.6.1 Short horizons: one and fifteen minutes

At a horizon of one minute the task changes: 89% of the labels are CONST, the direction of a move is nearly unpredictable and the whole gain is in whether the mid will move (Section 3.7.1). The early LOB-only models with undersampling scored 0.550 at 60 s against 0.536 for Mean Reversion, 0.532 at 120 s and 0.518 at 300 s; the numbers below are from the final configuration of Section 5.3.5 (five lookbacks, hierarchical memory, time-first encoder, hand features, two passes), trained at 60 s and 900 s. The controls of Table 5.3 sample one tensor row in twenty at the row's time, with the label of the last tensor row before $t + H$; the deep runs sample every market of every block at the block's time, with the label of the mid-price index. At one hour the two populations coincide, because the control was built from the run's own predictions; at one minute only 27 of the 598,872 tensor rows coincide with a run sample. The control was therefore rebuilt on the runs' own validation samples at each horizon (9.9 M samples at both 60 and 900 s), with the same features, learner and protocol as in Section 5.2.2. Every model then receives the same two cross-fitted logit offsets of Section 5.1, with the grid widened to ±2: at one minute both learners sit at the edge of the ±0.6 grid used at one hour, and the offsets are worth +5.4 points to LightGBM (0.6086 → 0.6623) and +5.8 to the network. Any one-minute number without this calibration compares decision thresholds, not models; at fifteen minutes the same offsets are worth +0.5, and at one hour +0.3.

Table 5.16 gives the result, and it reverses with the horizon. At one minute the stateful model without trees is +0.62 [+0.46, +0.82] above the calibrated trees on all samples and +1.74 [+1.31, +2.22] on the deduplicated ones, ahead on every subset (+0.45 at extreme mids, +0.64 central). TabM on the same sixty-one features and the same samples, with the same calibration, is +0.33 on all samples and −0.10 on the deduplicated ones: half of the gain on all samples is the learner, and the whole of the gain on the sparse markets is the state, +1.84 over TabM. Given the cross-fitted trees as a base, the residual model is the best model of this thesis at one minute, +1.37 [+1.16, +1.58] on all samples and +2.04 [+1.76, +2.30] on the deduplicated ones, above the calibrated trees on every subset and +1.0 to +2.1 above TabM. At fifteen minutes the stateful model ties the calibrated trees on all samples (−0.07 [−0.34, +0.19]) and is below them on the deduplicated (−0.39) and extreme (−0.27) ones, while TabM is +0.22 and +0.51: against the same learner the online, time-ordered

regime costs −0.3 on all samples and −0.9 on the sparse ones. At one hour it costs −1.4 and the residual model only recovers it (Section 5.3.7). Before calibration the ordering is the same but the gaps are not: the raw stateful model is +0.23 (not significant) at one minute and −0.76 [−1.08, −0.45] at fifteen, and the prior correction of Section 5.3.5 is worse than nothing at fifteen minutes (0.6084 → 0.5587).

The reading is that the window of book events, the five lookbacks and the memory carry information about the next minute of a market that the sixty-one summary features throw away, chiefly on the quiet markets where a summary of the last hour says little about the next sixty seconds, and that by fifteen minutes the summary is complete and the sequential regime only costs. The gain is small in absolute terms, under two points on a scale where the activity clock alone is worth seven, but it is the one place in this study where the stateful model beats a calibrated tree model on the same inputs with an interval that excludes zero on every subset. The residual model was not run at fifteen minutes.

5.6.2 One hour

One hour is the horizon at which most of the design was done, and its result is the tie of Section 5.3.7: calibrated against calibrated, the trees on sixty-one hand-made features (0.6044 on all samples, 0.5943 deduplicated), TabM on the same table (−0.11), and the two-pass residual stateful model (+0.04) are within their intervals of one another, the stateful model without trees is −1.4, and the best LOB-only model without the hand features is four points below. Half of the labels are CONST; the activity clock decides whether the price moves and the reversal of the recent change decides which way; the combination of the stateful model with the trees adds two to three tenths of a point, most of it on the sparse markets. The one-hour task is a summary task: everything a model needs is in a few dozen numbers computed at the sample time.

5.6.3 Six and twelve hours

At six and twelve hours the CONST class shrinks to a third of the samples and the direction is the task. The LOB-only stateful model of Section 5.3.3 is +1.5 above Mean Reversion at six hours and −1.6 below it at twelve on all samples (above on the deduplicated samples at both, +3.3 and +1.7), and the time-first encoder raises the six-hour gain to +3.3 on every subset; the hierarchical memory is worth most at twelve hours (+3.4), the horizon at which the morning matters most, and the three seeds of the twelve-hour model are below the rule on the full stream and above it on independent samples. The no-state control is six and a half points above the rule at both horizons (Table 5.3) and the price history alone already beats the stateful model (+1.7 at 6h, +5.3 at 12h): the long-horizon signal is a slow reversal of the 6h and 12h returns, modulated by the price level and by where the market is in its life (time to the end, age, duration, about 9% of the gain importance together), which trees read directly and a direct path that is linear in the returns cannot bend to the level. The stateful model with the hand features and the residual construction were not run at these horizons; the stateless control is the reference there.

5.6.4 Synthesis

Table 5.17 and Fig. 5.3 put the best model of each family at each horizon on one page. Three regimes appear. At one minute the task is activity, calibration is worth five points to every model, and the sequence of book events carries information about the next minute of a quiet market that the summary features lose: the stateful model beats calibrated trees by +0.6 alone and +1.4 as their residual, with intervals that exclude zero on every subset, and the

Table 5.17: The best model of each family at each horizon, all samples and deduplicated samples (balanced accuracy). At 1 min, 15 min and 1 h every model carries its own two cross-fitted logit offsets; at 6 h and 12 h the raw argmax (the stateless control was not calibrated there and the stateful models' probabilities were not saved). "Stateless" is LightGBM G5; "stateful" is the best run without trees; "residual" is the two-pass stateful model on the cross-fitted trees. Δ in points against the stateless control.

Horizon	Subset	MR	Stateless	Stateful	Δ	Residual	Δ
1 min	All	0.5363	0.6623	0.6685	+0.62 [+0.46, +0.82]	**0.6760**	+1.37 [+1.16, +1.58]
	Dedup	0.5012	0.6143	0.6317	+1.74 [+1.31, +2.22]	**0.6346**	+2.04 [+1.76, +2.30]
15 min	All	0.5373	**0.6213**	0.6206	−0.07 [−0.34, +0.19]	—	
	Dedup	0.5253	**0.6197**	0.6158	−0.39 [−0.63, −0.13]	—	
1 h	All	0.5311	0.6044	0.5902	−1.42 [−1.83, −1.00]	**0.6048**	+0.04 [−0.18, +0.24]
	Dedup	0.5180	**0.5943**	0.5797	−1.46 [−1.76, −1.14]	0.5933	−0.10 [−0.26, +0.05]
6 h	All	0.5069	**0.5716**	0.5396	−3.20	—	
	Dedup	0.5002	**0.5665**	0.5370	−2.95	—	
12 h	All	0.5071	**0.5715**	0.4914	−8.01	—	
	Dedup	0.4995	**0.5680**	0.5168	−5.12	—	

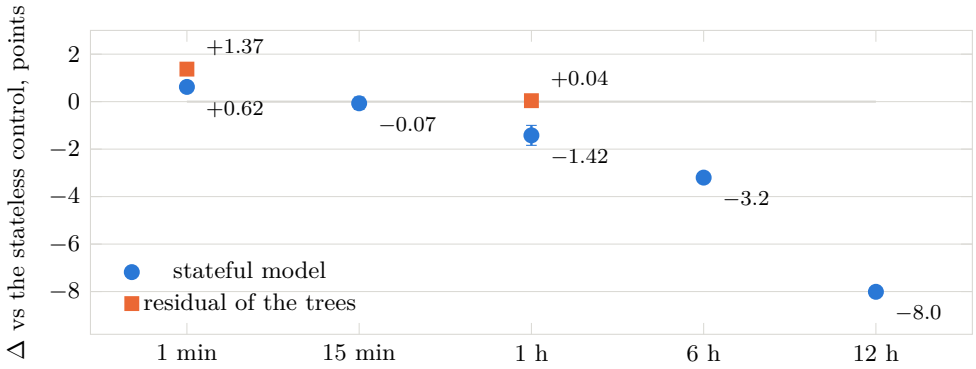


Figure 5.3: The gain of the stateful family over the stateless control at each horizon (all samples; calibrated against calibrated at 1 min, 15 min and 1 h with 95% hour-block bootstrap intervals; raw argmax at 6 h and 12 h). The sign reverses with the horizon.

gain over the same learner without state is concentrated on the sparse markets. At fifteen minutes and one hour the summary is complete: trees, a tabular network and the stateful model with the same inputs tie, the online regime costs the stateful model one to two points unless it is given the trees as a base, and the graph and the external sources add nothing. At six and twelve hours the task is the direction of a slow reversal read together with the price level and the market's lifecycle, the stateless control is six points above the rule and the stateful models of this study are below it. The same architecture cannot be the right one at both ends, and the reading the data give is that a model of this market should be a stateless learner on the activity clock, the first level, the long returns, the price level and the contract type, with a sequential state added only where the horizon is shorter than the summary.

5.6.5 Where the models win, by kind of market

The per-market analysis at 1 h (Table 5.18; the lookback model with the graph against Mean Reversion on the same 9.86 million samples, before the direct path and the hand features) shows where a learned model of the book is strong or weak relative to the rule. The model is above the rule only on weather markets (+1.8) and on markets that last one hour or less (+13.7, a small group of 402 markets in which the rule is near chance); it is closest to the

Table 5.18: Balanced accuracy at 1 h by kind of market: the lookback stateful model (graph variant, seed 0) against Mean Reversion on the same validation samples, with the class mix of each group. Δ in points.

Group	samples	markets	CONST share	Model / MR	Δ
By category					
politics	3,762,863	4,502	0.64	0.500 / 0.543	−4.3
sports	3,088,176	6,720	0.54	0.512 / 0.535	−2.3
culture	1,404,248	726	0.37	0.474 / 0.490	−1.6
economy, tech	889,481	1,197	0.55	0.470 / 0.498	−2.8
crypto	292,307	1,160	0.56	0.461 / 0.526	−6.5
weather	200,158	696	0.30	0.480 / 0.463	+1.8
esports	150,765	1,213	0.33	0.504 / 0.517	−1.3
By time to the end date					
less than 1 h	49,044	539	0.09	0.371 / 0.425	−5.4
1 h to 6 h	136,031	1,250	0.25	0.490 / 0.466	+2.4
6 h to 1 day	219,307	1,615	0.55	0.472 / 0.471	+0.1
1 day to 1 week	2,260,895	2,798	0.44	0.483 / 0.505	−2.2
more than 1 month	4,914,916	7,711	0.65	0.496 / 0.532	−3.7
By market duration					
1 h or less	24,215	402	0.15	0.514 / 0.376	+13.7
1 h to 1 day	413,522	1,466	0.17	0.466 / 0.479	−1.3
more than 3 months	5,954,469	8,484	0.61	0.501 / 0.531	−3.0
Sports, by phase					
before the game	1,160,800	3,932	0.63	0.467 / 0.494	−2.8
game started	557,082	1,818	0.11	0.464 / 0.530	−6.6
By traded volume in the training days					
top quartile	3,071,194	2,291	0.48	0.510 / 0.512	−0.3
bottom quartile	1,373,930	2,159	0.65	0.493 / 0.543	−4.9
no trades	1,987,456	7,179	0.49	0.499 / 0.552	−5.3

rule on markets one to six hours from their end date (+2.4) and on the most traded quartile (−0.3), and farthest on the fifteen-minute crypto markets (−6.5), on sports markets while the game is being played (−6.6), on political markets (−4.3) and on markets that had no trades in the training days (−5.3). By price level (Fig. 5.4) both the model and the rule are best between 0.05 and 0.2 and between 0.8 and 0.95, and the model collapses above 0.95 (−10.6), where the process is a one-tick oscillation that the rule reads directly. The pattern has one reading: the model wins where the book moves for reasons the book shows (weather resolves on a schedule, a short market lives inside its own activity), and loses where the price is driven by an external reference the model does not see, the exchange price for the crypto markets [14], the game for in-play sports, the news for politics. Every later model of the thesis inherits this map; the hand features and the trees raise every row, and the control's gain over the rule is positive in every category (Section 5.2.2), but the ordering of the categories, crypto and in-play sports hardest, does not change.

5.6.6 Robustness

Seeds. Three seeds of the best LOB-only configuration give 0.5547, 0.5546 and 0.5549 at one hour, a spread of three ten-thousandths, and the time-first encoder and the forecast features reproduce their gains on a second seed to within a tenth of a point (Sections 5.3.3, 5.3.4 and 5.5.1); the tabular network is reported as the mean of three seeds. At twelve

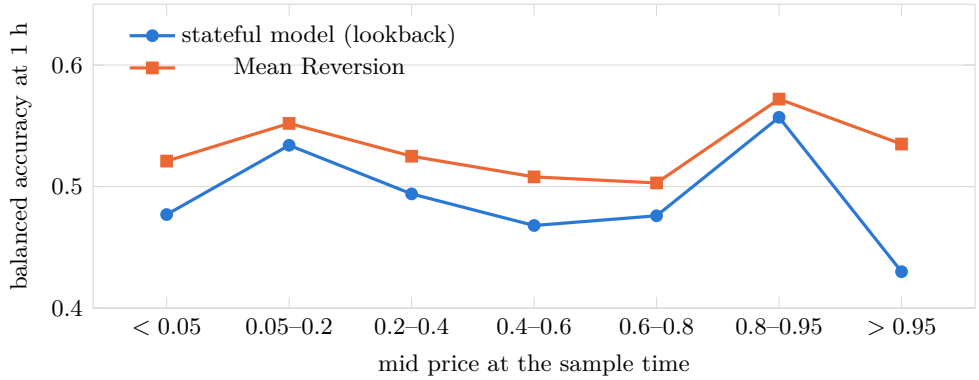


Figure 5.4: Balanced accuracy at 1 h by price level: the lookback stateful model against Mean Reversion on the same samples. Both are best just inside the boundaries and the model collapses above 0.95.

hours the seeds spread by 0.009 and straddle the rule on the full stream, which is why the twelve-hour result is stated as "real on independent samples, fragile on the full stream". The hour-block bootstrap intervals of every difference are the second guard: a difference whose interval includes zero is reported as a tie.

Calibration. The two cross-fitted logit offsets of Section 5.1 are worth +0.3 points to the trees at one hour and +5.4 at one minute, and the prior correction of Menon et al. [43] is destructive on a focal-loss model (Section 5.3.5); every ordering reported in this section survives the calibration, several gaps do not, and the one-hour tie between trees and the residual stateful model only appears once both are calibrated (Section 5.3.7).

Two views of every sample. Every number is given on the full stream and on the deduplicated subset, and the two disagree in a systematic way: the full stream rewards what is right on the busy markets, the deduplicated subset what is right on the sparse ones. The time-first encoder and the forecast features gain on the first; the hand features, the second pass and the residual construction on the second. A result that holds on one view only is reported as such.

Short tests versus full-length runs

A recurring methodological lesson: a two-day short test is not a reliable predictor of a seven-day result. The Mean-Reversion initialisation of the direct path gained two points in the short test and lost two at full length; the five-lookback and global-state additions kept their sign. The explanation is that a short test is in the regime where the recurrent part has learned little and the heuristic is all there is; the full run is in the regime where the residual has been learned. Every claim of the thesis rests on full-length runs, and the short tests are reported as what they are: screening.

5.7 Test set

The locked period of 24–26 March was evaluated once, after every choice of the preceding sections had been made, with the protocol below. The numbers of this section are the final ones of the thesis; where they differ from the validation day, the discussion of Section 5.8 follows them.

Protocol. The stateful models are the saved checkpoints of the validated runs, weights only, run over the test stream in evaluation mode with no learning: three warm-up days, 21–23 March, rebuild the per-market states, the hierarchical memory and the book buffers that a checkpoint does not hold, and the three days that follow are recorded. The evaluation mode was checked on the validation day first. Re-run on 13 March after a two-day warm-up, the one-hour reference checkpoint reproduces the hard predictions of its original validation file on 99.1% of the samples and its balanced accuracy to the third decimal (0.5547 against 0.5551); the two-pass residual model reproduces them on 99.99% of the samples, with a mean absolute difference in probability of 10^{-4}; the fourteen other checkpoints of the battery agree with their original files on 94.9 to 99.4% of the samples. The agreement is lowest at six and twelve hours, where the state lives longest, rises through the day in every run, and a seven-day warm-up moves it by less than one point: the residual disagreement comes from the original validation states, which were built while the weights were still changing, not from the length of the warm-up. The same re-runs supply the validation probabilities of the older runs, which had not saved them, so that at six and twelve hours the comparison is calibrated against calibrated here, as at the shorter horizons. Every model receives two logit offsets (on DOWN and UP) chosen once on the whole validation day, on the grid of ± 2 in steps of 0.1, and applied unchanged to the test days: unlike the cross-fitted offsets of Section 5.1, this is a calibration fixed on one day and used eleven days later, and how well it transfers is itself one of the results. The stateless control is LightGBM G5 refitted exactly as in validation (fit rows of 7–11 March, early stopping on 12 March, seed 0) and evaluated on the samples of the stateful test run of the same horizon, so that every comparison is paired; the tabular networks are trained on the same table; Mean Reversion is computed on the same samples; the intervals are 95% hour-block bootstraps over the test hours (2,000 resamples). The samples are the 17.1 million of the one-minute run and the 17.0 million of the fifteen-minute run (72 hours), 16.7 million at one hour (71 hours), 14.1 million at six (66 hours) and 12.5 million at twelve (60 hours, the last twelve of 26 March having no label). The zero-shot forecaster is evaluated as in Section 5.5.1, on 60,000 random samples of the deep test run of each horizon. Last, a second seed of each of the two one-minute stateful models, alone and as the residual of the trees, trained after this evaluation with the recipe of the validated runs unchanged and evaluated the same way, measures the seed noise of the one-minute results; nothing in them was chosen on the test days.

Coverage of the test days. The three days are not three ordinary days. The collector holds 14.1, 12.5 and 6.6 million order-book snapshots for 24, 25 and 26 March (and 6.8 million for 23 March, a warm-up day), against 6.6 to 19.7 million per day over the whole window and 12.0 million on the validation day. Every hour of 26 March is present, but each holds about 40% of the usual rows. Whether the gap is the collector's or the exchange's could not be established; the results are therefore given per day as well as pooled, and Table 5.21 shows that the sparse day changes no ordering.

Results. Table 5.19 gives the test numbers in the shape of Table 5.17; Table 5.20 every one-hour model of the study; Table 5.21 the main models by day; Table 5.22 the zero-shot forecaster.

Three things carry over from the validation day to the test days, and one does not.

The ordering of the families, and the whole one-hour chain. At every horizon the calibrated control is the best model or within its interval of the best, every learned model stands where it stood on 13 March, and the sparse day moves no sign. At one hour (Table 5.20) the twelve runs of the study keep the order of Sections 5.3 to 5.5 to the last step, and every gap is within a few tenths of its validation value: the two-pass residual model ties the control (-0.11 [-0.26,

Table 5.19: Test days 24–26 March: the best model of each family at each horizon, all samples and deduplicated samples (balanced accuracy), every model with its two logit offsets chosen once on the validation day and applied unchanged. "Stateless" is LightGBM G5 refitted as in validation and evaluated on the test samples of the stateful run; "stateful" is the best validated run without trees (two passes with the hand features at 1 min, 15 min and 1 h, the time-first encoder at 6 h, the reference model, seed 0, at 12 h); "residual" is the two-pass stateful model on the cross-fitted trees. Δ in points against the stateless control, 95% hour-block bootstrap intervals over the test hours. TabM, not shown, is +0.27 [+0.21, +0.34] at 1 min, +0.15 [+0.02, +0.27] at 15 min and −0.08 [−0.25, +0.09] at 1 h; a second seed of the one-minute stateful model is +0.39 [+0.24, +0.57] on all samples and +1.52 [+1.22, +1.83] on the deduplicated ones, and a second seed of the one-minute residual model +0.96 [+0.85, +1.08] and +1.82 [+1.61, +2.02].

Horizon	Subset	MR	Stateless	Stateful	Δ	Residual	Δ
1 min	All	0.5637	0.6496	0.6572	+0.76 [+0.60, +0.93]	**0.6586**	+0.90 [+0.78, +1.03]
	Dedup	0.5342	0.6336	0.6443	+1.07 [+0.80, +1.34]	**0.6539**	+2.03 [+1.79, +2.26]
15 min	All	0.5471	**0.6088**	0.6064	−0.25 [−0.44, −0.05]	—	
	Dedup	0.5301	**0.6167**	0.6090	−0.77 [−0.93, −0.60]	—	
1 h	All	0.5268	**0.5944**	0.5806	−1.38 [−1.69, −1.07]	0.5933	−0.11 [−0.26, +0.03]
	Dedup	0.5125	0.5887	0.5704	−1.83 [−1.96, −1.70]	**0.5895**	+0.08 [−0.02, +0.17]
6 h	All	0.5049	**0.5851**	0.5468	−3.83 [−4.11, −3.53]	—	
	Dedup	0.5025	**0.5745**	0.5323	−4.22 [−4.43, −4.00]	—	
12 h	All	0.4724	**0.5853**	0.5353	−5.00 [−5.34, −4.64]	—	
	Dedup	0.4766	**0.5749**	0.5258	−4.91 [−5.19, −4.60]	—	

+0.03] on all samples, +0.08 [−0.02, +0.17] on the deduplicated ones; +0.04 and −0.10 in validation), TabM ties it (−0.08 [−0.25, +0.09]; −0.11 in validation), the two-pass model without trees is −1.38 [−1.69, −1.07] (−1.42), the time-first encoder is 1.5 points above the GRU reference (1.5 calibrated against calibrated in validation), the forecast features add +0.05 and +0.3 on the two seeds (+0.3 and +0.6), the transaction graph is 0.4 below the reference (0.3), and the three seeds of the reference are within 0.26 points of one another. At fifteen minutes TabM is +0.15 [+0.02, +0.27] above the control (+0.22 in validation) and the two-pass stateful model −0.25 [−0.44, −0.05] below it (a tie, −0.07, in validation), −0.77 [−0.93, −0.60] on the deduplicated samples (−0.39); 26 March is the one day on which it is above the control (0.6186 against 0.6171). At one minute the gain of the sequential state is confirmed: the two-pass stateful model is +0.76 [+0.60, +0.93] above the calibrated control on all samples (+0.62 [+0.46, +0.82] in validation) and +1.07 [+0.80, +1.34] on the deduplicated ones (+1.74), with the same sign on each of the three days (+0.7, +0.7, +1.0 points). A second seed of the same model, trained after this evaluation and within a tenth of a point of the first on the validation day, is +0.39 [+0.24, +0.57] on all samples and +1.52 [+1.22, +1.83] on the deduplicated ones (+0.4, +0.3 and +0.8 by day): the sign and the exclusion of zero hold on every subset and every day for both seeds, and the size moves by four tenths of a point from one seed to the other, in opposite directions on the two subsets, more than either interval, which is the measure of the seed noise at this horizon. The residual model, the same stateful model on the cross-fitted trees, is +0.90 [+0.78, +1.03] above the control on all samples (+1.37 [+1.16, +1.58] in validation) and +2.03 [+1.79, +2.26] on the deduplicated ones (+2.04), +0.9, +0.8 and +1.1 points by day, and its margin over the stateful model alone, 0.8 points in validation, is 0.1 on all test samples and 1.0 on the deduplicated ones. A second seed of the residual model, within a tenth of a point of the first on the validation day (0.6764 against 0.6770), is +0.96 [+0.85, +1.08] on all samples and +1.82 [+1.61, +2.02] on the deduplicated ones (+1.0, +0.8 and +1.2 by day): the seed moves

Table 5.20: Every one-hour model of the study on the test days (balanced accuracy, all samples and deduplicated samples, each model with its validation-day offsets; the validation column is the validation day with the same offsets). Δ in points against the calibrated control, 95% hour-block bootstrap intervals.

Model	Validation	Test	Δ [95% CI]	Dedup.	Δ
LightGBM G5 (control)	0.6044	**0.5944**	—	0.5887	—
TabM, 3 seeds	0.6036	0.5936	−0.08 [−0.25, +0.09]	0.5865	−0.22
MLP, 3 seeds	0.6002	0.5923	−0.22 [−0.40, −0.03]	0.5834	−0.54
Stateful, 2 passes, residual of the trees	0.6048	0.5933	−0.11 [−0.26, +0.03]	**0.5895**	+0.08
Stateful, 2 passes	0.5907	0.5806	−1.38 [−1.69, −1.07]	0.5704	−1.83
Stateful, 1 pass, weight averaging	0.5879	0.5782	−1.62 [−1.97, −1.29]	0.5715	−1.72
Stateful, 1 pass	0.5842	0.5738	−2.07 [−2.40, −1.73]	0.5667	−2.20
Time-first, no hand features, seed 0	0.5718	0.5634	−3.11 [−3.48, −2.73]	0.5502	−3.85
Time-first, no hand features, seed 1	0.5689	0.5604	−3.40 [−3.78, −3.03]	0.5483	−4.05
GRU reference + Chronos-2 features, seed 0	0.5598	0.5492	−4.52 [−4.92, −4.15]	0.5357	−5.30
GRU reference + Chronos-2 features, seed 1	0.5623	0.5511	−4.33 [−4.71, −3.95]	0.5350	−5.38
GRU reference, seed 0	0.5569	0.5487	−4.57 [−5.00, −4.18]	0.5326	−5.61
GRU reference, seed 1	0.5557	0.5480	−4.65 [−5.04, −4.26]	0.5314	−5.73
GRU reference, seed 2	0.5561	0.5461	−4.83 [−5.23, −4.42]	0.5253	−6.34
GRU reference with the transaction graph	0.5543	0.5447	−4.97 [−5.39, −4.56]	0.5268	−6.19
Mean Reversion	0.5311	0.5268	−6.76 [−7.23, −6.31]	0.5125	−7.62

the residual model by a tenth of a point on all samples and by two tenths on the deduplicated ones, against four tenths for the stateful model alone, and its margin over the stateful model of the same seed is 0.1 and 0.6 on all samples and 1.0 and 0.3 on the deduplicated ones. TabM, the best stateless model at one minute, is +0.27 [+0.21, +0.34] above the control (+0.33 in validation) and +0.58 [+0.43, +0.73] on the deduplicated samples (−0.10), between the control and the stateful models on every day: at one minute the stateful models beat the strongest learner on the summary features, not only the trees. At six hours the time-first model is −3.83 [−4.11, −3.53] (−3.0 in validation) and at twelve the reference model is −5.00 [−5.34, −4.64] (−3.3), its three seeds −5.00, −5.41 and −5.63, within six tenths of a point and in the same order on every day: the long-horizon gaps widen by 0.8 and 1.7 points on the test days, in the control's favour. The graph models stand where Section 5.4 put them, 0.4 below the LOB-only reference at one hour, 1.4 below the time-first model at six hours and between the seeds of the reference at twelve (−5.23 [−5.62, −4.79] against the control).

The control. Its balanced accuracy is within 1.4 points of the validation value at every horizon: 0.6496 at one minute, 0.6088 at fifteen minutes and 0.5944 at one hour (0.6627, 0.6214 and 0.6044 on 13 March), 0.5851 at six hours and 0.5853 at twelve (0.5720 and 0.5717); at the long horizons it is higher on the test days than on the validation day.

The rule. Mean Reversion is 0.5637, 0.5471, 0.5268, 0.5049 and 0.4724 at the five horizons (0.5363, 0.5373, 0.5311, 0.5069 and 0.5071 in validation): the one-minute reversal is almost three points stronger on the test days, the twelve-hour one falls below 0.5, and the control's margin over the rule runs from +6.2 points at fifteen minutes to +11.3 at twelve hours. The zero-shot forecaster (Table 5.22) behaves as on the validation day: the sign of its median is 7 to 15 points below the rule at every horizon, because it almost never predicts CONST; with the half-tick dead zone it is 1.4 to 2.8 points below the rule from one minute to one hour, ties it at six hours and is 3.7 points above it at twelve, where the rule itself fell and the forecaster kept its validation value (0.504 there, 0.508 here); it agrees with the rule's directional calls 60 to 67% of the time and, where the two disagree and the price moves, it is right 48 to 53% of the time. It re-discovers the reversal, reads it a little more robustly than the rule on days when the twelve-hour reversal weakens, and stays far below every learned model.

What does not carry over is the *calibration*. The offsets chosen on 13 March, worth +0.5

Table 5.21: The main models by test day (balanced accuracy on all samples, calibrated as in Table 5.19).

Horizon	Model	24 March	25 March	26 March
1 min	Mean Reversion	0.5596	0.5675	0.5647
	Stateless	0.6449	0.6479	0.6643
	TabM	0.6477	0.6503	0.6678
	Stateful, seed 0	0.6522	0.6547	0.6743
	Stateful, seed 1	0.6485	0.6505	0.6720
	Residual, seed 0	0.6536	0.6562	0.6754
	Residual, seed 1	0.6545	0.6563	0.6765
15 min	Mean Reversion	0.5469	0.5483	0.5437
	Stateless	0.6048	0.6083	0.6171
	TabM	0.6065	0.6092	0.6195
	Stateful	0.6036	0.6036	0.6186
1 h	Mean Reversion	0.5297	0.5237	0.5251
	Stateless	0.5971	0.5895	0.5945
	TabM	0.5958	0.5900	0.5917
	Stateful	0.5797	0.5808	0.5797
	Residual	0.5948	0.5891	0.5945
6 h	Mean Reversion	0.5088	0.4989	0.5108
	Stateless	0.5923	0.5785	0.5754
	Stateful	0.5558	0.5403	0.5387
12 h	Mean Reversion	0.4730	0.4699	0.4842
	Stateless	0.5990	0.5691	0.5863
	Stateful, seed 0	0.5506	0.5199	0.5301
	Stateful, seed 1	0.5449	0.5152	0.5390
	Stateful, seed 2	0.5415	0.5140	0.5402
	Graph	0.5481	0.5157	0.5417

and +0.3 points to the control there, cost it 0.4 and 0.3 on the test days at fifteen minutes and one hour (0.6131 → 0.6088, 0.5972 → 0.5944), are worth +0.2 and +0.4 at six and twelve hours, and at one minute, where they were worth +5.4 in validation, they are worth +2.0 (0.6300 → 0.6496), and +1.8 to the tabular network (+5.5 in validation). For the stateful models the same frozen offsets help at every horizon, and most where the model is worst calibrated: +4.8 points at twelve hours (0.4874 → 0.5353, offsets of −0.8 on both classes; −0.9 to −1.2 for the other seeds), +1.6 at six hours for the reference model and +0.1 for the time-first one, +2.5 at one minute (0.6319 → 0.6572; +5.8 in validation) and +2.1 for the one-minute residual model (0.6380 → 0.6586; +5.9; +2.4 for its second seed), +0.2 at fifteen minutes and one hour. A decision threshold fitted on one day is thus worth a few tenths of a point on another day, in either direction, for a learner whose class mix is already right, and several points for a model whose class mix is wrong; the comparisons of Tables 5.19 and 5.20 are calibrated against calibrated and do not depend on it, but any single raw number does, and the one-minute numbers most of all.

The test days therefore confirm the three regimes of Section 5.6.4 on data that no choice of the thesis had seen: at one minute the sequence beats the summary, at fifteen minutes and one hour the stateless learner, the tabular network and the stateful model on the trees tie, and at six and twelve hours the stateless learner is alone.

Table 5.22: Zero-shot Chronos-2 on 60,000 random test samples per horizon, as in Table 5.14 (raw decisions, no offsets): balanced accuracy of Mean Reversion-H, of the sign of the forecast median and of the median with a half-tick dead zone, the difference of the latter from the rule with its 95% hour-block interval, the share of the rule's directional calls on which the forecaster agrees with it, and the share of the disagreements, on samples whose price then moved, in which the forecaster was right.

Horizon	MR	Median	Dead zone	Δ dead zone − MR	agrees with MR	right when they disagree
1 min	0.5646	0.4145	0.5364	−2.82 [−3.70, −1.92]	60%	48%
15 min	0.5485	0.4280	0.5249	−2.36 [−3.08, −1.64]	62%	52%
1 h	0.5267	0.4205	0.5133	−1.35 [−2.01, −0.73]	66%	51%
6 h	0.5072	0.3957	0.5028	−0.44 [−1.05, +0.11]	63%	51%
12 h	0.4710	0.3969	0.5077	+3.67 [+2.75, +4.46]	67%	53%

5.8 Discussion: the four questions

The results of this chapter, read together, answer the four questions of Section 1.3 as follows.

Q1, the process. Confirmed as stated in Section 3.7.4: the price is a bounded, discrete process whose most regular feature is its own activity clock, conditioned by the price level and the contract type; its trading is one connected graph organised by category, contract type and date, with a dense core and a long tail of one-fill wallets; its features are seventeen quantities organised by market and activity.

Q2, the information. The market's own history, activity and first level carry nearly all of it; the long returns read with the price level and the lifecycle carry the long-horizon direction (Section 5.2.2). The wallets, the siblings, the news flow and the zero-shot forecasts add nothing reliable at one hour (Sections 5.4 and 5.5); the forecasts add a few tenths in the central price range on the full stream only.

Q3, the models. Calibrated trees, a tabular network and the stateful model given the same inputs and the trees as a base tie at one hour and fifteen minutes; the online, time-ordered regime of the stateful model costs one to two points on its own; the graph never adds a reliable gain; at one minute the sequential state is worth +0.6 alone and +1.4 as the residual of the trees, with intervals that exclude zero on every subset (Section 5.6.4).

Q4, the horizon. It changes the task, from activity at one minute to a slow reversal at twelve hours, and the right family with it; the live-game regime and the crypto interval contracts are the hard cases at every horizon, because their price is driven by a reference the data do not contain (Section 5.6.5).

Chapter 6

Conclusions

6.1 What the thesis found, question by question

This thesis treated a decentralised prediction market as a data-mining object and asked four questions (Section 1.3): what its price process looks like and how its trading is organised, which public information predicts its direction, which families of models can read it, and how the answers change with the horizon. The answers, on three weeks of complete order-book and on-chain data, are these.

The process (Q1). The price is a bounded, discrete process whose most regular feature is its own activity clock: whether it will move is written in how recently and how often it has moved, and the direction, when it moves, is weakly in the first level of the book and in the reversal of its recent change. The price level and the kind of contract condition everything; the neighbours of a market, the wallets that trade it and the contracts of its event, carry nothing that its own book does not already show. Read as a graph, the trading is one connected component of 646 thousand wallets and 158 thousand markets with a dense core of a few thousand nodes, a long tail of wallets with a single fill, and a map organised by category, contract type and date. The sixty-one features that summarise these properties are about seventeen independent quantities; the space they span is organised by market and by activity, not by the label; the part of the label that is linear or local is *whether* the price moves, and the information the label asks for moves from the activity clock at minutes to the shape of the book at hours.

The information and the models (Q2, Q3). A stateless learner on sixty-one hand-made features that encode these properties is the strongest model at every horizon from fifteen minutes to twelve hours, six to seven points above the best rule; a properly trained tabular network ties it; a continuous-time stateful model reaches it only when given the same inputs and the trees as a base, and beats it, by a small margin with an interval that excludes zero, only at one minute, where the sequence of book events says something about the next minute of a quiet market that no summary of the last hour contains.

The horizon (Q4). At one minute the task is activity and the sequence of book events carries information that the summary loses; at fifteen minutes and one hour the summary is complete and a stateless learner is the right model; at six and twelve hours the task is the direction of a slow reversal read with the price level and the lifecycle of the market, and the stateless learner is six points above the rule while the stateful models of this study are below it. The live-game regime and the crypto interval contracts are the hard cases at every horizon.

In more detail. The thesis built, from raw order-book and on-chain data, a leakage-free benchmark for short-horizon price-direction prediction on Polymarket, and used it to compare a family of models against the strongest simple rule on the data. A continuous-time model that contains the mean-reversion rule as a special case, encodes the order book with a recurrent network, keeps a three-rate hierarchical memory and reads five lookback price changes beats the rule by 2.4 balanced-accuracy points at one hour and 1.5 at six hours, with a gain that is stable across seeds at one hour; reading the order-book window one quantity at a time before mixing quantities (the time-first encoder) raises the one-hour gain to 3.3 points, reproduced on a second seed, and the six-hour gain to 3.3 points; at twelve hours the model is below the rule on the full stream and above it on independent samples. The wallet–market transaction graph never adds a reliable gain; zero-shot time-series foundation models re-discover the rule and do not beat it, and as inputs of the best model they add between three and six tenths of a point at one hour over two seeds, a gain confined to central prices. The last experiment turns the picture around: a gradient-boosted model with no state, on sixty-one hand-made features from the clean data, beats the best deep model by 3.7 points at one hour, 3.2 at six hours and 8.0 at twelve, and is 7.3 points above the rule at one minute; the features that carry the gain (the time since the last price change, the number of recent changes, the first level of the book, the long returns read together with the price level) are exactly the ones the event-clock tensor hides or the linear direct path cannot use. Given those same features as inputs, the deep model recovers half of the gap at one hour (0.584); the other half is the cost of its online, time-ordered training regime, and with the cross-fitted trees as a base and the same calibration as the trees it ties them. At one minute, with the control rebuilt on the runs' own samples and every model calibrated the same way, the stateful model is +0.6 above the trees alone and +1.4 as their residual; at fifteen minutes it ties them. The locked test days of 24–26 March reproduce every one of these orderings, with the one-minute margins at +0.4 to +0.8 alone and +0.9 to +1.0 as the residual, over two seeds each, and the long-horizon gaps one to two points wider (Section 5.7).

6.2 Limitations

- One validation day and one training week. The three locked test days of 24–26 March (Section 5.7) reproduce every ordering of the validation day, but they are three days of the same month, and the long-horizon gaps widened by one to two points on them.
- Balanced accuracy is a statistical, not an economic, measure: a one-tick move predicted correctly is not a profitable trade once spread and fees are paid. The thesis makes no claim about trading.
- The graph was tested with one family of message-passing operators and a shared decay; a wallet memory at a slower time scale remains untested.
- Hyper-parameters were chosen by a small number of screening runs, not by a systematic search; the comparison between components is paired and fair, the absolute level could be higher.
- Three weeks of one venue in one month: the regularities found are those of Polymarket in March 2026, and a different season of events, or a different exchange, could weigh the families differently.

6.3 Towards deployment

The thesis stops before deployment. Balanced accuracy is a statistical measure; a trading use of these models would have to answer at least four further questions, none of which the present data settle. *Costs*: most correctly predicted moves are one tick, and a tick is

smaller than the median spread of three of the four families of markets of Table 3.9, so a correct direction is not a profitable trade unless the position is taken as a resting order, which changes the problem. *Latency*: a prediction at block t uses the book up to t and the fills of t; the order that acts on it reaches the operator later, and at one minute the signal is mostly "the price is about to move", which a resting order cannot use. *Capacity*: the sparse markets, where the sequential state is worth most, are sparse because little money trades them. *Non-stationarity*: the families were selected on one validation day; a deployed model would be retrained on a rolling window, and the stability of the rankings across days is measured here only by the locked test period. What the thesis does settle is what a model of this market should be made of, and that is the content of the next section.

6.4 What the thesis would start from today

Taken together, the control suggests the design the thesis would start from today: a regular time grid per market (one-minute bars, aggregated to fifteen minutes for the long horizons) instead of the event clock, a feature set reduced to the first level, the microprice, the recent-mean distance, the activity clock and the contract type, the time-first encoder on those bars, and gradient boosting as the control that any deep model must beat.

6.5 Future work

- A rebuilt input pipeline on a regular time grid (one-minute bars, fifteen-minute bars for the long horizons) with the features the no-state control found decisive, the time-first encoder on those bars, a learned contract-type embedding and one multi-horizon model [96]; gradient boosting as the control that every deep model has to beat; and the residual construction at fifteen minutes and at the long horizons, where it was not run.
- A learned, sparse market–market graph restricted to markets of the same event, in the spirit of Cini, Zambon, and Alippi [76], in place of the wallet graph.
- Smoothed labels at long horizons [28], to make the CONST class less sensitive to one-tick noise.
- The Binance price as a covariate for the crypto markets, following Young [14].
- Foundation-model features (Section 3.6.2) at the horizons where the model is already strong, and a fine-tuned or domain-pretrained variant if the frozen one helps.
- The time-first encoder with the elapsed time between snapshots as an input channel, and the time-first encoder together with the foundation-model features, since the two gains are measured on the same base model and may add up.
- A different task on the same data: the generation of the next edges of the transaction graph (which wallet will trade which market), the estimation of the real probability of the event against the price, or the distribution of the price at the horizon rather than its sign; and a graph model with a wallet memory at the time scale of days, which Section 3.7.2 suggests is the only scale at which most wallets have a history.
- An ensemble of the seeds already trained, the cheapest known improvement of a deep classifier [97], and event-based labels such as the triple-barrier method [98] in place of the fixed-horizon label.

Appendix A

The Sixty-One Hand-Made Features

The features of the stateless control (Section 3.6.2) are computed for every tensor row from the clean data strictly before the row's time, with DuckDB window functions over the market's own history (`hand_features.py`). Sizes are in contract units, times in seconds, prices in ticks of the market where noted; `log` is $\ln(1+x)$. The category is a one-hot indicator in the networks and a categorical column in the trees.

Group	Name	Definition
G1	`mid`	mid price of the last book state
	`ret_15m`, `ret_1h`, `ret_3h`, `ret_6h`, `ret_12h`	change of the mid over the past 15 min, 1, 3, 6 and 12 h (the lookbacks of the direct path)
	`sgn_15m` … `sgn_12h`	sign of each change
	`hour`	hour of the day (UTC)
G2	`spread_ticks`	best ask minus best bid, in ticks
	`imb1`, `imb5`, `imb10`	depth imbalance $(b-a)/(b+a)$ over the first 1, 5 and 10 levels
	`micro_minus_mid_ticks`	microprice (size-weighted best quotes) minus the mid, in ticks
	`log_bid_sz0`, `log_ask_sz0`	log size at the best bid and ask
	`log_bid_sz10`, `log_ask_sz10`	log size over the first ten levels of each side
	`bid_levels`, `ask_levels`	number of filled levels among the ten stored
	`depth_skew`	skew of the depth between the two sides (tensor feature)
	`bid_rel1`, `ask_rel1`	distance of the best bid and ask from the mid, relative (tensor features)
G3	`ofi`, `ofi_5`, `ofi_20`, `ofi_15m`	order-flow imbalance accumulated over the last event, the last 5 and 20 events and the last 15 minutes
	`ofi_20_z`, `ofi_15m_z`	the same, divided by the volume of book changes over the same span
	`log_vol_15m`	log volume of book changes in the last 15 minutes
	`buy_ratio`, `cancel_ratio`	share of bid-side events and of size removals among recent events (tensor features)
	`log_dt_prev`	log time since the previous book event
	`since_row`	time since the market's last tensor row (control only; the stateful model reads the current row)
	`n_1m`, `n_5m`, `n_15m`, `n_60m`	number of book events in the last 1, 5, 15 and 60 minutes
	`moves_60m`	number of mid changes in the last hour
	`rv_60m`	realised volatility of the mid over the last hour
	`range_pos_60m`	position of the mid in its one-hour range, centred at zero
	`mid_minus_mean60`	mid minus its one-hour mean
	`since_move`	time since the last mid change (the strongest feature at every horizon)

Group	Name	Definition
	`last_move_sign`	sign of the last mid change
G4	`flow_5m`, `flow_15m`, `flow_60m`	signed taker flow of the on-chain fills over 5, 15 and 60 minutes (oriented to the book's token)
	`usd_15m`, `usd_60m`	log collateral traded over 15 and 60 minutes
	`ntr_15m`	number of fills in the last 15 minutes
	`flow_ratio_15m`	signed flow divided by collateral traded, 15 minutes
	`last_trade_minus_mid_ticks`	price of the last fill minus the mid, in ticks (within the last hour)
	`log_since_trade`	log time since the last fill
G5	`category`	market category from the tags (politics, sports, crypto, weather, esports, culture, economy and tech, other)
	`is_yesno`	whether the outcomes are Yes/No (token orientation)
	`log_hours_to_end`	log hours to the market's end date
	`log_duration_h`	log duration of the market from deployment to end date
	`log_hours_since_start`	log age of the market

Appendix B

Run Labels and Configurations

Every stateful run is identified by a label that encodes its configuration, and every number in the thesis points to one. The grammar of the labels of the final study (`run_hand_runs_v4.sh`) is

`<H>_lb_hm_sg_lb5_<enc><K>_hand[_xp][_tree[_tro]][_ep<N>][_ema<d>]_ablation_pred_s<seed>`

with the components of Table B.1. Earlier runs use subsets of the same components (`1h_lb_pred_s0` is the lookback model with the graph, `1h_lb_ablation_pred_s0` its LOB-only counterpart, `1h_lb_hm_sg_lb5_ablation_pred_s0` the best LOB-only model before the time-first encoder). Validation predictions are written to `val_predictions_<label>.parquet`, one row per sample with the time, the market, the true and predicted class, the probabilities and the deduplication flag, so that any two runs can be compared on exactly the same samples.

Table B.1: Components of the run labels.

Component	Meaning
`<H>`	horizon: `60s`, `900s`, `1h`, `6h`, `12h`
`lb`	lookback price features appended to the window rows
`hm`	hierarchical memory (minute and hour states, periods 60 and 3600 s)
`sg`	zero-initialised direct path from the lookbacks to the logits, plus the global state
`lb5`	five lookbacks (0.25, 1, 3, 6, 12 h) instead of three (1, 6, 12 h)
`tf64` / `gru10` / `gru32`	window encoder and window length: time-first over 64 snapshots, or the GRU over 10 or 32
`hand`	the 61 hand-made features of Appendix A in the window rows and the direct path
`xp`	the hand features also read by the predictor (`--extra-to-predictor`)
`tree`	the cross-fitted LightGBM log-probabilities as inputs (`--tree-dir`)
`tro`	the log-probabilities added to the output logits (`--tree-offset`): the residual model
`ep<N>`	N passes over the training days (states reset, optimiser kept)
`ema<d>`	exponential moving average of the weights with decay d, swapped in for validation
`ablation`	no transaction edges (LOB-only); its absence means the graph variant
`pred`	validation predictions saved
`s<seed>`	random seed

Training configuration of the stateful model

State dimension 128; per-block, minute and hour states per market, one per-block state per wallet; learned shared decay rate (converges to about $0.04\,\mathrm{s}^{-1}$); window of K book states, 52 tensor features plus 3 within-window momentum columns plus the lookback and hand columns; time-first encoder with a shared $\mathbb{R}^K \to \mathbb{R}^{32}$ time map and one self-attention layer over the channels; predictor MLP of two layers of 64 units with dropout; focal loss $\gamma = 2$ with class weights 2/1/2; AdamW, learning rate 10^{-4}; truncated backpropagation through five consecutive blocks; one or two passes over 6–12 March in time order; validation on 13 March. Runs take 9 to 17 hours on one GPU of the shared server depending on the horizon and the load.

Configuration of the stateless control

LightGBM multiclass with class-balanced weights, learning rate 0.05, 63 leaves, at least 500 rows per leaf, feature and row subsampling 0.8, L2 regularisation 1, at most 600 rounds with early stopping (patience 40) on 12 March, seed 0; fit rows: one tensor row in ten of 7–11 March whose market belongs to the model's universe and whose last event is at or after $t + H$; validation samples: those of the stateful run at the same horizon. The tabular networks (MLP and TabM-mini, $k = 16$) use three layers of 512 units, a piecewise-linear encoding with 16 quantile bins, dropout 0.15, batches of 8,192 rows, a one-cycle schedule at peak rate 10^{-3} over 16 epochs with patience 4, class-balanced cross-entropy, seeds 0–2.

Appendix C

Reproduction

Data and code availability. The raw sources are public (Section 3.2): the order-book archive on Kaggle, the Polygon blockchain and the Polymarket metadata API. Rebuilding every derived dataset from them is possible but long, so the datasets that the models read are deposited as they are: the market table (175,292 rows), the hourly transaction partitions with the market and the token orientation of every fill (11 GB), the order-book tensors (27 GB), the hand-made features (7.7 GB), the cross-fitted tree log-probabilities (6.6 GB), the foundation-model forecasts (0.7 GB), the news and sibling features and the identifier mappings, about 55 GB in all, together with the trained checkpoints and the reports from which every table is taken [TODO: URL of the data repository once chosen]. With them, every model of the thesis can be retrained and evaluated on the same inputs without touching the raw streams. The code is a Python package (`polymarket_gnn`) managed with `uv`, with the data pipeline in DuckDB and NumPy and the models in PyTorch [TODO: URL of the code repository]; the raw data under `data/raw/` are never modified and every derived dataset lives under `data/derived/`.

Steps. The steps that produce the numbers of the thesis are, in order:

1. **Collection.** `scripts/preprocessing/00_legacy_raw_downloads/`: the Gamma API pager for the markets, the markets-by-token completion, and the Polygon block reader for the fills, a modified `poly-trade-scan` [89] (Section 3.2).
2. **Market tables.** `sql/markets/00–09` run by `scripts/preprocessing/00_build_markets_tables.py`: the Gamma rows flattened into one table per market, every fill joined to its market through the two outcome tokens, the union of the market identifiers seen in the five sources and the completion of the missing ones by condition identifier, with the three API scrapers that run between the steps and the expected row counts in `sql/markets/README.md`.
3. **Preprocessing.** `scripts/preprocessing/01–06`: multi-day aggregates, hourly order-book partitions with the 52-feature vectors, deltas, text embeddings, node features, wallet features; `precompute_tensors.py` for the per-block tensors read by the stateful model.
4. **Causal baselines.** `causal_baselines.py --horizon H` writes `baseline_results_<H>.json` with every rule on all and deduplicated samples.
5. **Stateful runs.** `run_hand_runs_v4.sh <device> <H>` with the environment variables of Appendix B (`EPOCHS`, `TREE_DIR`, `TREE_OFFSET`, `ENC`, `W`, `SEED`); each run writes its validation predictions.
6. **Hand features and the control.** `hand_features.py` builds the 61 features for every tensor row; `scripts/data_audit/feature_audit.py H <label> <label> 10 <out-dir>` builds the control on the samples of the run `<label>`, group by group, with

the univariate diagnostics and the importances; `control_proba.py --horizon` refits G5 for the class probabilities.

7. **Data-mining study.** `scripts/data_audit/datamining.py --horizon H` reads the feature parquets of the previous step and writes, per horizon, the PCA, correlation, random-projection, MDS, UMAP and t-SNE outputs, the LDA, logistic, kNN, tree and mutual-information results and the market clustering of Section 3.7.3, with a `report.md`; CPU only, about twenty-five minutes per horizon.
8. **Transaction graph.** `scripts/data_audit/graph_animation.py --stage E,S` builds the edge events and the statistics of Section 3.7.2 (degree distributions, components, k-cores, hourly and daily series); `graph_layout_anchor.py` (with `--cat-weight0.5` for the second map) computes the SVD and UMAP layout, `graph_islands.py` the HDBSCAN islands, `--stageF,V` the frames and the video, and `graph_summary_plots.py` the summary figure; CPU only.
9. **Cross-fitted trees and the residual model.** `tree_features.py` writes the day-cross-fitted log-probabilities; the residual run is step 4 with `TREE_DIR` and `TREE_OFFSET=1`.
10. **Tabular networks.** `scripts/data_audit/tabular_net.py --horizon --variants tabm --seeds 0,1,2`.
11. **Comparisons.** `hand_vs_lgb_report.py <label> --horizon --prior --crossfit` (raw, prior-corrected and one-sided offsets, by subset and category); `scripts/data_audit/offset_fair.py --horizon --deep <labels> --tab <probas> --grid-max 2.0 --grid-step 0.2` (the same calibration for every model); `analyze_predictions.py` (per-market breakdown); `scripts/data_audit/leadlag.py` (sibling lead–lag); `news_features.py`, `sibling_features.py` and `scripts/data_audit/news_audit.py` (external features in the control).
12. **Reports.** Every table of the thesis is in `reports/experiments/<study>/`, with the run labels in the file names; the development log (`DEVELOPMENT_LOG.md`) records every run with its date, device and duration.

Unit tests cover the label construction, the mid-price index, the hand-feature index and the news index (`tests/`). The thesis is built with `latexmk` from the `thesis/` directory; the bibliography is tagged by the kind of verification of each entry (`[L]` a PDF in `literature/`, `[V]` checked online, `[A]` abstract only).

Appendix D

Supplementary Tables

Table D.1: The stateless control by market category at 15 minutes, on the validation samples of the 15-minute run (balanced accuracy inside the category; p_{move} is the share of samples whose mid moved within the horizon, p_{up} the share of UP among the moves). The stateful model is the two-pass run without trees.

Category	samples	markets	p_{move}	p_{up}	MR	LightGBM	stateful
esports	162,228	1,266	0.53	0.48	0.552	**0.589**	0.585
sports	3,117,846	6,844	0.33	0.48	0.551	**0.621**	0.609
crypto	292,527	1,160	0.30	0.47	0.553	**0.623**	0.620
weather	200,158	696	0.49	0.45	0.480	0.556	**0.558**
politics	3,764,416	4,502	0.23	0.46	0.536	**0.622**	0.610
economy, tech	889,609	1,197	0.30	0.47	0.520	**0.600**	0.596
culture	1,405,618	726	0.44	0.49	0.501	**0.579**	0.578
other	76,127	119	0.27	0.42	0.518	**0.586**	0.580

Table D.2: Univariate diagnostics of the strongest features at 1 h, on the control's validation samples: directional hit rate on the samples that moved (best of the two orientations), and area under the ROC curve of the feature's magnitude for separating CONST from a move.

Feature	hit rate on moves	AUC move vs CONST
mid minus its one-hour mean	0.573	—
level-1 imbalance	0.557	—
microprice minus mid	0.555	—
on-chain taker flow (15 min)	0.526	—
ten-level imbalance	0.508	—
tensor order-flow imbalance	0.500–0.505	—
time since the last mid change	—	0.85
At 1 min (on the samples of the 1-minute run)		
level-1 imbalance	0.541	—
mid changes in the last hour	—	0.94
time since the last mid change	—	0.91
events in the last minute	—	0.89

Glossary and Notation

Terms

activity clock the quantities that describe how recently and how often a market's mid has changed: the time since the last mid change, the number of mid changes and of book events in the last minutes and hour. The strongest predictors of whether the mid will move (Section 3.7.1).

balanced accuracy the mean of the per-class recalls of a classifier; 1/3 for a constant three-class rule (Section 5.1).

block the unit of time of the Polygon blockchain, about two seconds; the time step of the stateful model (Section 3.5.1).

calibration (offsets) two additive constants on the DOWN and UP logits of a model, chosen on the even validation hours and applied to the odd ones and vice versa, so that every model's decision rule is placed the same way before models are compared (Section 5.1).

central / extreme samples samples whose mid is inside $[0.05, 0.95]$ and outside it (Section 3.7.1).

CLOB central limit order book, the matching mechanism of Polymarket: off-chain order matching, on-chain settlement (Section 2.2.1).

conditional token an outcome token of a market; a pair of complementary tokens is minted from one unit of collateral and the winning one is redeemed for it after resolution (Section 2.2.1).

control (no-state) the LightGBM model on the sixty-one hand-made features, the stateless reference of every comparison (Section 5.2.2).

cross-fitting fitting a quantity (offsets, tree log-probabilities, a combination weight) on one part of the data and applying it to another, so that no sample is evaluated with a quantity fitted on itself.

deduplicated samples at most one sample per market per fifteen minutes; the subset on which samples are closest to independent (Section 3.6.1).

direct path a zero-initialised linear layer from the lookback price features (and the hand features) to the three logits, which contains Mean Reversion as a special case (Section 4.4.1).

fill a matched trade, recorded on-chain with maker, taker, token, side, quantity and price; the rows of the transactions table.

hierarchical memory the minute and hour states of a market, each updated by its own GRU cell at its own period (Section 4.4.1).

rank-gauss transform the monotone map of a feature to its rank and then through the inverse

normal distribution, fitted on training rows; it removes heavy tails and scale before distance-based methods (Section 3.7.3).

hour-block bootstrap resampling of the 24 validation hours with replacement to obtain confidence intervals for differences between models (Section 5.1).

lead–lag whether the changes of one contract precede those of another; measured at the minute scale between sibling contracts (Section 3.7.1).

lookback the change of the mid over a fixed past span (15 min, 1, 3, 6, 12 h).

Mean Reversion-H the rule that predicts the opposite of the sign of the change over the past H, and CONST if the price did not move (Section 4.2).

mid price the average of the best bid and the best ask of the Yes-side book.

residual model the stateful model that receives the cross-fitted tree log-probabilities as inputs and adds them to its logits (Section 4.4.2).

resolution the decision of a market's outcome through UMA's optimistic oracle after the event (Section 2.2.4).

sibling contracts markets of the same event: the two sides of a game, the candidates of an election, the strikes of a ladder (Section 3.6.2).

stateful / stateless model a model that keeps a state per market updated from the stream, against one that reads a table of features at the sample time (Sections 5.2 and 5.3).

TabM a parameter-efficient ensemble of multilayer perceptrons sharing one backbone, the tabular network used in Section 5.2.3.

tensor row one 52-feature description of a market's book at one book event, the unit of the derived order-book tensors (Section 3.5.2).

time-first encoder the window encoder that reads each book quantity as its own series before mixing quantities (Section 4.4.1).

token orientation whether the book (and the label) refers to token 1 or token 2 of the market; the sign of a trade is corrected accordingly (Section 3.4).

Acronyms

AMI adjusted mutual information between two partitions of the markets.

AUC area under the receiver operating characteristic curve.

BA balanced accuracy.

CI confidence interval (95%, hour-block bootstrap unless stated).

CLOB central limit order book.

CT-DGNN continuous-time dynamic graph neural network, the name of the stateful model in the code.

GDELT Global Database of Events, Language and Tone, the news source of Section 5.5.2.

GRU gated recurrent unit.

kNN k-nearest-neighbour classifier.

LDA linear discriminant analysis.

LOB limit order book.

MI mutual information.

MLP multilayer perceptron.

MR Mean Reversion-H.

OFI order-flow imbalance.

PCA principal component analysis.

PLE piecewise-linear encoding of numeric features.

TBPTT	truncated backpropagation through time.
TSFM	time-series foundation model.
t-SNE	t-distributed stochastic neighbour embedding.
UMA	the optimistic oracle protocol that resolves Polymarket markets.
UMAP	uniform manifold approximation and projection.
USDC	the dollar stablecoin used as collateral.

Notation

$\mathcal{M}$, $\mathcal{W}$	the sets of markets and wallets; m, w one of each (Section 2.4).
$(t_i, \mathbf{b}_i)$	the book stream of a market: the time of its i-th event and the 52 numbers that describe the book just after it.
H	the prediction horizon (60 s, 900 s, 1 h, 6 h, 12 h).
$\mathrm{mid}_m(t)$	the mid price of market m at the last book update strictly before t.
$y_{m,t}$	the label: UP, CONST or DOWN according to whether $\mathrm{mid}_m(t+H)$ is above, equal to or below $\mathrm{mid}_m(t)$ (Eq. (3.1)).
(m, t)	a sample: market m at block t, a block in which m had a book event.
Δ_L^-	the change of the mid over the past lookback L.
$\mathbf{x}_{m,t}$	the 61 hand-made features of sample (m, t).
$\mathbf{z}_{m,t}$, $\hat{\mathbf{p}}_{m,t}$, $\hat{y}_{m,t}$	the three logits, the three probabilities and the predicted class of a model at (m, t); $\boldsymbol{\delta}$ the calibration offsets added to the logits (Eq. (4.3)).
$\mathbf{h}_m$, $\mathbf{h}_w$	the per-block states of market m and wallet w, 128 numbers each; $\mathbf{h}_m^{\min}$, $\mathbf{h}_m^{\mathrm{hr}}$ the minute and hour states; $\bar{\mathbf{h}}$ the global state (Eqs. (4.1) and (4.2)).
r	the learned decay rate of the states; $e^{-r\Delta t}$ the decay over Δt seconds.
K, $\mathbf{W}_{m,t}$	the number of book states in the window read by the encoder, and the window itself; ϕ the window encoder.
$G = (V, E)$	a graph with nodes V and edges E; E_t the fills of block t, $N_t(v)$ those incident to node v, $\mathbf{a}_e$ the seven attributes of fill e (Eqs. (4.5) and (4.6)).
BA, recall_c	balanced accuracy and the recall of class c (Section 2.9).
Δ	a difference of balanced accuracies in percentage points, followed by its 95% interval in brackets.
G1 … G5	the nested groups of hand-made features (Appendix A).

Bibliography

[1] J. Wolfers and E. Zitzewitz. "Prediction Markets". In: *Journal of Economic Perspectives* 18.2 (2004), pp. 107–126.

[2] K. J. Arrow et al. "The Promise of Prediction Markets". In: *Science* 320.5878 (2008), pp. 877–878.

[3] J. Berg, R. Forsythe, F. Nelson, and T. Rietz. "Results from a Dozen Years of Election Futures Markets Research". In: *Handbook of Experimental Economics Results*. Ed. by C. R. Plott and V. L. Smith. Vol. 1. Elsevier, 2008, pp. 742–751.

[4] C. F. Manski. "Interpreting the predictions of prediction markets". In: *Economics Letters* 91.3 (2006), pp. 425–429.

[5] E. Snowberg, J. Wolfers, and E. Zitzewitz. *Prediction Markets for Economic Forecasting*. NBER Working Paper 18222. National Bureau of Economic Research, 2012.

[6] L. Page and R. T. Clemen. "Do Prediction Markets Produce Well-Calibrated Probability Forecasts?" In: *The Economic Journal* 123.568 (2013), pp. 491–513.

[7] M. Cardozo and J. I. Rivero-Wildemauwe. "The Favorite–Longshot Bias in Prediction Markets: Evidence from Polymarket". In: *arXiv preprint* (2026). arXiv: 2609.12878.

[8] P. Akey, V. Grégoire, N. Harvie, and C. Martineau. *Who Wins and Who Loses in Prediction Markets? Evidence from Polymarket*. Discussion Paper 21615. CEPR, 2026.

[9] J. Gebele, T. Mutzel, and F. Matthes. *Executable Arbitrage and Market Efficiency in Prediction Markets*. 2026. arXiv: 2608.00666 [cs.CE].

[10] Bocconi Students Investment Club. *Polymarket Microstructure Analysis Project: Behavioral Wallet Classification, Kyle's λ Estimation, Avellaneda–Stoikov Market Making, and Directional Strategy Backtesting on the 2024 US Presidential Election Markets*. Project report, Bocconi Students Investment Club. 2026.

[11] O. Saguillo, V. Ghafouri, L. Kiffer, and G. Suarez-Tangil. "Unravelling the Probabilistic Forest: Arbitrage in Prediction Markets". In: *arXiv preprint* (2025). arXiv: 2508.03474.

[12] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison. "Limit order books". In: *Quantitative Finance* 13.11 (2013), pp. 1709–1742.

[13] P. D. Dubach. "The Anatomy of a Decentralized Prediction Market: Microstructure Evidence from the Polymarket Order Book". In: *arXiv preprint* (2026). arXiv: 2604.24366 [q-fin.TR].

[14] G. Young. "OpenMarket: A Synchronized Polymarket–Binance Dataset for High-Frequency Prediction-Market Research". In: *arXiv preprint* (2026). arXiv: 2607.26245.

[15] Polymarket. *How Are Prediction Markets Resolved?* Polymarket Help Center. 2026. URL: https://help.polymarket.com/en/articles/13364518-how-are-prediction-markets-resolved (visited on 10/23/2026).

[16] P. Cheng, J. Liu, and Y. Long. "PolyBench: Benchmarking LLM Forecasting and Trading Capabilities on Live Prediction Market Data". In: *arXiv preprint* (2026). arXiv: 2604.14199.

[17] R. M. Barot and A. S. Borkhatariya. "PolySwarm: A Multi-Agent Large Language Model Framework for Prediction Market Trading and Latency Arbitrage". In: *arXiv preprint* (2026). arXiv: 2604.03888.

[18] H. Jia, L. Zhou, W. Zhang, et al. "Unlocking the Forecasting Economy: A Suite of Datasets for the Full Lifecycle of Prediction Markets". In: *arXiv preprint* (2026). arXiv: 2604.20421.

[19] F. Reichenbach and M. Walther. *Exploring Decentralized Prediction Markets: Accuracy, Skill, and Bias on Polymarket*. Working Paper 5910522. SSRN, 2025.

[20] K. P. Tsang and Z. Yang. "The Anatomy of Polymarket: Evidence from the 2024 Presidential Election". In: *arXiv preprint* (2026). arXiv: 2603.03136.

[21] R. Cont, A. Kukanov, and S. Stoikov. "The Price Impact of Order Book Events". In: *Journal of Financial Econometrics* 12.1 (2014), pp. 47–88.

[22] E. F. Fama. "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2 (1970), pp. 383–417.

[23] A. W. Lo and A. C. MacKinlay. "Stock Market Prices Do Not Follow Random Walks: Evidence from a Simple Specification Test". In: *The Review of Financial Studies* 1.1 (1988), pp. 41–66.

[24] R. Cont. "Empirical properties of asset returns: stylized facts and statistical issues". In: *Quantitative Finance* 1.2 (2001), pp. 223–236.

[25] Z. Zhang, S. Zohren, and S. Roberts. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books". In: *IEEE Transactions on Signal Processing* 67.11 (2019), pp. 3001–3012.

[26] A. Ntakaris, M. Magris, J. Kanniainen, M. Gabbouj, and A. Iosifidis. "Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods". In: *Journal of Forecasting* 37.8 (2018), pp. 852–866.

[27] A. Briola, S. Bartolucci, and T. Aste. "Deep Limit Order Book Forecasting: A Microstructural Guide". In: *arXiv preprint* (2024). arXiv: 2403.09267.

[28] L. Berti and G. Kasneci. "TLOB: A Novel Transformer Model with Dual Attention for Price Trend Prediction with Limit Order Book Data". In: *arXiv preprint* (2025). arXiv: 2502.15757.

[29] J. Sirignano and R. Cont. "Universal features of price formation in financial markets: perspectives from deep learning". In: *Quantitative Finance* 19.9 (2019), pp. 1449–1459. arXiv: 1803.06917.

[30] P. N. Kolm, J. Turiel, and N. Westray. "Deep order flow imbalance: extracting alpha at multiple horizons from the limit order book". In: *Mathematical Finance* 33.4 (2023), pp. 1044–1081.

[31] L. Lucchese, M. S. Pakkanen, and A. E. D. Veraart. "The short-term predictability of returns in order book markets: a deep learning perspective". In: *International Journal of Forecasting* 40.4 (2024), pp. 1587–1621. arXiv: 2211.13777.

[32] A. Zeng, M. Chen, L. Zhang, and Q. Xu. "Are transformers effective for time series forecasting?" In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 9. 2023. arXiv: 2205.13504.

[33] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. "A time series is worth 64 words: long-term forecasting with transformers". In: *arXiv preprint* (2022). arXiv: 2211.14730.

[34] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[35] D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. 2015.

[36] I. Loshchilov and F. Hutter. "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*. 2019.

[37] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. "Learning Representations by Back-Propagating Errors". In: *Nature* 323 (1986), pp. 533–536.

[38] A. Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019.

[39] S. Elfwing, E. Uchibe, and K. Doya. "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning". In: *Neural Networks* 107 (2018), pp. 3–11.

[40] D. Hendrycks and K. Gimpel. "Gaussian Error Linear Units (GELUs)". In: *arXiv preprint arXiv:1606.08415* (2016).

[41] J. L. Ba, J. R. Kiros, and G. E. Hinton. "Layer Normalization". In: *arXiv preprint* (2016). arXiv: 1607.06450.

[42] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. "Focal Loss for Dense Object Detection". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2980–2988.

[43] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar. "Long-Tail Learning via Logit Adjustment". In: *International Conference on Learning Representations (ICLR)*. 2021.

[44] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[45] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15 (2014), pp. 1929–1958.

[46] Y. Bengio, P. Simard, and P. Frasconi. "Learning Long-Term Dependencies with Gradient Descent is Difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[47] R. Pascanu, T. Mikolov, and Y. Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. 2013, pp. 1310–1318.

[48] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[49] S. Hochreiter and J. Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[50] R. J. Williams and J. Peng. "An Efficient Gradient-Based Algorithm for On-Line Training of Recurrent Network Trajectories". In: *Neural Computation* 2.4 (1990), pp. 490–501.

[51] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, pp. 5998–6008.

[52] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu. "Recurrent Neural Networks for Multivariate Time Series with Missing Values". In: *Scientific Reports* 8 (2018), p. 6085.

[53] J. Koutník, K. Greff, F. Gomez, and J. Schmidhuber. "A Clockwork RNN". In: *Proceedings of the 31st International Conference on Machine Learning (ICML)*. Vol. 32. PMLR. 2014, pp. 1863–1871.

[54] A. F. Ansari et al. "Chronos: Learning the Language of Time Series". In: *arXiv preprint* (2024). arXiv: 2403.07815.

[55] A. F. Ansari, O. Shchur, J. Küken, A. Auer, et al. "Chronos-2: From Univariate to Universal Forecasting". In: *arXiv preprint* (2025). arXiv: 2510.15821.

[56] E. Rahimikia, H. Ni, and W. Wang. "Re(Visiting) Time Series Foundation Models in Finance". In: *arXiv preprint* (2025). arXiv: 2511.18578.

[57] K. Dewage, S. De Silva, and S. Mondal. "Hybrid Neural-Classical Correction for Frozen Time Series Foundation Models: A Comprehensive Ablation Study on High-Frequency Stock Prediction". In: *arXiv preprint* (2026). arXiv: 2608.08825.

[58] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart. "Representation Learning for Dynamic Graphs: A Survey". In: *Journal of Machine Learning Research* 21.70 (2020), pp. 1–73.

[59] A. Gravina and D. Bacciu. "Deep learning for dynamic graphs: models and benchmarks". In: *IEEE Transactions on Neural Networks and Learning Systems* (2024).

[60] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. "The Graph Neural Network Model". In: *IEEE Transactions on Neural Networks* 20.1 (2009), pp. 61–80.

[61] A. Micheli. "Neural Network for Graphs: A Contextual Constructive Approach". In: *IEEE Transactions on Neural Networks* 20.3 (2009), pp. 498–511.

[62] T. N. Kipf and M. Welling. "Semi-Supervised Classification with Graph Convolutional Networks". In: *International Conference on Learning Representations (ICLR)*. 2017.

[63] D. Bacciu, F. Errica, A. Micheli, and M. Podda. "A gentle introduction to deep learning for graphs". In: *Neural Networks* 129 (2020), pp. 203–221.

[64] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. "Neural Message Passing for Quantum Chemistry". In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. PMLR. 2017, pp. 1263–1272.

[65] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. "Graph Attention Networks". In: *International Conference on Learning Representations (ICLR)*. 2018.

[66] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. "Gated Graph Sequence Neural Networks". In: *International Conference on Learning Representations (ICLR)*. 2016.

[67] A. Gravina, D. Bacciu, and C. Gallicchio. "Anti-Symmetric DGN: a stable architecture for Deep Graph Networks". In: *International Conference on Learning Representations (ICLR)*. 2023.

[68] A. Ceni, A. Gravina, C. Gallicchio, et al. "Message-Passing State-Space Models: Improving Graph Learning with Modern Sequence Modeling". In: *arXiv preprint* (2025). arXiv: 2505.18728.

[69] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein. "Temporal Graph Networks for Deep Learning on Dynamic Graphs". In: *arXiv preprint* (2020). arXiv: 2006.10637.

[70] S. Kumar, X. Zhang, and J. Leskovec. "Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. 2019, pp. 1269–1278.

[71] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha. "DyRep: Learning Representations over Dynamic Graphs". In: *International Conference on Learning Representations (ICLR)*. 2019.

[72] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan. "Inductive Representation Learning on Temporal Graphs". In: *International Conference on Learning Representations (ICLR)*. 2020.

[73] A. Gravina, G. Lovisotto, C. Gallicchio, D. Bacciu, and C. Grohnfeldt. "Long Range Propagation on Continuous-Time Dynamic Graphs". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. 2024.

[74] O. Feldman, K. S. I. Mantri, M. Eliasof, and C. Baskin. "Revisiting Node Affinity Prediction in Temporal Graphs". In: *International Conference on Learning Representations (ICLR)*. 2026.

[75] K. Martin, S. Heilig, M. F. C. Haddad, et al. "A Critical Audit of Spatiotemporal Forecasting Benchmark Datasets and Models". In: *arXiv preprint* (2026). arXiv: 2608.20980.

[76] A. Cini, D. Zambon, and C. Alippi. "Sparse Graph Learning from Spatiotemporal Time Series". In: *Journal of Machine Learning Research* 24.242 (2023), pp. 1–36.

[77] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[78] J. H. Friedman. "Greedy Function Approximation: A Gradient Boosting Machine". In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[79] T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.

[80] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems 30*. 2017, pp. 3146–3154.

[81] S. Makridakis, E. Spiliotis, and V. Assimakopoulos. "M5 accuracy competition: Results, findings, and conclusions". In: *International Journal of Forecasting* 38.4 (2022), pp. 1346–1364.

[82] D. Holzmüller, L. Grinsztajn, and I. Steinwart. "Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data". In: *Advances in Neural Information Processing Systems 37*. 2024.

[83] Y. Gorishniy, I. Rubachev, and A. Babenko. "On Embeddings for Numerical Features in Tabular Deep Learning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2203.05556. 2022.

[84] Y. Gorishniy, A. Kotelnikov, and A. Babenko. "TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling". In: *International Conference on Learning Representations (ICLR)*. 2025.

[85] D. Jiang et al. *OmniTabBench: Mapping the Empirical Frontiers of GBDTs, Neural Networks, and Foundation Models for Tabular Data at Scale*. 2026. arXiv: 2604.06814.

[86] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann. "The Balanced Accuracy and Its Posterior Distribution". In: *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)*. 2010, pp. 3121–3124.

[87] H. R. Künsch. "The Jackknife and the Bootstrap for General Stationary Observations". In: *The Annals of Statistics* 17.3 (1989), pp. 1217–1241.

[88] M. Gozo. *Polymarket Tick-Level Orderbook Dataset*. Version 3 of 27 March 2026, downloaded in May 2026; licence CC BY-NC 4.0. Kaggle. 2026. URL: https://www.kaggle.com/datasets/marvingozo/polymarket-tick-level-orderbook-dataset (visited on 05/19/2026).

[89] martkir. *poly-trade-scan: CLI for streaming Polymarket trades in real time and downloading full historical trade data*. GitHub repository, version 0.1.0; modified by the author of this thesis. 2026. URL: https://github.com/martkir/poly-trade-scan (visited on 10/28/2026).

[90] M. Raasveldt and H. Mühleisen. "DuckDB: an Embeddable Analytical Database". In: *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*. 2019, pp. 1981–1984.

[91] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019, pp. 3982–3992.

[92] D. T. Tran, J. Kanniainen, and A. Iosifidis. "How Informative is the Order Book Beyond the Best Levels? Machine Learning Perspective". In: *NeurIPS 2021 Workshop on Machine Learning meets Econometrics (MLECON)*. 2021. arXiv: 2203.07922.

[93] N. A. Le. *Domain-Specific Calibration Dynamics in Prediction Markets*. 2026. arXiv: 2602.19520 [stat.AP].

[94] L. McInnes, J. Healy, and J. Melville. "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction". In: *arXiv preprint arXiv:1802.03426* (2018). [U] bibliographic data from memory, to be checked.

[95] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011). [U] bibliographic data from memory, to be checked, pp. 2825–2830.

[96] Z. Zhang and S. Zohren. "Multi-horizon forecasting for limit order books: novel deep learning approaches and hardware acceleration using intelligent processing units". In: *arXiv preprint* (2021). arXiv: 2105.10430.

[97] B. Lakshminarayanan, A. Pritzel, and C. Blundell. "Simple and scalable predictive uncertainty estimation using deep ensembles". In: *Advances in Neural Information Processing Systems 30*. 2017, pp. 6402–6413. arXiv: 1612.01474.

[98] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.